THÈSE DE DOCTORAT

Programmation logique pour les preuves interactives : inférence des classes de type et analyse du déterminisme

Davide FISSORE

Centre Inria d'Université Côte d'Azur, équipe STAMP

Présentée en vue de l'obtention du grade de docteur en Informatique **d'**Université Côte d'Azur

Dirigée par : Yves BERTOT, Directeur de recherche, Centre Inria d'Université Côte d'Azur

Soutenue le : 16 septembre 2026

Devant le jury, composé de :

Enrico TASSI, Chargé de recherche, Centre Inria d'Université Côte d'Azur

Catherine DUBOIS, Professeur, ENSIIE

Matthieu SOZEAU, Chargé de recherche, Université de Nantes

Chantal KELLER, Maîtresse de Conférences, Université Paris-Saclay

Assia MAHBOUBI, Directrice de recherche, Université de Nantes

Programmation logique pour les preuves interactives : inférence des classes de type et analyse du déterminisme

---

Logic programming for interactive proofs: Type Classes inference and Determinacy Analysis.

Davide Fissore

Jury :

Rapporteurs

Catherine Dubois, Professeur, ENSIIE
Matthieu Sozeau, Chargé de recherche, Université de Nantes

Examinateurs

Enrico Tassi, Chargé de recherche, Centre Inria d'Université Côte d'Azur
Chantal Keller, Maîtresse de Conférences, Université Paris-Saclay
Assia Mahboubi, Directrice de recherche, Université de Nantes

Directeur de thèse

Yves Bertot, Directeur de recherche, Centre Inria d'Université Côte d'Azur

---

Université Côte d'Azur

Davide Fissore

Programmation logique pour les preuves interactives :inférence des classes de type et analyse du déterminisme

xi+138 p.

Ce document a été préparé avec LaTeX2e et la classe `these-ISSS` version v. 2.10.

Impression : main.tex – 6/8/2026 – 7:39

Révision pour la classe : these-ISSS.cls,v 2.10 2020/06/24 14:16:37 mpelleau

Programmation logique pour les preuves interactives : inférence des classes de type et analyse du déterminisme

Résumé

La résolution de classes de types est un mécanisme central de l'élaboration dans les assistants de preuve modernes, assurant l'inférence d'arguments implicites et la désambiguïsation des symboles surchargés. Sa proximité avec la programmation logique suggère des implémentations alternatives fondées sur l'exécution de programmes logiques.

Cette thèse propose une implémentation d'un solveur de classes de types pour Rocq basé sur ELPI, un interpréteur Lambda-Prolog emarqué. Nous introduisons une compilation des classes de types et instances vers une base de connaissances logique, permettant de déléguer la résolution à ELPI. Nous analysons l'expressivité de cette approche et ses performances en la comparant au solveur natif de Rocq.

Un défi majeur provient des divergences entre Rocq et ELPI en matière d'unification et de représentation des termes. Bien que les deux systèmes supportent l'unification d'ordre supérieur et la $\beta\eta$-conversion, leurs encodages diffèrent, ce qui peut empêcher la préservation de l'unification après traduction. Pour remédier à ce problème, nous développons une technique de compilation fondée sur des contraintes différées : les sous-termes problématiques sont remplacés par des liens enregistrant des contraintes qui seront vérifiées au cours de l'exécution, ce qui permet de retrouver le comportement d'unification attendu.

Nous étudions également la prédictibilité des programmes logiques au moyen d'une analyse de déterminisme pour ELPI. Celle-ci garantit qu'un prédicat donné produit au plus une solution, en supportant prédicats d'ordre supérieur, clauses dynamiques et cut. Cette analyse est particulièrement pertinente dans le contexte de la résolution de classes de types, où un comportement déterministe est généralement attendu. Enfin, nous formalisons la correction de cette analyse dans Rocq via un modèle d'ordre un de l'interpréteur ELPI et une preuve de sûreté du vérificateur statique.

Dans l'ensemble, ce travail met en évidence le rôle des techniques de programmation logique dans la mise en œuvre et l'analyse de mécanismes d'élaboration, éclairant les liens entre résolution de classes de types, programmation logique et vérification formelle.

Mots-clés : Programmation logique, Théorie des types, Elpi, Rocq, Classes de type, Analyse de déterminisme.

Logic programming for interactive proofs: Type Classes inference and Determinacy Analysis.

Abstract

Type-class resolution is a central component of elaboration in modern proof assistants, providing implicit argument inference and the disambiguation of overloaded symbols. Its close relationship with logic programming suggests alternative implementations based on the execution of logic programs.

This thesis presents an implementation of a type-class solver for the Rocq proof assistant based on ELPI, an embeddable Lambda-Prolog interpreter. We introduce a compilation scheme that translates type classes and their instances into a logic-programming knowledge base, allowing type-class resolution to be delegated to ELPI. We study the expressiveness of this approach and evaluate its performance by comparing it with Rocq's native solver.

A major challenge arises from the differences between Rocq and ELPI with respect to unification and term representation. Although both systems support higher-order unification and $\beta\eta$-conversion, their term encodings differ, which may prevent unification from being preserved by the translation. To address this issue, we develop a compilation technique based on delayed constraints: problematic subterms are replaced with links recording constraints that are checked during execution, thereby recovering the intended unification behavior.

We also investigate the predictability of logic programs through a determinacy analysis for ELPI. The analysis guarantees that a given predicate produces at most one solution while supporting higher-order predicates, dynamic clauses, and the cut operator. This analysis is particularly relevant to type-class resolution, where deterministic behavior is generally expected. Finally, we formalize the correctness of this analysis in Rocq by mechanizing a first-order model of the ELPI interpreter and proving the soundness of the static checker.

Overall, this work demonstrates the role that logic-programming techniques can play in the implementation and analysis of elaboration mechanisms, shedding new light on the connections between type-class resolution, logic programming, and formal verification.

Keywords: Logic Programming, Type Theory, Elpi, Rocq, Type Classes, Determinacy Analysis.

Acknowledgement*

TODO

*This work has been supported by the French government, through the France 2030 investment plan managed by the Agence Nationale de la Recherche, as part of the "UCA DS4H" project, reference ANR-17-EURE-0004.

Contents

Chapter 1

Introduction

Logical reasoning plays a central role in the way humans justify actions, choices, and conclusions, particularly when decisions are not primarily driven by emotions or other non-rational factors.

For example, if "All men are mortal" and "Socrates is a man", then we can conclude that "Socrates is mortal". This particular form of reasoning, known as an Aristotelian syllogism, is irrefutable: the conclusion follows directly from the premises.

A counterpart to these philosophical ideas can be found in mathematical reasoning. Much like a syllogism, the validity of a mathematical statement is established through a proof constructed from a collection of axioms and inference rules.

In his 1879 work Begriffsschrift ("concept writing" in English), Gottlob Frege attempted to formalize a complete logical foundation for mathematics based on a set of axioms.

The axioms underlying a theory must satisfy a fundamental requirement: they should not give rise to paradoxes, otherwise the theory itself becomes meaningless. Frege's logical system was later shown to be inconsistent following Russell's discovery of the paradox that now bears his name. Today, several foundational frameworks coexist, including classical set theory, intuitionistic logic, and various forms of type theory.

In the middle of the twentieth century, with the advent of computer science, Curry and Howard [How80] discovered a deep connection between the representation of propositions and their proofs on the one hand, and typed computation on the other. Under this correspondence, a proof of a proposition can be viewed as a well-typed program, while the proposition itself corresponds to the type of that program.

Using software to construct proofs has increasingly become a viable alternative to pen-and-paper proofs: once a proof is provided to a machine, it becomes the machine's responsibility to verify that the proof guarantees the desired property. Proof assistants are software systems designed to support the construction and verification of formal proofs while providing interactive human-machine collaboration. As a result, one can state with a high degree of confidence that a program satisfies a formally verified specification.

For example, the operator + can be described as a program that takes two integers and computes their sum. Within a proof assistant, it is possible to prove in a single line of code, using induction, that 0 is a neutral element for addition, i.e., $\forall a, a + 0 = a$.

The importance of certified algorithms is particularly evident in safety-critical domains such as medicine and aerospace engineering.

In such settings, the tolerance for implementation errors is extremely low. For instance, when launching a rocket into space, it is unacceptable for the mission to fail because of a

software bug. A well-known example is the maiden flight of the Ariane 5 rocket in 1996, which ended in failure due to a software error.

Beyond checking proofs, proof assistants also help users write proofs by automating parts of the proving process.

In general, formal proofs developed in a proof assistant must be extremely detailed and are often difficult for humans to read. To reduce the effort required during proof development, modern proof assistants, such as the ROCQ [BC04] proof assistant (formerly known as COQ), provide various forms of automation that allow users to write incomplete proof terms, leaving some information to be inferred automatically by the system.

These mechanisms are collectively referred to as the *elaborator* [BM26, Chap. 6]. The purpose of elaboration is to bridge the gap between the notation written by users and the internal representation manipulated by the proof assistant. Just as a computer does not directly understand ordinary text, the elaborator translates user-written terms into a machine-interpretable form. Moreover, elaboration allows incomplete proof terms to be submitted and automatically completed by the system. In some situations, ambiguities may even be left intentionally in the source code, with the elaborator responsible for resolving them using information available in the current environment.

A common source of ambiguity arises from symbol overloading. In mathematics, it is customary to reuse the same notation in different contexts. For example, the symbol $+$ may denote addition over natural numbers in the expression $1 + 2$, or vector addition in the expression $[\texttt{pi}, e] + [\sqrt{2}, \frac{1}{3}]$. A human reader can usually infer the intended meaning from the surrounding context.

A similar ambiguity exists in programming languages. In object-oriented languages, a common interface may admit multiple implementations, and the compiler is responsible for selecting the appropriate one. Functional programming languages vary in the support they provide for overloading. For instance, in OCAML, different operators are typically required for different notions of addition, whereas HASKELL supports overloading through its type-class mechanism [WB89]. Since many proof assistants are based on functional programming languages, they often provide a type-class system as well. In ROCQ, for example, type classes were introduced in 2008 [SO08].

The role of the type-class solver is to determine which implementation should be used in a given context. Conceptually, the expression $1 +_? 2$ can be viewed as containing a missing piece of information, represented here by the placeholder $?$: the implementation of addition appropriate for the current types. The task of the type-class solver is to infer this implementation automatically.

As observed in [BM26, Chap. 6], elaboration mechanisms are often based on inference rules reminiscent of logic programming. In this work, we investigate an alternative implementation of type-class resolution based on a logic programming language. This approach offers an immediate advantage: the possibility of reusing existing logic-programming techniques developed by the logic-programming community, as well as existing inference engines, rather than reimplementing the resolution procedure from scratch. At the same time, it raises several questions. How naturally can ROCQ's inference problems be translated into a logic-programming setting? What are the performance implications of such a translation? More generally, what are the benefits and limitations of this design choice?

To explore these questions, we rely on ELPI [DGSCT15], the *Embeddable Lambda-Prolog Interpreter*. On the one hand, ELPI is an implementation of λ-PROLOG [NM88];

on the other hand, it integrates easily with a host application such as Rocq. The Rocq-Elpi plugin [Tas18] provides such an integration between Rocq and Elpi.

This setup gives rise to a form of metaprogramming in which Rocq serves as the object language and delegates part of its computation to Elpi, the meta-language. The key idea is that the object language is embedded within the meta-language, meaning that terms of the former are translated into terms of the latter. In our encoding, terms are represented using a shallow embedding, allowing users to inspect and manipulate them directly. Furthermore, a collection of APIs for the object language is exposed, enabling the meta-language to leverage some of the object language's functionalities.

In this work, we study the integration of Rocq with an alternative type-class solver implemented in Elpi. This requires translating type classes and instances from Rocq into an Elpi database, which can be viewed as a compilation step. We show in section 3.2 that a basic compiler can be written in only a few lines while relying on the existing Elpi interpreter for resolution. We then extend this translation to support more complex instances and compare the resulting solver with Rocq's native solver (sections 3.3 to 3.5).

A central challenge in this approach arises from the mismatch between the object language and the meta-language. Although both support higher-order unification [Mil91] and $\beta\eta$-conversion, the shallow encoding of terms, which underlies our metaprogramming setup, creates terms whose λ-abstractions and applications are represented using rigid constructors rather than the native constructs of the meta-language. As a consequence, additional constraints are introduced during unification.

As a result, terms that are unifiable in Rocq may fail to unify after translation into Elpi. Simply delegating unification to the meta-language is therefore insufficient. This motivates another contribution of this thesis.

To address this unification mismatch, we propose a compilation scheme in chapter 4 that preserves most of the structure of terms while isolating problematic subterms. These subterms are replaced with unification variables only when necessary and recorded to the original term with *links*, representing delayed constraints. These links are checked during execution, allowing us to recover the intended behavior without reimplementing unification.

Beyond type-class resolution, the intrinsic nature of logic programming raises another important concern, especially for users unfamiliar with logic programming. Controlling the backtracking performed by the interpreter is fundamental to managing performance. This motivates a second line of work developed in this thesis: a determinacy analysis for Elpi programs (chapter 5). Determinacy ensures that selected predicates produce at most one solution and do not introduce unexpected choice points for any call. This property helps control the behavior of logic programs. In our setting, type classes correspond to predicates and instances to rules. If a class is declared deterministic, the analysis ensures that the program generated by the instances of this class satisfies the property that any call to the class produces at most one solution.

We design and implement an analysis based on inspecting clauses and tracking argument modes. The algorithm supports most features of Elpi, including higher-order predicates, dynamic clauses, and the cut operator. Handling cut is important, as it can enforce determinism while interacting in subtle ways with logical semantics.

To increase confidence in the static analysis, we formalize the expected properties of the checker in the Rocq proof assistant by encoding the Elpi interpreter (chapter 6). To

simplify this development, we restrict the model to a first-order fragment of the language, where terms do not contain higher-order variables. In this setting, the absence of higher-order variables simplifies the formalization and allows us to produce a fully mechanized proof.

Although this model does not capture all higher-order features of ELPI, it encompasses the aspects relevant to our analysis and provides a first foundation for establishing machine-checked correctness guarantees for the ELPI language.

1.1 Contributions and structure of the thesis

This thesis is organized as follows. We first present the context of the work (chapter 2). In particular, we describe the type-class resolution mechanism implemented in ROCQ, together with its main options. Then, we introduce the ELPI language and give a formal description of its syntax and semantics. Finally, we present the ROCQ-ELPI plugin, which enables the interaction between ELPI and ROCQ.

This thesis contains five main contributions, organized as follows:

Sections 2.3.1 and 2.3.2 We present a formalization of the ELPI language, covering both its syntax and its semantics. This formalization first appeared in the appendix of [FT26]. We include it in chapter 2, the chapter describing the context of this work, because this placement fits the overall structure of the thesis better.

Chapter 3 We study the implementation of a type-class solver written in ELPI for ROCQ [FT23]. We present the core part of the compiler that translates ROCQ classes and instances into an ELPI database. This database is then used by the interpreter to search for inhabitants of type classes. We also compare our solver with ROCQ's native solver, focusing on performance, modes, and unification issues.

Chapter 4 We analyze unification problems arising from the higher-order abstract syntax used to represent ROCQ terms in ELPI. In ROCQ, terms that are equal up to $\beta\eta$-conversion can unify, while in ELPI such unification may fail, even if its algorithm supports these conversions. Based on our work [FT24], we explain how to address this issue by delaying problematic unifications using a system of links.

Chapter 5 We present a static determinacy checker implemented in ELPI, available since version 3. This tool verifies that predicates declared as deterministic by the user effectively behave deterministically, meaning that they never create additional choice points. This result is based on our work [FT26].

Chapter 6 We mechanize the determinacy checker in the ROCQ proof assistant. The formalization considers a restricted fragment of ELPI, namely its first-order part, where variables only represent data. Within this setting, we prove the main determinacy property: in a program that satisfies the checker, any call to a deterministic predicate produces at most one solution.

Finally, appendix A provides a list of the ELPI built-ins and auxiliary predicates used in the code snippets throughout this thesis.

Chapter 2

Context: languages and tools

At the core of this manuscript, there are two important pieces of software. On the one hand, we consider the Rocq proof assistant; on the other hand, we consider the logic programming language Elpi.

Rocq [BC04] is one of the main proof assistants. It is based on type theory and allows formal reasoning about mathematical statements and program specifications. In this setting, logical propositions are interpreted as types, and proofs are interpreted as programs (or terms) of these types. The underlying type theory of Rocq [CH88] ensures that the coherence between a program and its type is checked, thus providing guarantees of correctness. Proof development in Rocq is carried out using a language of tactics, which allows the user to construct proofs interactively.

Elpi [DGSCT15] is a language from the field of logic programming. In particular, it is a dialect of λ-Prolog [NM88] which features higher-order programming and unification, backtracking control through the cut operator, unification control through predicates modes. A plugin, called Rocq-Elpi [Tas18], allows one to extend Rocq with new commands and tactics [KvdMT25, vdM24, GLT23].

In Rocq-Elpi:

- *Commands* extend the Rocq vernacular language by introducing new definitions or theorems that are generated by Elpi programs. Examples of libraries that intensively use this feature include Derive [Tas19], which produces equality theorems for inductive types, and Hierarchy Builder [CST20], which helps define complex hierarchies of algebraic structures. Hierarchy Builder supports the Mathematical Components [MT22] ecosystem and mechanizations such as the Odd Order and Four Color theorems [GAA+13, Gon23].

- *Tactics* extend the proof system of Rocq by providing additional forms of automation during proof development. For example, the Trocq [BCC+23, CCM24] library provides a tactic for proof transfer that both translates a user's goal into a related variant and constructs a proof of the corresponding implication. In contrast, the Algebra Tactics [Sak22] library offers tactics for reasoning about polynomial and rational equations.

In the following sections, we introduce the key concepts that form the foundation of chapters 3 to 6. In particular, in section 2.1, we briefly introduce the notion of parametric

polymorphism (also called system F [Gir86]), so that we can provide a very simple intuition about the role of the elaboration. Elaboration is treated in section 2.2, where we focus on type-classes, a central component of the elaborator for symbol overloading. In section 2.3, we present the core aspects of the ELPI language, including its syntax and semantics, which are necessary to understand its execution model. Finally, in section 2.4, we describe the main ideas behind the ROCQ-ELPI plugin.

2.1 Parametric polymorphism: first steps into elaboration

As anticipated, the ROCQ language is based on type theory [CH88]. At its core lies the pure λ-calculus, introduced by Church in 1932 [Chu32]. The λ-calculus is language whose syntax is defined by *terms*:

$$t := x \mid t\ t \mid \lambda x.t$$

A term can be a variable x, an application $t\ t$, or an abstraction $\lambda x.t$. A *subterm* is any term occurring inside another term. In an application, the first (sub)term is called the *head* and the second the *argument*. An abstraction $\lambda x.t$ binds the variable x in its abstraction body t, we say that x is a *bound* variable in $\lambda x.t$. A variable that occurs in an expression but is not bound by any enclosing lambda abstraction is called *free*.

For instance, the term $\lambda x.\lambda y.x$ (often called `fst`) contains two bound variables, x and y. The term $\lambda y.x$ is a subterm of `fst`, and x is free in it.

Evaluation of λ-terms is defined by *reduction* rules. The main rules used here are:

$$\frac{}{(\lambda x.t_1)\ t_2 \to_\beta t_1[x \backslash t_2]}\ \beta\text{-rule} \qquad \frac{x \text{ is not free in } t}{\lambda x.t\ x \to_\eta t}\ \eta\text{-rule}$$

The β-rule describes function application: when the head is an abstraction, the argument replaces all free occurrences of x in t_1. A term where this rule applies is called a β-redex. A term with no such reductions is in β-normal form. For example, $((\lambda x.\lambda y.x)\ 1)\ 2$ reduces first to $(\lambda y.1)\ 2$, and then to 1.

The η-rule expresses a form of extensionality: if x does not occur free in t, then $\lambda x.t\ x$ is equivalent to t. Terms where this rule applies are called η-redexes.

Two terms are considered equivalent if they reduce to the same term.

In simple type theory, the pure λ-calculus is extended with typing rules. These constrain how terms can be formed by assigning a *type* to each term. This extension is known as the simply-typed λ-calculus [Chu40]:

$$\tau := \alpha \mid \tau \to \tau$$

Types are either base types α or function types $\tau \to \tau$, where the first component is the domain and the second the codomain. Term abstractions are annotated accordingly:

$$t := x \mid t\ t \mid \lambda(x : \tau).t$$

A *typing environment* Γ maps variables to types. We write $\Gamma + (x : \tau)$ for the extension of Γ with a new binding. The typing rules are:

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}\ T_x \qquad \frac{\Gamma \vdash t_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash t_2 : \tau_1}{\Gamma \vdash t_1\ t_2 : \tau_2}\ T_@ \qquad \frac{\Gamma + (x : \tau) \vdash t : \tau'}{\Gamma \vdash \lambda(x : \tau).t : \tau \to \tau'}\ T_\lambda$$

The judgment $\Gamma \vdash t : \tau$ means that t has type τ under Γ. If such a derivation exists, the term is said to *typecheck*. The algorithm implementing these rules is called the *typechecker*.

However, the simply-typed λ-calculus can be too restrictive for expressing more abstract constructions. Several extensions exist, as illustrated by the Barendregt cube [Bar91]. One important extension is the polymorphic λ-calculus (System F) [Gir86], where terms may depend on types.

Consider the identity function $\lambda x.x$. In the simply-typed setting, it must be assigned a fixed type, for example $\lambda(x : \texttt{bool}).x : \texttt{bool} \to \texttt{bool}$. A separate definition would be needed for each type, such as `nat`. This duplication can be avoided in the polymorphic calculus.

Polymorphism allows quantification over types. The syntax is extended as follows:

$$t := \ldots \mid \tau \qquad \tau := \ldots \mid \forall(x : \square).\tau$$

The constructor $\forall$ expresses that a type may depend on another type. The $\forall(x : \square), \tau$ constructor is a generalization of $\square \to \tau$, in particular the two representation are equivalent if x does not appear as a free variable in τ.

At the term level, we write $\lambda(x : \square).t$ to abstract over a type. With this, the polymorphic identity function can be written as $\lambda(T : \square).\lambda(y : T).y$.

The typing system is extended with:

$$\frac{\Gamma + (T : \square) \vdash t : \tau}{\Gamma \vdash \lambda(T : \square).t : \forall(T : \square).\tau}\ T_\forall$$

Using this definition, the identity function can be applied as $(\lambda(T : \square).\lambda(y : T).y)\ \texttt{nat}\ 3$, which reduces to 3 by successive β-steps.

In practice, explicitly passing types can be inconvenient. If the typing environment already assigns 3 the type `nat`, the system can infer that the first argument should be `nat`. One may then write $(\lambda(T : \square).\lambda(y : T).y)\ \_\ 3$, where the underscore denotes missing information to be inferred.

Such placeholders, often called *holes*, correspond to existentially quantified metavariables that the typechecker must instantiate. In systems like Rocq, these arguments are often omitted.

This illustrates how type inference improves usability, allowing the user to leave parts of a term unspecified when the system is able to reconstruct them automatically.

2.2 Elaboration in the Rocq proof assistant

The possibility of writing terms with holes, as indicated in the previous section, is just one of the functionalities supported by the elaborator [BM26, Chap. 6]. In this section, we study more deeply this technical component that mediates the interaction between the user and the Rocq system. This component is present in all proof assistants. Its role is to accept expressions that may be incomplete or ambiguous, and to reconstruct the missing information automatically, simplifying therefore the exchanges with the user.

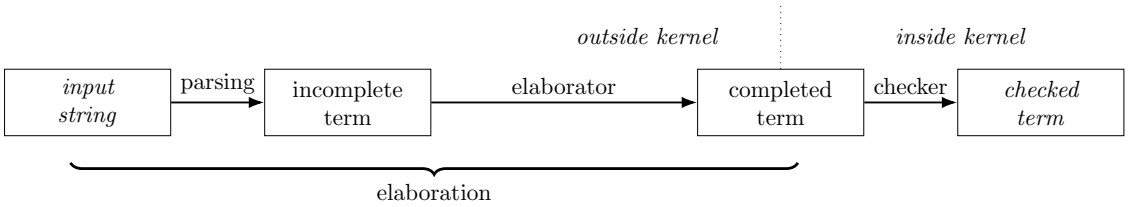


Figure 2.1: Elaboration in Rocq

Figure 2.1, adapted from [BM26, Chap. 6], outlines the main steps of this process. Starting from a raw input string, the system first parses it into a Rocq term, which can still contain implicit parts. This term is then elaborated into a fully explicit one. The resulting term is passed to the kernel, which typechecks it. If the check succeeds, the term is a valid inhabitant of its type, that is, a proof of the corresponding statement.

Elaboration performs several tasks, including notation resolution, type inference, resolution of overloaded symbols...

Notation resolution removes syntactic ambiguity by replacing user-defined notations with their underlying constructors in the abstract syntax tree. The obtained term is unambiguous, although it may still contain holes. For example, the term `[1, 0]` uses common mathematical notation for lists and natural numbers. After this step, it becomes:

```
cons _ (S O) (cons _ O (nil _))
```

Here, `cons` and `nil` are constructors for lists, representing respectively insertion at the head and the empty list. The symbols `S` and `O` denote the successor function and the constant 0.

Type inference fills in the missing information using the typing environment. It relies on a set of rules that propagate type constraints through the term.

As an example, consider the following typing environment:

Symbol		Type
`cons`	:	`∀ T, T → list T → list T`
`nil`	:	`∀ T, list T`
`S`	:	`nat → nat`
`O`	:	`nat`

Running the typechecker on the previous term produces the fully elaborated version:

```
cons nat (S O) (cons nat O (nil nat))
```

Finally, elaboration also supports overloading, allowing a symbol to be used with different types. Since ad-hoc polymorphism is important for this work, we study this mechanism in more detail in the next subsection.

2.2.1 Ad-hoc polymorphism

Polymorphism is a central feature of functional programming languages. In his lecture notes, Strachey [Str00] distinguishes two main forms: parametric polymorphism and ad-hoc polymorphism. Parametric polymorphic functions are *uniform*, meaning that their

behavior does not depend on the specific type at which they are instantiated. For example, if we take the function `id` from section 2.1, we can note that when applied to either `1` or `["foo","bar"]`, it always return its input, that is, the same implementation is reused for every instantiation of its type parameter.

Although parametric polymorphism is widely used, it is not well suited to modeling overloading as it appears in mathematics. For example, in mathematical notation, the meaning of the symbol + depends on the context in which it is used. An informed reader can easily infer that, in the expression $1 + 2$, the symbol + denotes addition on integers, whereas in `pi` $+ e$ it denotes addition on real numbers.

More generally, reusing the same symbol in different contexts makes it possible to capture the common properties that are typically associated with the corresponding operation. For instance, the operator + is generally assumed to be commutative and to admit a neutral element, conventionally denoted by 0.

In contrast, parametric polymorphism does not support multiple type-dependent implementations of the same symbol. This limitation appears in languages such as OCaml, where integer and floating-point additions are represented by different operators, + and +.. One writes $1 + 2$ for integers and π +. e for floating-point numbers. In general, a new symbol must be introduced for each type implementing the same mathematical operation.

To address this issue, Wadler and Blott introduced type classes in 1989 [WB89]. The idea was later implemented in Haskell [HHPJW96], where it became a core feature. Similar mechanisms were then adopted in other languages such as C++ [BJZ+08], Java [OMO10], and proof assistants like Rocq [SO08] and Lean [dMU21].

Symbol overloading Type classes provide a principled form of ad-hoc polymorphism. They allow a single symbol to be associated with different implementations while preserving static type safety. A type class is parameterized by one or more types t and declares a set of functions, called methods, whose types may depend on t. An instance of a type class gives a concrete implementation for a specific type.

Unlike parametric polymorphism, where there exists only one function implementing each symbol, type classes rely on instance resolution. Instances are stored in a database. When a method is used, the elaborator selects the appropriate implementation by searching for an instance matching the required type.

As an example, consider the type class `Add`, parameterized by a type `T`, with a method `plus` for addition. In Rocq, it is defined as follows.

```
Class Add T := {plus: T → T → T}.
```

The method `plus` has the following implementation `fun T (f : Add T) => f.(plus)` where `f.(plus)` is the syntax for retrieve the method plus inside the instance `f`. The type of `plus` is $\forall\{\texttt{T}\}$, $\{\texttt{Add T}\} \to \texttt{T} \to \texttt{T} \to \texttt{T}$, where the type parameters surrounded by curly brackets are implicit. Typechecking the term `plus 3 4` gives:

```
plus nat ?a 3 4 : nat where ?a : [ |- Add nat]
```

The typechecker successfully infers that the expression has type `nat`, but leaves a typeclass constraint unsolved. Specifically, it cannot infer an instance of `Add nat`, since no such instance has been registered. Consequently, the meta-variable `?a`, whose type is constrained to be `Add nat`, remains unresolved.

```
module type Add =
sig
  type t
  val plus : t → t → t
end

module AddNat : Add with type t = int =
struct
  type t = int
  let plus = Int.add
end

module AddProd (T1 : Add) (T2 : Add) :
  Add with type t = T1.t * T2.t =
struct
  type t = T1.t * T2.t
  let plus (a, b) (c, d) =
    T1.plus a c, T2.plus b d
end
```

Figure 2.2: The `Add` type-class with modules in OCAML

Using the function `Nat.add`, of type `nat → nat → nat` computing the sum between natural numbers, we can define:

```
Instance addNat : Add nat := {plus := Nat.add}.
```

The declaration `addNat : Add nat` provides an implementation of `plus`. After registering it, the term `plus 3 4` is elaborated as: `plus nat addNat 3 4 : nat` which is equivalent by β-reduction to `Nat.add 3 4 : nat`. TODO: N7

Similarly, using `Rplus`, computing addition over `Real` numbers, we define the instance for real numbers:

```
Instance addR : Add R := {plus := Rplus}.
```

The same symbol `plus` can now be used for both `nat` and `R`. Using the notation + for the symbol `plus`, we can write $1+2$ and $\pi+e$, and the system selects the correct instance automatically.

A key feature of type classes is the support for *conditional* instances. For example, the instance for product types is conditional and defined as follows:

```
Instance addProd : ∀ T1 T2, Add T1 → Add T2 → Add (T1 * T2) :=
  {plus '(x1,y1) '(x2, y2) := (plus x1 x2, plus y1 y2)}.
```

This instance defines `Add (T1 * T2)` provided that `Add T1` and `Add T2` are available. Addition on pairs is derived from addition on their components, that is, we can add $(\pi, 3)$ with $(e, 4)$, $(3, (\pi, 4))$ with $(7, (e, 0))$ ...

The type class `Add` can be expressed in a similar way in HASKELL, with minor syntactic differences.

It is useful to compare this with the OCAML module system. Module types in OCAML play a similar role, as they specify an interface implemented by modules. In figure 2.2, the module type `Add` requires a type `t` and a function `plus`. The module `AddNat` implements this interface for integers, while `AddProd` is a functor building an implementation for pairs from two modules.

However, the two approaches differ in practice. In ROCQ, instance resolution automatically selects the correct implementation of `plus`. In OCAML, the programmer must explicitly specify which implementation to use. For instance, the implementation of `plus`

in the module `AddProd` uses `T1.plus` and `T2.plus` to compute its output, instead of calling an overloaded symbol `plus`.

Moreover, using modules requires defining specific instances in advance. To compute the sum between $(1, 2)$ and $(4, 5)$, one must first define a module for pairs of integers:

```
module NN = AddProd (AddNat) (AddNat)
```

Only then the addition can be performed:

```
NN.plus (1,2) (3,4)
```

This becomes cumbersome when many combinations are needed. For example, with-both integers and floating-point numbers, all combinations of pairs require separate modules, that is 4 distinct implementations. Type classes avoid this duplication of code by resolving instances automatically.

Overloading theorem symbols In a proof assistant such as Rocq, the type class mechanism reuses the same intuition behind symbol overloading so that not only function symbols can be overload but also theorem symbols. For instance, one may enrich `Add` with the commutativity specification:

```
Class Add T := { plus: T → T → T; comm: ∀ a b, plus a b = plus b a}.
```

An instance of this refined class must now provide both an implementation of the function `plus` and a proof that `plus` is commutative. Then, if `addNat`, `addR` and `addProd` are implemented according the new definition of `Add`, the `comm` symbol can be used as a proof that addition is commutative over natural and real numbers and pairs.

As an example, the specification:

$$\texttt{plus}\ (3, e)\ (4, \sqrt{2}) = \texttt{plus}\ (4, \sqrt{2})\ (3, e)$$

can be proved by using the overloaded theorem `comm`, and the type-class solver will be charged of finding an instance for the type `nat * R`.

Instance search Type classes are widely used in several major Rocq libraries, including Tlc [Cha21], Stdpp [KJJ+23], and Iris [JKJ+18]. The type-class solver automatically infers instances by searching a database of declared instances. This database is organized as an ordered list.

The position of an instance in this list is determined by its priority. A priority can be assigned explicitly at the end of the declaration giving an integer. When no priority is specified, Rocq computes one automatically by assigning a weight of 1 to each premise that is itself a type-class constraint.

Instances with lower priorities are placed earlier in the list. When two instances have the same priority, their relative order is determined by their declaration order in the source code: the instance declared first is placed before the other.

The search procedure is similar to a depth-first search and closely resembles the Prolog SLD-resolution algorithm [Gal03]. To solve a goal, the type-class solver traverses the instance database in order and attempts to unify the conclusion of each instance with the current goal. An instance whose conclusion successfully unifies with the goal is said to be *applicable* to that goal.

When unification succeeds, the premises of the selected instance become new subgoals. These subgoals are solved recursively using the same procedure. If all premises can be resolved, the instance provides a solution to the original goal. Otherwise, the solver backtracks and continues the search with the next matching instance.

Because instances are explored according to their position in the database, the ordering of the database can have a significant impact on the search process. In particular, it may affect the performance of instance resolution. In certain scenarios, an improperly ordered database may even compromise termination.

As an example, consider the instances defined for the type class `Add`, ordered as follows: `addNat`, `addR`, and `addProd`. Performing the addition `plus` $(3, \texttt{pi})$ $(4, e)$ asks the elaborator to find an implementation for symbol `plus` used on the type `nat * R`. Said in other words, we need to find an instance for following goal `Add (nat * R)`. The type-class solver examines the instances in the order in which the instances are declared. The first two instances are discarded because their conclusions do not unify with the goal. The instance `addProd` can be applied and generates the subgoals `Add nat` and `Add R`. These subgoals are solved by `addNat` and `addR`, respectively, yielding the solution `addProd nat R addNat addR`.

Consider now the same set of instances, but assume that `addProd` has the highest priority. When solving the goal `Add ?X`, where `?X` is an existential variable, `addProd` matches by instantiating `?X` as `?X1 * ?X2`. This produces the subgoal `Add ?X1`. Since `addProd` is considered first, it is used again and instantiates `?X1` as `?X3 * ?X4`, generating a new subgoal `Add ?X3`. The same situation repeats indefinitely, leading to a non-terminating search.

To control the search, several mechanisms are available. One of the most common is the use of *modes*, which restrict the shape of class arguments during resolution. In Rocq, a mode is a list of symbols `+`, `!`, and `-`, indicating, respectively, ground terms (i.e., terms without unification variables), terms with a rigid head symbol (i.e., terms whose head is not a variable), and arguments for which no shape restriction is imposed.

A type class may declare multiple modes. During type class resolution, these modes are checked dynamically against the current goal. Resolution proceeds only if the arguments of the goal satisfy at least one of the declared modes.

As an example, suppose that the class `Add` is declared with the mode `!` on its first argument, and consider the goal `Add (nat * ?X)`. Before searching the instance database, the solver first checks whether the goal satisfies the mode constraints. In this case, the first argument, `nat * ?X`, has the application symbol `*` at its head, which is rigid. Therefore, the mode constraint is satisfied, and the search proceeds.

Among the available instances, the only applicable one is `addProd`, which reduces the original goal to the two subgoals `Add nat` and `Add ?X`. The first subgoal satisfies the mode constraint and is immediately solved by the instance `addNat`.

The second subgoal, however, is `Add ?X`. Its argument consists solely of the unification variable `?X`, whose head is therefore not rigid. Consequently, the mode constraint is violated. Since no declared mode matches the current goal, type class resolution fails at this point, and the original goal cannot be solved. In this example, the infinite search loop that would occur without modes is prevented immediately by the mode check.

Another important aspect is that multiple instances may match the same goal, which requires backtracking. Some libraries, such as Stdpp, exploit this behavior to emulate logic programming. For example, the type class `SetUnfold` is used to rewrite set expressions

into logical statements. An inclusion $X \subseteq Y$ can be transformed into a formula of the form $\forall x, P\ x \to Q\ x$, under the hypothesis that $\forall x, x \in X \leftrightarrow P\ x$ and $\forall x, x \in Y \leftrightarrow Q\ x$. When no specific rule applies, the default instance `set_unfold_default P : SetUnfold P P` leaves the goal unchanged. This default instance acts as a catch-all rule, since it applies to any goal.

While backtracking is useful in such patterns, it can introduce ambiguities and lead to behavior that is difficult to predict in practice.

In Rocq, backtracking for a given type class can be disabled using the flag `Typeclasses Unique Instances`. When enabled, once a solution is found, the solver does not backtrack, assuming that the solution is canonical.

2.3 The Elpi language

Elpi [DGSCT15] stands for *Embeddable Lambda-Prolog Interpreter*. It is a dialect of λ-Prolog [Mil91, MN12] that supports higher-order logic programming and integrates a goal suspension mechanism based on CHR (Constraint Handling Rules [Frü98, GSCT19]).

The term "embeddable" highlights that Elpi is designed to be used as an extension language. Its main application is the Rocq-Elpi plugin (see section 2.4), which enables communication between Elpi as the meta-language and the Rocq proof assistant as the object language.

A comprehensive presentation of the Elpi language and its applications can be found in Tassi's "habilitation à diriger les recherches" [Tas25]. Here we briefly introduce the aspects of Elpi that are relevant to this document.

Sections 2.3.1 and 2.3.2 present the syntax and the operational semantics of Elpi. A first formal description appeared in the appendix of our contribution [FT26], and was later revised in [Tas25]. In this thesis, we follow a slightly different presentation. The goal is to be consistent with the conventions used in chapters 5 and 6. We leave aside the syntax and semantics of the CHR component of Elpi. This part is discussed only informally in section 2.3.5.

In sections 2.3.3 and 2.3.4, we first present an example of execution, highlighting the role of the cut operator in the Elpi language.

Finally, in sections 2.3.5 and 2.3.6, we outline two features of the Elpi language: the constraint handling rules system, and rule naming and grafting.

2.3.1 Abstract syntax of Elpi (without CHR)

Syntax of Elpi programs Figure 2.3 presents the abstract syntax of Elpi programs.

The basic syntactic categories are data $\mathbb{K}$, predicates $\mathbb{P}$, unification variables $\mathbb{U}$, and nominal variables $\mathbb{N}$. Nominal variables represent binders introduced either by λ-abstractions or by `pi`-quantification.

A term $\mathbb{Tm}$ is either a basic construct or an application. More precisely, terms are formed by applying a term to a λ-term $\mathbb{L}$. Lambda terms introduce abstractions; in the concrete Elpi syntax, an abstraction is written `x\ t`. Function application follows a curried style. In contrast with Prolog, arguments are not passed within parentheses, which allows partial application. We write, for example, `p 1 X`, instead of `p(1, X)`.

An formula $\mathbb{A}$ represents an executable unit. It can be:

Program	$\mathbb{K}$	$::= f, g, \ldots$	datum
	$\mathbb{P}$	$::= p, q, \ldots$	predicate
	$\mathbb{U}$	$::= X, Y, \ldots$	unification variable
	$\mathbb{N}$	$::= x, y, \ldots$	name
	$\mathbb{Tm}$	$::= \mathbb{K} \mid \mathbb{P} \mid \mathbb{U} \mid \mathbb{N} \mid \mathbb{Tm}\ \mathbb{L}$	term
	$\mathbb{L}$	$::= \lambda\mathbb{N}.\mathbb{L} \mid \mathbb{Tm}$	λ–term
	$\mathbb{A}$	$::= \text{cut} \mid \mathbb{Tm} \mid \texttt{pi}\ \mathbb{N}.\ \mathbb{A} \mid \mathbb{R} \Rightarrow \mathbb{A}$	formula
	$\mathbb{R}$	$::= \mathbb{Tm}\ \texttt{:-}\ \mathbb{A}^*$	rule
Types	$\mathbb{K}_\alpha$	$::= \texttt{kind}\ \mathbb{K}\ \mathbb{T}_\alpha$	type declaration
	$\mathbb{T}_k$	$::= true, false, nil, cons, \ldots$	constructornames
	$\mathbb{T}_\alpha$	$::= \mathbb{Ty} \mid \mathbb{T}_\alpha \to \mathbb{T}_\alpha$	type arity
	$\mathbb{K}_s$	$::= \texttt{type}\ \mathbb{K}\ \mathbb{Ty} \mid \texttt{type}\ \mathbb{P}\ \mathbb{Ty}$	constructor declaration
	$\mathbb{T}_b$	$::= \mathbb{T}_k \mid \mathbb{U} \mid \mathbb{T}_b\ \mathbb{Ty}$	type
	$\mathbb{Ty}$	$::= \mathbb{T}_\alpha \overset{?\mathbf{m}}{\to} \mathbb{T}_\alpha \mid \mathbb{T}_b$	λ–type
	$\mathbf{m}$	$::= input \mid output$	mode

Figure 2.3: Abstract syntax of ELPI programs

1. the cut operator (`!` in the concrete syntax),
2. a predicate call (called atom in the literature),
3. a `pi`-quantification, written `pi x\ t` in the concrete syntax, which introduces a fresh nominal variable `x` visible in `t`, or
4. the dynamic insertion of a rule, written `r => t`, which makes the rule `r` locally available in the scope of `t`.

A rule consists of a head and a (possibly empty) body, which is a list of atoms. The atoms in the body are called the *premises* of the rule.

For simplicity, we omit constraints from the syntax, since they do not play an immediate role in the presentation. They are briefly described in section 2.3.5.

In this abstract syntax, logical connectives such as negation and disjunction, which are usually encoded as native symbols of a logic programming-language, can be treated as first-class objects and implemented directly as ELPI rules.

Formula position in implication We say that a subformula occurs either in a *positive* or *negative* position within a formula [MN12].
A formula at the top level is in positive position. For an implication $f_1 \Rightarrow f_2$, the position tag is reversed on the left-hand side and preserved on the right-hand side. More precisely, if the whole implication is in positive (resp. negative) position, then f_1 is in negative (resp. positive) position, while f_2 is in positive (resp. negative) position.
In the example on the right we explicitly annotate the positive/negative occurrences of each subformula.

$$\underbrace{\underbrace{(\underbrace{x}_{pos} \Rightarrow \underbrace{y}_{neg})}_{neg} \Rightarrow \underbrace{z}_{pos}}_{pos}$$

This notion of positive/negative position of a term will be used again in section 3.4 when discussing the encoding of instances.

Syntax of Elpi types In Elpi, users can define the algebraic data types required by their development as well as the signatures of predicates. A new algebraic data type is declared with the `kind` keyword, while constructors of that type are introduced using the `type` keyword. A type is either a base type, a variable, or the application of a type to another in the case of parametric types. The arrow symbol is used to type function spaces and in Elpi it takes an optional mode information. The mode, input or output, tells the runtime which unification algorithm to use when unifying the arguments of two terms. We delay this discussion in the next section, in particular at definitions 2.3.4 and 2.3.5. By default, no mode information is compiled to output.

For example, the constructors for natural numbers and polymorphic lists, introduced in section 2.2, can be defined in Elpi as follows:

```
kind nat type.
type z nat.
type s nat → nat.
```

```
kind list type → type.
type nil list A.
type cons A → list A → list A.
```

The type `nat` contains the constant `z` representing zero and the constructor `s` representing the successor of another natural number. A `list` is either empty `nil` or the constructor `cons` that combines a head element with a tail list. Lists are polymorphic: the type of the head element must coincide with the type of the elements stored in the tail.

A predicate is a symbol whose type ends in `prop`. The type `o` is an equivalent notation following the convention of [MN12]. Elpi provides a dedicated syntax for declaring predicates, which also allows the programmer to specify the modes of their arguments.

The declaration `pred p` $t_1, \ldots, t_m \to t_{m+1}, \ldots, t_n$, where each t_i $(1 \le i \le n)$ is a type, is equivalent to the declaration `type p` $t_1 \to \cdots \to t_n \to$ `prop`. In addition, it specifies the modes of the predicate's arguments.

Arguments appearing before the arrow in a `pred` declaration are considered input arguments, whereas those appearing after the arrow are considered output arguments.

2.3.2 Operational semantics of Elpi (without CHR)

In this section, we use the notation $\overrightarrow{x}$ to denote an ordered list of elements. The binary operator +, when applied to partial mappings, denotes their disjoint union.

A substitution σ of type Σ is a partial mapping from unification variables $\mathbb{U}$ to terms $\mathbb{Tm}$. The empty substitution is denoted by ε. The notation $\sigma = \{X \mapsto t\}$ represents a substitution σ where the variable X is assigned to the term t. The domain dom σ of a substitution σ is the set of variables that are assigned in σ.

Definition 2.3.1 (Substitution extension)**.** *A substitution σ' is an extension of σ, noted $\sigma \subseteq \sigma'$, iff*

$$\exists \sigma'',\ \sigma + \sigma'' = \sigma' \ \wedge\ dom\ \sigma'' \cap dom\ \sigma = \varnothing$$

Definition 2.3.2 (Substitution application (or term dereferencing))**.** *A substitution σ applied to a term t, noted σt, is the operation that recursively replaces the variables in t by their assignment in σ.*

We use the notation $\mathbb{E}$ to represent a typing environment. It maps symbol names to their type.

A program $\mathcal{P}$ of type $\mathbb{P}$ is made by a triple $(\overrightarrow{\mathbb{R}}, \overrightarrow{\mathbb{N}}, \mathbb{E})$.

$$\frac{}{\text{runE } ((\sigma,\ \varnothing) :: a) \rightsquigarrow \text{Some } (\sigma, a)}\ \text{runE}_\top$$

$$\frac{}{\text{runE } \varnothing \rightsquigarrow \text{None}}\ \text{runE}_\bot$$

$$\frac{\text{runE } ((\sigma,\ g) :: a) \rightsquigarrow r}{\text{runE } ((\sigma,\ (\_,\ \text{cut}, a) :: g) :: \_) \rightsquigarrow r}\ \text{runE}_!$$

$$\frac{\mathcal{B}_E(\mathcal{P}, (\sigma\ t), \sigma, a, g) = a' \qquad \text{runE } (a' +\!\!+ a) \rightsquigarrow r}{\text{runE } ((\sigma,\ (\mathcal{P},\ t, \_) :: g) :: a) \rightsquigarrow r}\ \text{runE}_\mathcal{B}$$

$$\frac{y \# \mathcal{P} \quad \text{runE } ((\sigma,\ (y +_\forall \mathcal{P},\ \text{t[x/y]}, a) :: g) :: a') \rightsquigarrow r}{\text{runE } ((\sigma,\ (\mathcal{P},\ \texttt{pi}\ x.\ t, a) :: g) :: a') \rightsquigarrow r}\ \text{runE}_\forall$$

$$\frac{\text{runE } ((\sigma,\ (h +_\Rightarrow \mathcal{P},\ t, a) :: g) :: a') \rightsquigarrow r}{\text{runE } ((\sigma,\ (\mathcal{P},\ h \Rightarrow t, a) :: g) :: a') \rightsquigarrow r}\ \text{runE}_\Rightarrow$$

Figure 2.4: Operational semantics of ELPI

The first element represents the ordered list of rules characterising the program. We use the notation $\mathcal{P}.\texttt{index}$ to get this element from $\mathcal{P}$. The second element of the triple contains the set of nominal constants that have already been introduced; we use the notation $\mathcal{P}.\texttt{names}$ to get this element for the program $\mathcal{P}$. Finally, the last element of the triple contain the typing environment, we use the notation $\mathcal{P}.\texttt{sig}$ to get this element.

We write $x\#\mathcal{P}$ to denote that the nominal variable x is fresh with respect to $\mathcal{P}.\texttt{names}$, i.e. $x \notin \mathcal{P}.\texttt{names}$. The notation $x +_\forall \mathcal{P}$ denotes the program obtained by adding x to $\mathcal{P}.\texttt{names}$. Similarly, $h +_\Rightarrow \mathcal{P}$ denotes the program obtained by prepending the rule h to $\mathcal{P}.\texttt{index}$.

A goal g of type $\mathcal{G} = \mathbb{P} \times \mathbb{A} \times \overrightarrow{\mathcal{A}}$ is a triple consisting of a program, an atom, and a list of *cut-to alternatives*. An alternative a of type $\mathcal{A}$ is a pair $\Sigma \times \overrightarrow{\mathcal{G}}$ composed of a substitution and a list of goals.

Figure 2.4 presents the derivation rules of the ELPI interpreter, defined in terms of the function

$$\text{runE} : \overrightarrow{\mathcal{A}} \to \text{option } (\Sigma, \overrightarrow{\mathcal{A}}).$$

This semantics is inspired by [DM88, Sec. 4.3] and [Tas25]. Execution is modeled as the evaluation of a stack of alternatives $\mathcal{A}$. The interpreter processes alternatives sequentially. The evaluation of the alternative list either succeeds, producing a substitution and a new list of alternatives, or fails, returning None.

In figure 2.4, rule $\text{runE}_\top$ captures success: if the first alternative contains an empty list of goals, the interpreter returns its associated substitution together with the remaining alternatives. Conversely, rule $\text{runE}_\bot$ states that evaluation fails when the list of alternatives is empty.

The remaining rules of runE, namely $\text{runE}_!$, $\text{runE}_\mathcal{B}$, $\text{runE}_\forall$, and $\text{runE}_\Rightarrow$, apply when the first alternative contains a non-empty list of goals. Such alternative list has the form

$((q :: g) :: a)$, where q is the current query, g the list of goals to be solved after q if q succeeds, and a is the list of alternatives to execute if it the execution of $(q :: g)$ brings no solution. The derivation rule to use depends on the syntactic form of q, which can be one of the four forms of atoms listed in figure 2.3.

If q is a cut, rule runE! discards the current alternatives a and replaces them with the cut-to alternatives stored in q. Evaluation then continues with the remaining goals g.

If q is a `pi`-quantification `pi` $x.t$, the interpreter generates a fresh nominal variable y not occurring in $\mathcal{P}$, replaces all free occurrences of x in t with y, and continues evaluation with the resulting atom.

If q corresponds to the dynamic insertion of a rule h, evaluation continues with atom t in the program extended with the local rule h. Intuitively $h \Rightarrow t$ amounts to a well-bracketed use of Prolog's `assert/1` and `retract/1`.

Finally, if q is a predicate call, the interpreter performs *backchaining* over the rules stored in $\mathcal{P}$.`index`. Backchaining retrieves the rules that can be used on the query q. In standard Prolog this is done by selecting rules whose head unifies with q. In Elpi the process depends on predicate modes, which determine if a argument should be *matched* or *unified*.

Elpi provides two procedures, unify and matching. Both have type

$$\mathbb{Tm} \to \mathbb{Tm} \to \Sigma \to \text{option } \Sigma,$$

taking two terms and an initial substitution σ. They either fail (returning None) or succeed with an updated substitution σ' that is an extension of σ.

The choice of providing two distinct procedures for selecting a rule from the database is mainly motivated by a practical need: Elpi aims to give the user finer control over the rule selection process. In particular, the input mode treats the input arguments as patterns used to identify applicable rules, rather than as terms to be instantiated through unification. Consequently, variables occurring in input positions are not assigned during the matching of a query input argument; instead, any such assignments must be explicitly performed by the rule body. This separation makes the direction of information flow explicit and allows rule selection to be used as a controlled matching mechanism, closer in spirit to the pattern matching found in functional languages, rather than as an implicit source of variable instantiation.

Definition 2.3.3 (Pattern fragment)**.** *A λ-term belongs to Miller's pattern fragment [Mil91] if in every subterm of the form $F\ t_1\ t_2 \ldots t_n$, where F is a higher-order unification variable, each term t_i is a bound variable distinct from any t_j (with $i \neq j$).*

The procedure *unify* implements higher-order unification restricted to the pattern fragment, and is used for arguments in output position. The procedure *matching*, inspired by B-Prolog's "matching clauses" [Zho12], is used for arguments in input position extended to the higher-order variable unification within the pattern fragment.

Definition 2.3.4 (unify)**.** *Two terms q and u* unify *under a substitution σ if there exists a substitution σ' extending σ such that $\sigma' q = \sigma' u$, and σ' is a most general unifier for the two terms.*

Definition 2.3.5 (matching)**.** *A query q* matches *a pattern u under a substitution σ if there exists a substitution σ' extending σ such that $\sigma' u = \sigma q$, where σ' assigns at most the variables occurring in u.*

For example, consider the query `q X` and the rule r whose head is `q 0`. If the signature of `q` is `pred q -> int`, then its argument is an output. Consequently, the procedure invokes `unify X 0` ε, which returns the substitution $\sigma' := \texttt{X} \mapsto 0$. This substitution is valid because it is the most general unifier of the two terms and $\sigma'\ X = \sigma'\ 0$.

On the other hand, if the signature is `pred q int ->`, then the predicate expects an input argument matching the value 0. In this case, the procedure invokes `matching X 0` ε, where the goal is to find a substitution σ' such that $\sigma'\ 0 = \varepsilon\ X$. Dereferencing the first term yields X, while dereferencing the second yields 0 (since 0 contains no variables, dereferencing leaves it unchanged). Because X and 0 are distinct terms, no such substitution exists. Therefore, the query cannot be resolved using the rule r.

The backchaining procedure is defined below by the function $\mathcal{B}$.

Definition 2.3.6 (Backchain, $\mathcal{B} : \mathbb{P} \times \overrightarrow{\mathbb{U}} \times \mathbb{Tm} \times \Sigma \to \overrightarrow{\Sigma \times \mathbb{A}}$)**.**

$$\mathcal{B}(\mathcal{P}, V, q, \sigma) = \left[(\sigma', b) \text{ if } \mathcal{S}(\mathcal{P}.\text{sig}, q) = \text{Some } m \wedge \mathcal{U}(m, q, h, \sigma) = \text{Some } \sigma' \;\middle|\; (h :\!- b) \in \mathit{fresh}(V, \mathcal{P}.\text{index}) \right].$$

Intuitively, $\mathcal{B}$ selects the rules from the program that can be applied to the current query q under a substitution σ. The variables occurring in the program rules are assumed to be disjoint from those in the current execution state. To guarantee this, the program is refreshed using the function `fresh`, which renames its variables with respect to the set V.

For each applicable rule, $\mathcal{B}$ returns the substitution obtained by matching/unifying the rule head with q, together with the corresponding rule body.

The definition relies on two auxiliary functions:

$$\mathcal{S} : \mathbb{E} \to \mathbb{Tm} \to \text{option } \mathbb{Ty}$$
$$\mathcal{U} : \mathbb{Ty} \to \mathbb{Tm} \to \mathbb{Tm} \to \Sigma \to \text{option } \Sigma$$

The function $\mathcal{S}$ retrieves the signature of the predicate occurring in a term. It returns the signature if the predicate is present in the environment, and None otherwise.

The function $\mathcal{U}$ tests whether the query q matches/unifies the head h of a rule under an initial substitution σ. The predicate signature determines the mode of each argument, which in turn selects the appropriate procedure: matching for input arguments and unify for output arguments. If all arguments satisfy the unification condition, $\mathcal{U}$ returns the resulting substitution σ'; otherwise it returns None.

Definition 2.3.7 (ELPI backchain, $\mathcal{B}_E : \mathbb{P} \times \mathbb{Tm} \times \Sigma \times \overrightarrow{\mathcal{A}} \times \overrightarrow{\mathcal{G}} \to \overrightarrow{\mathcal{A}}$)**.**

$$\mathcal{B}_E(\mathcal{P}, t, \sigma, a, g) = \left[\sigma', ([(\mathcal{P}, q, a) \mid q \in b] +\!\!+ g) \;\middle|\; (\sigma', b) \in \mathcal{B}(\mathcal{P}, F_v(t) \cup F_v(g :: a) \cup F_v(\sigma), t, \sigma) \right].$$

The function $\mathcal{B}_E$, used by rule runE$_\mathcal{B}$ in figure 2.4, builds a new list of alternatives from the result of $\mathcal{B}$ (F_v is the function retrieving the set of free variables in the term t, we lift the definition over list of terms, and substitution*). For each filtered rule, the atoms in the rule body become new goals whose cut-to alternatives are a and whose continuation is g. The substitution associated with each alternative is the one returned by $\mathcal{B}$.

Comparison with Tassi's presentation In the previous subsections, we presented the Elpi language, including its syntax and semantics. This formalization is one of our contribution. A comprehensive description of Elpi can be found in Enrico Tassi's work [Tas25].

We highlight here the main differences between his presentation and the one adopted in ours. First, in the syntax, we use binary application for terms, instead of the n-ary application used in [Tas25]. Second, in the semantics, programs carry not only the list of rules and nominal variables, but also the signature of declared predicates. We also omit the formal treatment of constraints, as it is not required for the remainder of this manuscript; a brief informal explanation is sufficient for our purposes (see section 2.3.5).

Another difference concerns the definition of runE. In Tassi's presentation, runE is defined as a procedure that takes two arguments: the current alternative and the list of future alternatives. This choice reflects closely the concrete implementation in Elpi, where execution starts from a query, and a query is represented as an alternative. In contrast, we define runE with a single input, namely a list of alternatives, from which the first element is extracted by pattern matching when needed. This design choice aims to simplify the presentation of chapter 5 and mechanization in Rocq of chapter 6. Despite these differences the two semantics can be proved equivalent when executed on non-empty lists of alternatives, modulo the absence of constraints.

2.3.3 Example of program execution in Elpi with cut

We believe that the main difficulty in the semantics presented in the previous section lies in the treatment of choice points and their interaction with the cut operator.

Several variants of the cut operator exist in the literature. The variant we consider, used in Elpi, is the so-called *hard cut* (see, for example, the documentation of SWI-Prolog [SWI26]). When the `backchain` function returns a non-empty list of alternatives, the first alternative is executed immediately, while the remaining ones are stored as choice points.

Within a selected alternative, premises are evaluated from left to right, each atom being executed in sequence. When a cut is encountered, it removes two kinds of choice points: (i) those generated by `backchain` for the current predicate call, and (ii) those created during the evaluation of premises to the left of the cut.

As a running example, we consider the program shown in figure 2.5. For clarity, figure 2.6 illustrates its execution, explicitly showing the pointers to the cut-to alternatives. We only highlight the alternatives targeted by the cut operator, since the alternatives stored for predicate calls do not play a role in the semantics discussed here.

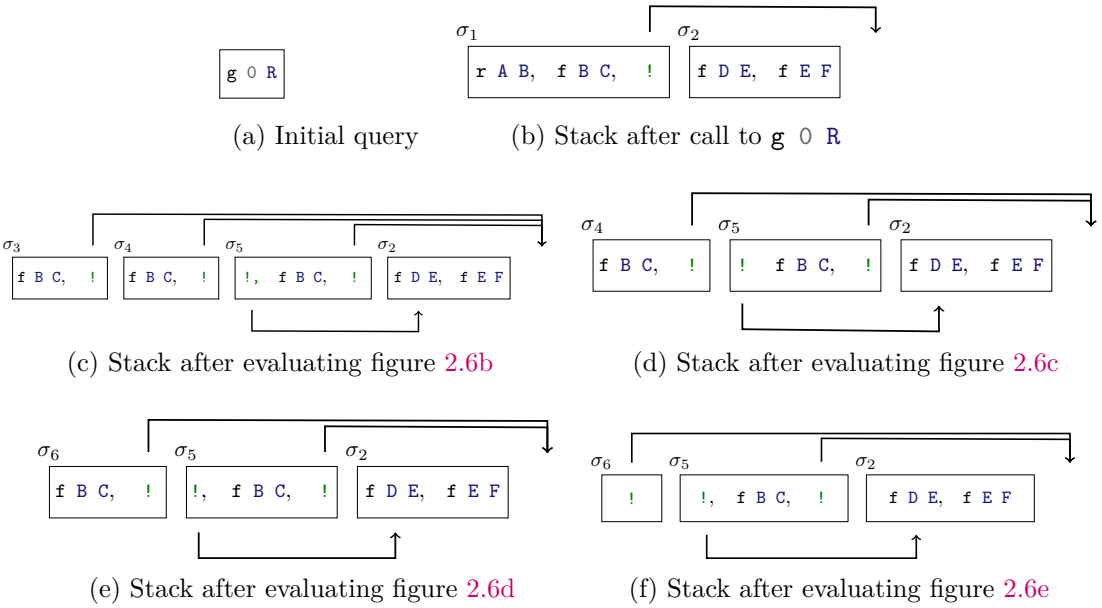
*$F_v(\sigma)$ is made by the domain of σ and the free variables in all terms appearing in the codomain in σ

```
pred f int → int.
f 1 2.
f 2 3.
```

```
pred r int → int.
r 0 0.
r 0 1.
r 0 2 :- !.
```

```
pred g int → int.
g A C :- r A B, f B C, !.
g D F :- f D E, f E F.
```

Figure 2.5: Example of an ELPI program



(a) Initial query

(b) Stack after call to `g 0 R`

(c) Stack after evaluating figure 2.6b

(d) Stack after evaluating figure 2.6c

(e) Stack after evaluating figure 2.6d

(f) Stack after evaluating figure 2.6e

Figure 2.6: Example of execution

Consider the program in figure 2.5 and the query `g 0 R`. Both rules for `g` apply to this query. Backchaining therefore returns the following two alternatives:

$$[(\sigma_1, [\texttt{r A B}, \texttt{f B C}, \texttt{!}]), (\sigma_2, [\texttt{f D E}, \texttt{f E F}])]$$

with $\sigma_1 := \{\texttt{A} \mapsto 0, \texttt{C} \mapsto \texttt{R}\}$ and $\sigma_2 := \{\texttt{D} \mapsto 0, \texttt{F} \mapsto \texttt{R}\}$. For simplicity, we choose an example where variable refreshing plays no role.

Execution continues with the first premise of the first alternative, namely `r A B` under σ_1, i.e. `r 0 B`. The remaining alternatives are stored as choice points. Backchaining `r 0 B` returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [\texttt{!}])]$ where $\sigma_3 := \sigma_1 + \{\texttt{B} \mapsto 0\}$; $\sigma_4 := \sigma_1 + \{\texttt{B} \mapsto 1\}$ and $\sigma_5 := \sigma_1 + \{\texttt{B} \mapsto 2\}$.

The next step of interpretation continues with `f B C` under σ_3, that is, `f 0 R`, and leaves $[(\sigma_4, []), (\sigma_5, [\texttt{!}])]$ as new choice points. This time, backchaining fails (no rule for `f` matches input 0). We therefore backtrack to the most recently introduced choice point and retry `f B C` under σ_4, i.e. `f 1 R`. This succeeds with $\sigma_6 := \sigma_4 + \{\texttt{R} \mapsto 2\}$.

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for `r` and one from the second rule for `g` (respectively, σ_5 and σ_2). Both are discarded, because they were created when, or after, the first rule for `g` was selected. In contrast, a cut does not discard choice points created earlier; in this example, there are none. The final solution is: $\{\texttt{A} \mapsto 0, \texttt{B} \mapsto 1, \texttt{C} \mapsto \texttt{R}, \texttt{R} \mapsto 2\}$.

2.3.4 Higher-order, dynamic and meta programs in Elpi

One of the main features of Elpi is the possibility to work with higher-order and dynamic and meta programs. In this section, we present some simple examples illustrating these two aspects. These examples will be used later as a reference when discussing determinacy checking in Elpi (see chapter 5).

Higher-order programming A well-known higher-order predicate in Elpi is `map`, whose implementation is shown below.

```
pred map (pred A → B), list A → list B.
map _ [] [].
map F [X|XS] [Y|YS] :- F X Y, map F XS YS.
```

The predicate takes a higher-order argument `F`, which is applied to each element of the input list. In the base case, the empty list is mapped to the empty list. In the recursive case, the head `X` is transformed into `Y` using `F`, and the predicate is then applied recursively to the tail `XS`.

Another common higher-order construct in Elpi is the predicate `once`, defined as follows.

```
pred once (pred) →.
once P :- P, !.
```

The predicate `once` acts as a wrapper around a goal. It executes the goal `P` and then removes all remaining choice points. As a consequence, only the first solution is returned. For example, the query `once (r 0 X)`, where `r` is defined in figure 2.5, produces a single result, binding `X` to `0`.

The behavior of the cut operator depends on where it is placed. The following snippets illustrate similar programs with different uses of cut.

```
pred p1 int → int.
p1 1 X :- r 0 X.
p1 1 3.
```

```
pred p2 int → int.
p2 1 X :- r 0 X, !.
p2 1 3.
```

```
pred p3 int → int.
p3 1 X :- once (r 0 X).
p3 1 3.
```

The query `p1 1 X` returns four solutions for `X`, namely `0`, `1`, `2`, and `3`. This is because no cut is used, so all possible solutions are explored by backtracking.

The query `p2 1 X` returns only one solution, `X = 0`. In this case, after the first solution of `r 0 X` is found, the cut removes all choice points, both for `r` and for `p2`.

Finally, the query `p3 1 X` returns two solutions, `0` and `3`. Here, the use of `once` removes the choice points generated by `r`, but it does not affect the second rule for `p3`.

Dynamic clauses The domain where Elpi really shines is the manipulation of higher-order data represented via the so-called higher-order abstract syntax (or λ-tree syntax) [Mil18] also called HOAS [PE88]. Such data is pervasive in interactive provers such as Rocq. In the paradigmatic example below `tm` is the data type for λ-terms.

```
kind tm type.                    % datatype of λ-terms in HOAS form
type app tm → tm → tm.           %  - binary application
type lam (tm → tm) → tm.         %  - λ-abstraction
```

In the HOAS encoding of syntax with binders we do not find a node for variables: we re-use the notion of bound variable of the programming language to encode the one of the syntax tree. At the level of types we see that the `lam` node carries an ELPI function over terms. It is by applying this function that one can recover the body of the λ-abstraction. A simple program manipulating higher-order data is the function performing a deep copy of a term:

```
pred copy tm → tm.
copy (app A B) (app C D) :- copy A C, copy B D.
copy (lam F)   (lam G)   :- pi x\ copy x x => copy (F x) (G x).
```

The copy of an application is built from the copies of the subterms. What is more interesting is how one copies under a binder. The "`pi x\ copy x x =>`" instruction explains how to copy the new constant `x`. Finally the code performs a recursive call over `F x`: by applying the ELPI function `F` to the fresh symbol `x` one obtains the body of the λ-abstraction where the bound variable is replaced by the new symbol `x`.

In ELPI, all predicates are *dynamic*, or open, hence the code of `copy` can be augmented with rules at runtime.

The `copy` predicate, as given, implements the identity and may seem rather useless, but it is not! Since the program is open, one can add new rules for `copy` and change its behavior. For example the code below computes the weak head normal form of a λ-term by contracting the application of a λ-abstraction to an argument and uses `copy` to perform the substitution:

```
pred whd tm → tm.
whd (app H A) R :- whd H (lam F), !,            % we fire the β-redex
  pi x\ copy x A => copy (F x) S, whd S R.      %   and we continue
whd X X.                                        % or we stop
```

The argument `A` is put in place of the (previously) bound variable in `F` by replacing that variable with `x` and by running `copy` with the addition of a special rule copying `x` to `A`.

Meta-programming In ELPI, code is a first-class object, meaning that it can be inspected and generated within the language itself. As an example, consider a program that performs the fusion of two calls to `map` [Wad89].

```
pred comp (pred A → B), (pred B → C), A → C.
comp F G X Z :- F X Y, G Y Z.

pred fuse (pred list A → list B), (pred list B → list C) →
          (pred list A → list C).
fuse (map F) (map G) (map (comp F G)).
```

The predicate `fuse` analyzes the syntax of the two programs. If both correspond to mapping a list, with functions `F` and `G`, it produces a new program of the form `map (comp F G)`, where `comp` denotes function composition.

Meta-programming is widely used in ROCQ-ELPI. It allows the generation of new rules that can be stored on disk and reused in later executions of the program.

2.3.5 CHR: goal suspension

Constraint Handling Rules (CHR, [Frü98, Frü15]) is a declarative, rule-based language designed to manipulate a multiset of constraints. A CHR program consists of rules that match patterns of constraints, optionally guarded by conditions, and transform the current constraint store by adding or removing elements. Operationally, execution proceeds by repeatedly selecting applicable rules and applying them to the constraint store.

From a conceptual point of view, CHR can be seen as a rewriting system over constraints, where rules describe how local configurations evolve. In contrast to standard logic programming rules, which operate on a single goal, CHR rules may inspect and rewrite several constraints at once.

In Elpi, CHR is integrated as an extension of the underlying λ-Prolog engine. The motivation for this integration is the need to combine and refine goals (seen as suspended constraints) that arise during proof search. This model explains how the system manages the constraint store. The main operations are simplification, propagation, and their combination, called simpagation. In the following, the notation $\overrightarrow{h}$, $\overrightarrow{g}$, $\overrightarrow{b}$, and $\overrightarrow{k}$ denotes lists of atoms.

Simplification, written $\overrightarrow{h} \iff \overrightarrow{g} \mid \overrightarrow{b}$, is triggered when the constraints $\overrightarrow{h}$ are present in the store and the guards $\overrightarrow{g}$ hold. In this case, $\overrightarrow{h}$ is removed, and the body $\overrightarrow{b}$ is executed, possibly introducing new constraints.

Propagation, written $\overrightarrow{k} \iff \overrightarrow{g} \mid \overrightarrow{b}$, is triggered under the same conditions, namely when $\overrightarrow{k}$ is in the store and $\overrightarrow{g}$ holds. The body $\overrightarrow{b}$ is then executed, but in contrast to simplification, the constraints $\overrightarrow{k}$ remain in the store.

Simpagation, written $\overrightarrow{k} \setminus \overrightarrow{h} \iff \overrightarrow{g} \mid \overrightarrow{b}$, combines the previous two mechanisms. It is triggered when both $\overrightarrow{k}$ and $\overrightarrow{h}$ are present in the store and the guards $\overrightarrow{g}$ hold. Then $\overrightarrow{h}$ is removed, while $\overrightarrow{k}$ is retained, and the body $\overrightarrow{b}$ is executed.

In Elpi, the concrete syntax for defining rules that handle the constraint store closely follows the mathematical notation above. A constraint is declared using the predicate `declare_constraint`, which takes two arguments: the constraint itself (an atom) and a list of variables on which the constraint is suspended. When a variable is assigned, the associated constraints are resumed. A more complete presentation can be found in [GSCT19, Sec. 4.3] and [Tas25, Sec. 2.4].

2.3.6 Rule names and insertion

In the Elpi database, the rules used during backchaining are stored in an ordered data structure. Each rule can optionally be given a name by the user.

These names are useful when adding new rules that should have higher or lower priority than existing ones. In this way, the user can control the position of rules in the search order.

In Elpi, inserting a rule before or after an existing one is called *grafting*.

```
:name "nth-fail"
nth _ _ _ :- error "List is to short".
```

```
kind term type.

type sort    sort → term.                    % Prop, Type@{i}
type global  gref → term.

type app     list term → term.               % app [hd|args]
type fun     name → term → (term → term) → term.
type prod    name → term → (term → term) → term.

...                      % Other constructors omitted
```

Figure 2.7: HOAS of Rocq terms in Elpi

The definition of `nth` above is a catch-all rule, since it applies to any query. This rule is named `nth-fail`. When defining additional rules for `nth`, one can write:

```
:before "nth-fail"
nth N [] _ :- N < 0, error "Index is negative".
```

This declaration places the new rule just before the catch-all rule, giving a more precise error in the case of negative index. A similar syntax can be used with `:after` to insert a rule after an existing one.

The usefulness of grafting becomes clearer in section 3.4.1, where type-class instances must be inserted at specific positions in the database to respect the intended priority.

2.4 Rocq-Elpi: An Elpi plugin for Rocq

The Rocq-Elpi language is a plugin for Rocq that enables the use of Elpi within the Rocq environment. In the following sections, we first explain the idea behind encoding Rocq terms in Elpi (see section 2.4.1) using higher-order abstract syntax (HOAS). In section 2.4.2, we provide some important API that are widely used in Rocq-Elpi. Finally, in sections 2.4.3 and 2.4.4, we provide a brief description of how database and commands and tactics are respectively written in Rocq-Elpi and how they integrate with Rocq.

2.4.1 HOAS: λ-terms representation

In section 2.3.4, we sketched how to encode the syntax of a language using the HOAS of Elpi. This syntax is inspired by the λ-calculus, which forms the core of the Rocq system.

The encoding of Rocq terms is performed using a Elpi type called `term`. The snippet depicted in figure 2.7 shows the main constructors for terms.

The base constructors are `sort`, which encodes the `Type` and `Prop` terms of Rocq, and `global`. The `global` constructor represents user-defined symbols such as types, inductives, and constants.

These symbols are identified by a `gref`, which is an opaque object containing the unique identifier of the corresponding Rocq symbol. For better space management we use a more compact notation: opaque Rocq symbols are written using guillemets, avoiding the explicit use of the `global` constructor. For instance, the Rocq constant `nat` is represented in HOAS form as `«nat»`.

Rocq terms	Elpi HOAS	description
`plus 0`	`app [«plus», «0»]`	term application
`fun x: int => ?F a`	`fun `x` «int» y\ app [F, «a»]`	λ-abstraction
`∀ x: bool, p x`	`prod `x` «bool» z\ app [«p», z]`	∀-quantif and binding

Table 2.1: Examples of Rocq terms encoded in Elpi using HOAS

Term application is represented by the constructor `app`, which takes a list of terms as its argument. A natural HOAS encoding of a Rocq application is a term of the form `app L`, where `L` is a list of length at least two: its first element is the head of the application, and the remaining elements are its arguments.

When translating Elpi terms back to Rocq, a term of the form `app [T]` is interpreted as the application of a single term and is therefore equivalent to the term `T` itself. Consequently, the translator simply returns the translation of `T`. By contrast, the term `app []` has no corresponding representation in Rocq. Encountering such a term therefore results in the error: *the app term constructor expects a non-empty list.*

λ-abstractions and ∀-quantifications are encoded using the `fun` and `prod` constructors, respectively. Each of these constructors carries the `name` of the Rocq binder (used only for pretty printing), the type of the binder, and an Elpi higher-order object of type `term → term`, which represents the abstraction of the bound variable in the term.

The Rocq-Elpi backend translates Rocq terms into HOAS Elpi terms and conversely when the two systems interact. In table 2.1, we show some examples of this encoding: the first column contains Rocq terms, and the second column their HOAS representation in Elpi.

The second and third rows illustrate how binders are handled for λ-abstraction and ∀-quantification. The original binder name from Rocq is stored in the `fun` and `prod` constructors, while the name of the bound variable in Elpi is not relevant. In the third row, the Elpi abstracted variable `z` is reused to represent the abstraction of the object language when applied to the predicate `p`.

The second row shows a Rocq unification variable `?F`. It is translated into an Elpi unification variable `G`, which internally corresponds to `?F`. When `G` is instantiated during the execution of the Elpi program, the result is translated back into a Rocq term and assigned to `?F`.

Quotations and Antiquotations Writing Rocq terms can be tedious, as it requires explicitly constructing their full HOAS representation. As shown in table 2.1, the Elpi HOAS syntax is significantly more verbose than standard Rocq syntax. Moreover, Rocq often includes implicit subterms, whereas Elpi does not provide such a mechanism.

To simplify development, Rocq-Elpi offers a system of quotations and antiquotations. This mechanism allows embedding Rocq syntax within Elpi code, and conversely Elpi syntax within Rocq code, in a recursive manner.

An illustrative example, adapted from [Tas25], is the following:

```
print {{ 1 + lp:{{app[«S», {{ 0 }} ]}} }}
```

(print: Elpi; 1 +: Rocq; app[«S», : Elpi; 0: Rocq;]: Elpi; }}: Rocq)

Within an ELPI code block, the syntax `{{ ... }}` denotes a quotation that allows writing ROCQ code. Conversely, within ROCQ code, the syntax `lp:{{ ... }}` denotes a quotation for embedding ELPI code. These constructs can be nested arbitrarily.

2.4.2 API in Rocq-Elpi

Thanks to the HOAS representation of ROCQ terms in ELPI, the two languages become more closely aligned and can share a common representation. This simplifies their interaction and allows computations to be interleaved between them. The ROCQ language exposes several APIs that can be invoked directly from ELPI[†]

```
pred coq.unify-eq term, term → diagnostic.
```

The predicate above is widely used in ELPI developments. It takes two terms and attempts to unify them. If unification fails, it returns a diagnostic explaining the error. Internally, this predicate relies on the ROCQ unification algorithm. When ELPIcoq.unify-eq is called, the HOAS representation of the terms is translated into ROCQ terms before performing the unification.

```
pred coq.typecheck term → term, diagnostic
```

For example, the predicate above invokes the ROCQ typechecker on a term. The call `coq.typecheck T Ty D` assigns the type of `T` to `Ty` if `T` can be typechecked. Otherwise, it leaves `Ty` unbound and assigns the corresponding type-error diagnostic message to `D`.

2.4.3 Declaring Databases in Rocq-Elpi

In [Tas25, Sec. 4.4.3], a database is defined as a programmatically extensible ELPI program. It contain a collection of code that can be included into commands or tactics by referring to its name.

When a program is executed, it does not use the state of the database at the moment when the command was defined. Rather, it accesses the current content of the database at execution time. This means that any update to the database is visible to all programs that include it.

An ELPI database can be dynamically modified by adding new predicate signatures and rules.

```
kind clause type.
%           Name      Grafting     Rule
type clause string → grafting → prop → clause.

%                            DB name   Indexing   PName
pred coq.elpi.add-predicate string,    string,    string,
                                    list (pair argument_mode string) →.
%                         Scope     DB name    Clause
pred coq.elpi.accumulate scope, string, clause →.
```

[†]These APIs are available at https://github.com/LPCIC/coq-elpi/blob/master/builtin-doc/coq-builtin.elpi.

The `coq.elpi.add-predicate` predicate takes as arguments the name of the database where the predicate is inserted, the indexing strategy[‡], the predicate name, and its specification (argument modes and types).

The `coq.elpi.accumulate` predicate adds a new clause to a database within a given Rocq scope (e.g. module, section, file ...). A clause is a specific type with a single constructor, which contains the clause name, its grafting with respect to the existing database, and the rule itself.

These last two predicates are particularly useful in the context of sections 3.3 and 3.4.

2.4.4 Writing Commands and Tactics in Rocq-Elpi

We now describe how to implement commands and tactics in Rocq-Elpi. A complete description is available at https://lpcic.github.io/coq-elpi/tutorial_coq_elpi_command.html and https://lpcic.github.io/coq-elpi/tutorial_coq_elpi_tactic.html, respectively. Table 2.2 summarizes how to define commands and tactics, how to invoke them, and their corresponding entry points.

Both commands and tactics are typically built on top of an Elpi database, which they can query and, if necessary, modify. A database is declared using the command `Elpi Accumulate Db db_name.`

The code of commands and tactics can be extended as follows:

```
Elpi Accumulate [command_name|tactic_name] lp:{{
  % New rules added here
}}.
```

As mentioned at the beginning of this chapter, commands are used to extend the Rocq language with new definitions and theorems. In other words, a command does not return a value; rather, it is primarily used to modify the internal state of the system. The entry point of a command is a predicate named `main`, with signature `pred main list argument →`. The type of command arguments is defined as follows:

```
kind argument type.
type int      : int    -> argument.
type str      : string -> argument.
type trm      : term   -> argument.
type open-trm : int -> term -> argument.
```

When invoked, an Elpi command typically receives arguments that parameterize its behavior. In particular, the Elpi command parser can parse integers, strings, and (possibly

[‡]We do not discuss indexing in detail, as it is not central for this manuscript. It determines the data structure used to store rules in order to improve search efficiency.

Commands	Tactic	Description
`Elpi Command command_name.`	`Elpi Tactic tactic_name.`	Declaration
`Elpi command_name args.`	`elpi tactic_name args.`	Invocation
`main`	`msolve` and `solve`	Entry point

Table 2.2: Command and Tactic main commands

```
kind goal type.
kind sealed-goal type.
type nabla (term → sealed-goal) → sealed-goal.
type seal  goal → sealed-goal.

typeabbrev goal-ctx (list prop).
%          Context     RS     Ty      Sol     Arguments
type goal  goal-ctx → term → term → term → list argument → goal.
```

Figure 2.8: Representation of goals and sealed goals in ROCQ-ELPI

open) terms. These correspond to the main kinds of data that a user may wish to pass to an ELPI command.

Tactics in ELPI are used to solve ROCQ goals through the ELPI interpreter. Their declaration is similar to that of commands. The entry point of a tactic is the predicate `msolve`, with the following type:

```
pred msolve list sealed-goal → list sealed-goal.
```

An ELPI tactic is a "multi-goal" tactic, meaning that it can operate on multiple goals simultaneously. To support this, the `msolve` predicate takes as input a list of sealed ROCQ goals and returns an updated list of sealed goals. The goals in the list may be reordered to improve proof search.

A `sealed-goal`, shown in figure 2.8, represents a goal that may contain abstractions, typically encoding the context in which the goal is defined. These abstractions are represented using the `nabla` constructor.

For example, consider the ROCQ goal

$$\frac{h : t}{?s : t}$$

where $?s$ denotes the proof term to be constructed for the conclusion t in a context containing the hypothesis $h : t$. In ELPI, this goal is represented as `nabla x\ t'`, where the variable x corresponds to the abstraction of the local hypothesis h in the term t'. The term t' is the ELPI encoding of the ROCQ conclusion t.

Under a spine of `nabla` abstractions, the `seal` constructor encodes a goal. A goal is a tuple with five components: a context C, a raw solution S_r, a type T_y, a final solution S, and a list of arguments L.

The context C assigns types to the quantified variables. The type T_y represents the type of the ROCQ goal to solve. The terms S_r and S are ELPI unification variables associated with the same ROCQ existential variable, and both are constrained by T_y.

Instantiating S_r triggers its elaboration with respect to T_y. By contrast, instantiating S triggers typechecking against T_y. The difference is that elaboration (see section 2.2) may transform the input term, for example by inserting coercions, while typechecking only verifies that the term has the expected type.

The list L contains additional arguments that guide the resolution process. In this work, we mainly use two projections of a goal: the context and the goal type.

We can now describe more precisely the translation of the proof state

$$\frac{h : t}{?s : t}$$

into Elpi. It is represented as `nabla x\ (goal [decl x «t»] (X x) «t» (Y x) [])`. The context `[decl x «t»]` records that `x` has type `t`. The two higher-order variables `X` and `Y` are associated with the Rocq existential variable `?s`.

Below we present a simple example of an Elpi tactic whose purpose is to solve the goal shown above, that is, a tactic that instantiates the Rocq existential variable `?s`.

```
Elpi Tactic C.
Elpi Accumulate lp:{{
  msolve [nabla (x\ seal (L x))] R :- pi x\ solve-ctx (L x) R.

  pred solve-ctx goal -> list sealed-goal.
  solve-ctx (goal [decl H T] _ T R []) [] :- R = H.
}}.
```

The tactic expects a singleton list containing a single sealed-goal. This sealed-goal is expected to abstract over a single `nabla`, corresponding to the hypothesis `h`. The auxiliary predicate `solve-ctx` checks that the type of the goal matches the type of the hypothesis in the context. When this is the case, the solution variable `R` is instantiated with `H`, and the empty list of goals is returned, indicating that the proof has been completed.

If the predicate `msolve L N` is not defined, or if it fails when applied to the list of goals `L`, the system attempts to solve each goal in `L` individually using the predicate `solve`. In this case, `solve` is invoked on the contents of the `seal` constructor. Consequently, all `nabla` abstractions have already been traversed, and the corresponding bound variables have been quantified and loaded in the current state.

```
pred solve goal → list sealed-goal.
```

Using the `solve` predicate, the implementation of the tactic `C` shown above can be simplified as follows:

```
Elpi Tactic C.
Elpi Accumulate lp:{{
solve (goal [decl H _ T] _ T R []) [] :- R = H.
}}.
```

In this implementation, there is no need to traverse the `sealed-goal` explicitly, since it has already been traversed before the call to the `solve` predicate.

The final point addressed in this section concerns the `refine` predicate.

Given a goal `G` and a proof term `P`, one may wish to typecheck `P` against the `G` and obtain the corresponding list of subgoals that must be solved recursively. The Rocq tactics performing this task is called `refine`.

In Elpi, the `refine` predicate exists and has the same behavior: it takes a proof term and a goal, it typechecks the proof against the goals and returns a list of subgoals in the form of a list of sealed-goals. It is customary for Elpi tactics to invoke `refine` in order to produce the output list for the `solve` predicate.

The `refine` procedure may be slow, especially when applied to large proof terms. This is because it type-checks the entire term. For performance reasons, Elpi also provides a variant, `refine.no_check`, which skips type checking and directly produces the list of sealed subgoals from the given term and goal. This approach is faster, although less reliable, since it assumes that the provided proof term is well typed.

Chapter 3

Architecture of a Type-Class Solver in Rocq-Elpi

Type classes provide an elegant mechanism for implementing ad-hoc polymorphism [Wad89]. As discussed in section 2.2.1, they allow the same symbol to be reused in different contexts. Moreover, as explained in section 2.2.1, in a proof assistant such as Rocq, instances of a type class may contain properties, making theorem overloading possible.

Using the type-class solver for overloaded theorems provides a form of automatic proof search. Given a theorem statement represented as a type-class goal, the solver searches the instance database for a suitable theorem instance and automatically constructs the corresponding proof.

This mechanism is closely related to query resolution in Prolog. A type-class goal can be viewed as a query to a predicate whose arguments correspond to the parameters of the class. Type-class instances play the role of rules stored in a database, and the solver resolves goals by backchaining on these rules until a proof is found.

A meta-language integration for type-class resolution must provide two components: (1) a compiler that translates Rocq class and instance declarations into the meta-language representation, and (2) a type-class solver that animates the search by interacting with the resulting database.

In our setting, the meta-language used to implement the meta-solver is Elpi. Consequently, the main task is to implement the class and instance compiler, which incrementally builds the database on which the Elpi program operates.

As explained in section 2.2.1, the type-class solver itself typically follows a Prolog-like resolution strategy: instances are tried according to their priority, and backtracking may reconsider previous choices when a branch fails. In Elpi, this behavior comes for free, since the resolution mechanism of the type-class solver strongly imitates the standard execution model of the Elpi runtime.

The remainder of this chapter is organized as follows. We first demonstrate how Rocq terms can be compiled into an Elpi database and how type-class goals can be translated into Elpi queries, enabling proof automation through Elpi's search mechanism. In section 3.2, we present a minimal instance compiler, showing how a basic type-class solver can be implemented with few lines of code. While simple, this approach lacks important features required for handling complex instances. We therefore introduce a more

powerful compiler in sections 3.3 and 3.4, where we discuss how rule priorities influence database construction, how ROCQ goals are encoded as ELPI queries, and how users can extend the database with custom rules to tailor the solver's behavior.

We then report on our experimental evaluation in section 3.5, where the solver is applied to real-world libraries such as STDPP and TLC. Benchmarks presented in section 3.5.1 compare the performance of ROCQ and ELPI, analyzing how execution time in ELPI is distributed across different phases of the search. In section 3.5.2, we examine the differences between ELPI and ROCQ modes and their impact on resolution. Finally, in section 3.5.3, we discuss the unification challenges that arise when using ELPI's unification algorithm on ROCQ terms. The discussion concludes in section 3.6.

3.1 Interfacing Rocq with Elpi for Type-Class Resolution

To address type-class unification problems, we extend ROCQ with the following APIs (simplified here for clarity):

```
(* API for Type-Class Compiler *)
val register_compiler   : string → (event → unit) → unit
val activate_compiler   : string → unit
val deactivate_compiler : string → unit

(* API for Type-Class Solver *)
val register_solver   : string → tc_solver → condition → unit
val activate_solver   : string → unit
val deactivate_solver : string → unit
```

These APIs allow one to declare new compilers and solvers inside the ROCQ system.

A compiler is implemented as an observer, which is triggered whenever a new class or instance is declared. To define a compiler, one provides a name and an event handler to `register_compiler`. Compilers are stored in a table that maps their names to their handlers. The handler is a higher-order function, invoked each time an event occurs. Each event contains the class or instance currently being defined in ROCQ. The handler processes this information and updates its internal database. Observers can be activated or deactivated when needed.

A type-class solver is registered using `register_solver`, which takes a name, a value of type `tc_solver`, and a condition. The `tc_solver` is a higher-order function that receives a ROCQ context together with a list of goals, and returns an updated environment where some type-class goals may be solved. The condition specifies whether the solver should be applied to the current list of goals. This is useful, for instance, when a solver is intended only for specific type-class problems.

Solvers are stored in a table indexed by their name. During resolution, the ROCQ type-class mechanism scans the registered solvers and selects the first active one whose condition holds. If the condition is not satisfied, the next solver is considered. As for observers, solvers can be activated or deactivated by the user.

```
Class Add T := {
  plus: T → T → T;
  comm: ∀ a b, plus a b = plus b a.
}.

Instance addNat : Add nat := ...
Instance addR : Add R  := ...
Instance addProd : ∀ T1 T2, Add T1 → Add T2 → Add (T1 * T2) := ...
```

Figure 3.1: The `Add` type class and its instances

```
%         Inst  Ty    Args       Prems        Res
pred comp term, term, list term, list prop → prop.
comp I {{lp:T → lp:Bo}} Ag P (pi y\ R y) :- !,  % r→
  pi x\ comp I Bo [x|Ag] [tc T x | P] (R x).
comp I {{∀ x, lp:(Bo x)}} Ag P (pi y\ R y) :- !, % r∀
  pi x\ comp I (Bo x) [x|Ag] P (R x).
comp I T Ag P (tc T Proof :- Body) :-   % rB
  rev Ag Ag',
  mk-app I A' Proof,
  rev P Body.

pred compile gref →.
compile G :- coq.env.typeof G Ty,
  comp (global G) Ty [] [] R,
  coq.elpi.accumulate _ "tc.db" (clause _ _ R).
```

Figure 3.2: Simple Elpi compiler

3.2 Simple instance compiler

Conceptually, a type-class goal can be translated into a query with two arguments: the goal itself and a concrete instance witnessing it. The search is expressed using the predicate `pred tc term → term.`

By invariant, a query of the form `tc T I` ensures that the term `I` has type `T`.

As an example, consider the `Add` type-class introduced in section 2.2.1. For convenience, the full definition of the class and its instances is reproduced in figure 3.1*.

After an instance is declared, a trigger activates the instance compiler shown in figure 3.2. The compiler is similar to the example presented in [Tas25, Sec. 4.5].

The entry point of the compiler is the predicate `compile`, which takes the global reference `G` of the instance as input. Using the `coq.env.typeof` API, the compiler retrieves the type `Ty` of `G`. Compilation then proceeds by invoking the auxiliary predicate `comp` to produce the rule `R`. Finally, `coq.elpi.accumulate` adds the clause `R` to the database, here called `tc.db`.

The predicate `comp` performs the core of the compilation. It takes four arguments: the instance term; its type, which is structurally consumed during recursion; and two

*The auxiliary predicates are provided in appendix A

accumulators storing the list of instance arguments and the list of premises that will form the body of the resulting rule. From these inputs, `comp` produces the ELPI rule corresponding to the compiled instance.

Compilation proceeds by analyzing the structure of the instance type. Three cases are relevant. Rules $r{\to}$ and $r{\forall}$ handle quantifications, while rule rB is the base case that constructs the final ELPI rule from the information accumulated during the recursion.

The base case is reached when the current type `Ty` is not a quantification. The compiler then constructs the final rule by assembling its head and body.

The head is the predicate `tc` applied to the instance type `T` and to a proof term `Proof`. The proof is obtained by applying the instance `I` to its arguments `rev Ag`. Since the arguments are accumulated in reverse order during the traversal, the list `Ag` is reversed before building the application.

The premises of the clause are stored in the accumulator `P`. As for the arguments, the list is reversed before constructing the final rule.

For example, compiling the instance `addNat` produces the rule

```
tc {{Add nat}} {{addNat}}.
```

When the current type `Ty` is a quantification, the compiler introduces a fresh nominal variable representing the bound variable in the recursive call. Two situations must be distinguished depending on the shape of the quantification.

If the quantification corresponds to a premise of the instance, rule $r{\to}$ is used. This rule introduces a fresh variable `x` representing the conditional instance with type `T`. The premise `tc T x` is added to the list of premises `P`, and the variable `x` is added to the list of instance arguments `Ag`.

Rule $r{\forall}$ handles dependent quantification corresponding to parameters of the instance, that are supposed not to have a class as type. In this case the abstraction is simply consumed: the list of premises `P` remains unchanged, while the variable `x` is added to the list of instance arguments.

Both rules $r{\to}$ and $r{\forall}$ return a term of the form `(pi y\ R y)`, where `R` is the result of the recursive call to `comp`. Since `R` has type `term → prop` and abstracts the local variable `x`, the compiler abstracts `x` in the result `pi y\ R y`. This ensures that the resulting rule is closed (ground), as required by the `coq.elpi.accumulate` predicate.

As an example, consider the instance `addProd`, whose type is

$$\forall \underbrace{x\ y}_{r\forall}, \underbrace{\texttt{Add}\ x \to \texttt{Add}\ y}_{r\to} \to \underbrace{\texttt{Add}\ (x * y)}_{rB}.$$

Compiling this instance yields the following ELPI rule:

```
pi L\ pi R\ pi PL\ pi PR\  % this line is optional in toplevel rules
  tc {{Add (lp:L * lp:R)}} {{addProd lp:L lp:R lp:PL lp:PR}} :-
    tc {{Add lp:L}} PL, tc {{Add lp:R}} PR.
```

This rule provides a proof for `Add (x * y)` provided that the two premises `Add x` and `Add y` can be solved. Note that the generated rule contains explicit `pi` quantifiers for the unification variable in the rule. In a hand-written rule they become optional.

As explained in section 2.4.4, a Rocq goal can be passed to an Elpi tactic responsible for solving it. Type-class goals are translated in the same way and can therefore be handled directly by Elpi.

To perform type-class resolution, we define an Elpi tactic whose entry point is captured by the following rule:

```
solve (goal _ _ Ty _ _ as G) S :- tc Ty P, refine P G S.
```

Given a Rocq goal `G`, the tactic retrieves its type `Ty`, performs a backchaining step on `tc Ty P` to obtain a proof `P`, and refines the goal `G` with `P`. The refinement produces a list of subgoals `S`, which are returned to Rocq.

Complete example of execution To illustrate how type-class goals are delegated from Rocq to Elpi, we present a complete example. Consider the following Rocq goal, using the class and instances defined in figure 3.1:

$$\frac{}{?s : \texttt{plus}\ (\pi, 3)\ (e, 4) = \texttt{plus}\ (e, 4)\ (\pi, 3)}$$

This goal can be solved using the commutativity property of addition. The interesting part is the associated type-class resolution problem, which consists in finding an implementation of `plus` from the instance database, that is, an instance of `Add` for the type `(R * nat)`.

The corresponding Elpi goal is `(goal [] X0 {{Add (R * nat)}} X1 [])`. This goal is solved through the recursive call `tc {{Add (R * nat)}} P`. Applying the rule corresponding to `addProd` generates two recursive type-class goals, which are solved using the instances `addR` and `addNat`, respectively. As a result, `P` is instantiated with `addProd R nat addR addNat`, which is returned to Rocq as the solution to the type-class goal.

Concluding remarks The implementation presented in this section illustrates the expressive power of the Elpi language: the core of the compiler fits in roughly eight lines of code, while a similar number of lines is required to complete the Elpi command for the compiler. The solver itself consists of a single rule.

However, this simplified implementation has some limitations it is worth to point out.

- The predicate `tc` does not expose the class arguments as Elpi arguments. Consequently, the Rocq modes (see section 2.2.1) declared on a class are not preserved in the Elpi solver. The compiler should therefore generate a specialized predicate for each class, with arguments and modes matching those of the class so that rule backchaining can be performed correctly. This problem is addressed in section 3.3.

- Rule $r\forall$ does not account for the possibility that the quantified variable x is itself a premise of the class. In complex instances, x may have a class as its type and should therefore appear as a premise in the generated rule. This problem is addressed in section 3.4.

The compiler does not handle instances of the form $(c_1 \to c_2) \to c_3$, where the implementation of c_3 requires a proof of c_2 in a context extended with a local instance of c_1. Supporting such cases requires generating complex formulas and therefore tracking the positive/negative position (see section 2.3.1) of the instance being compiled. This problem is addressed in section 3.4.

The `solve` rule of the solver ignores the context of the goal. As discussed in section 2.4.4, the first projection of a `goal` in ROCQ-ELPI is its context, which may contain local instances that should be compiled and used during resolution. This problem is addressed in section 3.4.2.

3.3 Class compilation

When a class c is declared in ROCQ, the compiler generates a corresponding ELPI predicate. If the class has n parameters, the generated predicate has $n + 1$ arguments: the first n arguments correspond to the parameters of c, while the final argument represents the instance witnessing the class in the current rule.

From the perspective of the ELPI solver, two aspects of a class declaration in ROCQ are particularly relevant: the global reference identifying the class and the modes associated with its parameters. In ROCQ, users may explicitly declare modes using the attribute `#[mode="..."]`. If no mode declaration is provided, the parameters of the class are considered outputs by default.

The compiler responsible for generating the corresponding ELPI predicate is shown in figure 3.3[†].

The entry point of the compiler is the predicate `add-class-pred`, which takes two arguments: the global reference `C` of the class and a string `S` describing its modes. The string `S` encodes the modes using the three ROCQ symbols `-`, `+`, `!`, separated by spaces. If the user does not provide an explicit mode declaration, `S` is the empty string.

Predicates can be created dynamically in ELPI using the API `coq.elpi.add-predicate` (see section 2.4.4).

Two cases must be distinguished depending on whether the mode string is empty. If it is empty, the compiler derives the modes from the type of the class. It retrieves the type of `C` and creates a pair `pr out "term"` for each quantification in that type. In the base case, an additional output argument corresponding to the instance is added, again represented by the pair `pr out "term"`.

As an example, consider the declaration of the class `Add` (cf. figure 3.1). When this class is declared, the ELPI class compiler is triggered and receives the ROCQ symbol `«Add»` as input. Since no mode attribute is provided, the compiler receives the empty string.

From the symbol `Add` the compiler retrieves its type and determines the number of parameters. In this example, the HOAS representation of the type of `Add` in ELPI is `{{Type → Type}}` which indicates that the class has a single parameter.

From this information the compiler generates the predicate associated with the class:

```
pred tc-Add → term, term.
```

[†]The auxiliary predicates are provided in appendix A.

```
pred dft-class-mode term → list mode-type.
dft-class-mode (prod _ _ B) [pr out "term" | L] :- !,
  pi x\ dft-class-mode (B x) L.
dft-class-mode _ [pr out "term"].

pred str→mode string → mode-type.
str→mode "-" (pr out "term").
str→mode "+" (pr in "term").
str→mode "!" (pr in "term").

pred str→modes gref, string → list mode-type.
str→modes C "" M :- !, dft-class-mode {coq.env.typeof C} M.
str→modes _ S  M' :-
  rex.split " " S SL,
  map SL str→mode M,
  append M [pr out "term"] M'.

pred add-class-pred gref, string →.
add-class-pred C S :-
  gref→pred-name C N,
  str→modes C S M,
  coq.elpi.add-predicate "tc.db" _ N M.
```

Figure 3.3: Class compiler

If `S` is not empty, the user has provided explicit modes for the class. In this case, each Rocq mode is translated into the corresponding Elpi mode. This translation is performed by the predicate `str->mode`, which maps `-` to the Elpi output mode, and both `+` and `!` to the Elpi input mode.

3.3.1 Rocq and Elpi mode discrepancies

Our type-class solver is not intended to replicate the Rocq solver. Instead, it aims to exploit, whenever possible, the features of the Elpi language. A natural claim is that mapping two Rocq mode to only one in Elpi is that the Rocq and Elpi type-class solvers will have different runtime behaviors.

We can note that there is no direct correspondence between Rocq modes and Elpi modes. While the Rocq mode `-` corresponds naturally to the Elpi output mode, neither `+` nor `!` has a precise counterpart among the Elpi input mode. Furthermore, a Rocq class may declare multiple mode specifications, whereas an Elpi predicate admits only a single mode declaration.

We recall that the Rocq modes `+` and `!` impose conditions on the shape of the argument: `+` requires a ground term, while `!` requires a term with a rigid applicative head. During the resolution phase, if a goal does not satisfy the mode constraint imposed by the class declaration, a failure immediately stops the instance search. In contrast, Elpi modes do not check the shape of the argument before backchaining the query into the database, it, instead, modifies the unification strategy by using the matching procedure (see section 2.3.2).

```
pred build-rule bool, prop, list prop → prop.
build-rule tt Head Prems (Head :- Prems).
build-rule ff Head Prems (Prems => Head).

%         Pol    Inst  Ty    Args        Prems         Res
pred comp bool, term, term, list term, list prop → prop.
comp B I {{∀ x: lp:Ty, lp:(Bo x)}} Ag P (pi y\ R y) :- % r→
  is-class? Ty, !, neg B B',
  pi x\
    comp B' x Ty [] [] (M x),
    comp B I (Bo x) [x|Ag] [M x | P] (R x).
comp B I {{∀ x, lp:(Bo x)}} Ag P (pi y\ R y) :- !,      % r∀
  pi x\ comp B I (Bo x) [x|Ag] P (R x).
comp B I Ty Ag P R :-                                   % rB
  safe-dest-app Ty C CAg,
  rev Ag Ag', mk-app I Ag' Proof,
  append CAg [Proof] Ags,
  make-rule-head C Ags Head,
  rev P P', build-rule B Head P' R.

pred compile gref →.
compile G :- coq.env.typeof G Ty,
  comp tt (global G) Ty [] [] R,
  coq.elpi.accumulate _ "tc.db" (clause _ _ R).
```

Figure 3.4: Instance compiler

In order to see the different mode behavior of the ROCQ and ELPI modes, we rework the implementation of the `Add` type class.

```
#[mode="+"] Class Add T :=

Instance addNat : Add nat.
Instance addDummy T : Add T.
```

Consider a goal g = `Add ?X` where `?X` is an existential variable. With the ROCQ mode system, this goal has no solution because the argument of the class is a non-ground term and therefore does not satisfy the mode constraint. In contrast, under the ELPI mode interpretation the instance `addDummy` is a valid solution.

In ELPI, the instance `addDummy` acts as a catchall rule: the first argument of the `Add` class is a variable, i.e., a pattern that can match any goal involving `Add`. Catchall rules have two main consequences. First, in the goal g above, the variable `?X` is not assigned: the proof term leaves a hole in the solution. Second, they introduce a source of nondeterminism in ELPI. As their name suggests, catchall rules can apply to any goal, and overlap with other applicable instances, resulting in multiple possible rule selections. This nondeterministic behavior is detected by the determinacy checker presented in chapter 5.

3.4 Instance compilation

As explained in section 3.1, instance compilation is triggered immediately after an instance declaration. Figure 3.4 shows an instance compiler more advanced than section 3.2. It takes into account the type of dependent quantification and the positive/negative position of the term being compiled.

Similarly to figure 3.2, we define a `compile` predicate which calls the `comp` auxiliary function. This time, `comp` takes one more boolean argument.

The compilation proceeds by analyzing the structure of the instance type. There are three relevant cases. Rules $r{\to}$ and $r\forall$ handle quantifications, while rule rB is the base case that constructs the final Elpi rule from the information accumulated during the recursive traversal.

When the current type has the form `∀x : Ty, Bo x`, the abstraction binds a variable of type `Ty` in the body `Bo`. The test `is-class? Ty` checks whether `Ty` has the shape $\forall x_1 \dots x_n$, `app[C|Ag]` (possibly with zero quantifiers) and `C` is a declared type class.

If this condition holds, we are compiling a conditional instance and rule $r{\to}$ is applied. The rule introduces a fresh nominal variable `x` representing both the abstracted variable in `Bo` and the local instance with type `Ty`. To construct the premise corresponding to `x`, the compiler recursively invokes `comp` on `x` while switching the boolean in the recursive call. This recursive call produces the premise `M`, which abstracts `x`. Compilation then continues on `Bo x`, while extending the accumulators with the new instance argument `x` and the new premise `M x`.

Rule $r\forall$ handles the case where the quantified type is not a type class. In this situation, the compiler behaves exactly as rule $r\forall$ in figure 3.2: it simply consumes the abstraction, leaving the list of premises `P` unchanged and adding the variable `x` to the list of instance arguments.

As in figure 3.2, the base case of `comp` is reached when the current type `Ty` is not a quantification. In this case, `Ty` is a type class `C` applied to a list of arguments `CAg`. The compiler then constructs the final rule by building its head and premises.

To build the head `Head`, the compiler first constructs the proof term `Proof`, obtained by applying the instance `I` to its arguments `rev Ag`. The predicate `make-rule-head` takes the class name as its first argument in order to generate the predicate corresponding to the class. The arguments of this predicate are obtained by appending the proof term to the list of the class arguments.

The premises of the clause are stored in the accumulator `P`. As with the instance arguments, they are reversed before constructing the final rule.

Finally, the predicate `build-rule` assembles the rule according to the current position tag: positive or negative. In the former case, it produces a standard clause `Head :- Prems`. Otherwise, it produces the implication `Prems => Head`.

Given the instance in figure 3.1, this version of the compiler produces the following rules:

```
tc-Add {{nat}} {{addNat}}.
tc-Add {{R}} {{addR}}.
pi L \ pi R \ pi PL \ pi PR \ % this line is optional in toplevel rules
  tc-Add {{lp:L * lp:R}} {{addProd lp:L lp:R lp:PL lp:PR}} :-
    tc-Add L PL, tc-Add R PR.
```

These rules are similar to those produced by the compiler in section 3.2, except for the specialization of the predicate `tc-Add` for the `Add` class.

```
Inductive a := p | q | r.
Variable to_prop : a → Prop.

Inductive mlogic :=
  | atom : a → mlogic
  | and : mlogic → mlogic → mlogic
  | impl : mlogic → mlogic → mlogic.

Fixpoint interp (p: mlogic) : Prop :=
  match p with
  | atom a => to_prop a
  | and a b => interp a /\ interp b
  | impl a b => interp a → interp b
  end.

Class Provable (T : mlogic) := { proof : interp T }.

Instance Pand F1 F2 :
  Provable F1 → Provable F2 → Provable (and F1 F2) := ...

Instance Pimpl F1 F2 :
  (Provable F1 → Provable F2) → Provable (impl F1 F2) := ...
```

Figure 3.5: Example with simple logic

An interesting example of compilation, where the positive/negative position of a subterm in the compilation phase plays a central role (and is incorrectly handled by the compiler in section 3.2), is shown in figure 3.5. The code illustrates an encoding of a fragment of propositional logic using the ROCQ type-class mechanism as a proof search engine.

We define an inductive type `mlogic` representing a restricted language of formulas consisting of atoms, conjunctions, and implications. The constructor `atom` corresponds to logic atoms, while `and` and `impl` correspond to conjunction and implication, respectively.

The function `interp` assigns a semantics to `mlogic` formulas by translating them into ROCQ propositions. This interpretation is compositional: an atom evaluates to a proposition using the `to_prop` function, a conjunction is interpreted as a logical conjunction, and an implication as a function type.

Automatic reasoning is expressed via the type-class `Provable T`, which packages a proof of `interp T`. This allows ROCQ's type-class resolution mechanism to act as a proof search procedure.

The inference rules of the logic are encoded as type-class instances. The instance `Pand` corresponds to conjunction introduction: if `F1` and `F2` are provable, then `and F1 F2` is also provable.

Similarly, the instance `Pimpl` captures implication introduction. It states that to prove `impl F1 F2`, it suffices to show that `F2` is provable under the assumption `F1`.

The instance `Pand` is compiled into:

```
pi L \ pi R \ pi PL \ pi PR \ % this line is optional in toplevel rules
```

```
tc-Provable {{and lp:L lp:R}} {{Pand lp:L lp:R lp:PL lp:PR}} :-
  tc-Provable L PL, tc-Provable R PR.
```

This rule closely mirrors the rule for `addProd`, since both rely on the same search principle: constructing a proof requires recursively constructing proofs for the subcomponents (the arguments of `and`, respectively $*$).

The rule corresponding to the instance `Pimpl` is more subtle, since it involves a higher-order hypothesis. Its type is:

$$\forall \texttt{F1 F2}, \underbrace{\underbrace{(\texttt{Provable F1}}_{positive} \to \underbrace{\texttt{Provable F2})}_{negative}}_{negative} \to \underbrace{\texttt{Provable (impl F1 F2)}}_{positive}$$

The compiled rule is:

```
pi F B Q \ % this line is optional in toplevel rule
  tc-Provable {{impl lp:F lp:B}} {{Pimpl lp:F lp:B lp:Q}} :-
    pi p \ tc-Provable F p => tc-Provable B {{lp:Q lp:p}}.
```

The structure of this rule matches the intended semantics: the head targets `impl F B`, while the body attempts to construct a proof of `B` in a context extended with the assumption `F`.

Even though this rule is very close to its final implementation, it is still not the implementation one would naturally write. In particular, the premise `tc-Provable B {{lp:Q lp:p}}` is not flexible enough, since the solution `{{lp:Q lp:p}}` requires a term with exactly the shape `app[Q, p]`. This is too restrictive, since the variable `Q` may depend on `p`. A more flexible solution is to replace this premise with `tc-Provable B (X p)`. In this way, we use the Elpi term application of a higher-order variable `X` depending on the proof term `p`.

However, we must be careful: `X` is not a Rocq term, but an Elpi term living in the function space. Therefore, some additional work is required to establish the correspondence between the variable `X` and the variable `Q` appearing in the rule head. This is precisely the subject of chapter 4.

3.4.1 Rule Priorities

When declaring instances, the priority assigned to each rule plays a fundamental role in the performance of the resolution phase. In Rocq, every instance is associated with a priority at declaration time.

To support rule priorities, we extend the instance trigger so that it also propagates priority information to the Elpi compiler. To reproduce a similar behavior in Elpi, we exploit its grafting mechanism (see section 2.3.6), which allows rules in the database to be named and enables new rules to be inserted before or after existing ones.

In the Elpi compiler, we predeclare a fixed number of hooks, each associated with an integer n representing a grafting position. We account for this mechanism by modifying the `compile` rule in figure 3.4 so that it takes this integer as input. When a rule is added to the database, we explicitly specify that it must be grafted `:before` the hook named n.

```
pred get-ITy prop → term, term.
get-ITy (decl I _ Ty) I Ty.
get-ITy (def I _ Ty _) I Ty.

pred compile-ctx prop → prop.
compile-ctx C R :- get-ITy C I Ty, is-class? Ty, comp tt I Ty [] [] R.

solve (goal C _ Ty _ _ as G) S :-
  map-filter C compile-ctx H,
  comp ff P Ty [] H R, R,
  refine P G S.
```

Figure 3.6: Goal compiler

The choice of grafting ROCQ instances into the ELPI database by number is only a partial solution, but it is necessary to preserve compatibility with ROCQ. In particular, we must ensure that ROCQ instances can be grafted into the ELPI database while preserving the declaration order of these instances. However, the ROCQ grafting mechanism is less expressive than the one provided by ELPI. Relying on a numerical ordering scheme is sometimes limiting: a user may wish to insert an instance relative to other instances already present in the database, for example by referring to them by name and specifying that a new instance should be placed before or after them. Such a mechanism would provide a more flexible way of controlling instance ordering. Nevertheless, in order to remain compatible with ROCQ, we reproduce its grafting behavior, where each instance is inserted at a position determined by its numerical priority.

3.4.2 Goal Encoding

With the database compilation in place, we now define the tactic used to solve type-class goals. The implementation of the `solve` predicate is shown in figure 3.6.

A ROCQ goal in ELPI contains, among other information, the context `C` in which the goal is defined and a term `Ty` representing the type to be solved. The first step is to traverse the context, select the hypotheses that correspond to type classes, and compile them using the `comp` procedure. This produces a list of local rules `H`, which are added to the program during resolution.

Next, we compile the goal type `Ty`. The compilation is performed with boolean set to `ff`. Concretely,

```
comp ff P Ty [] H R
```

constructs a formula in negative position. Here, `P` is a unification variable representing the proof term, `Ty` is the goal to be compiled, the list of instance parameters is empty, and `H` provides the premises extracted from the context. The result of the compilation is a formula `R` of the form `H => Q`, where `Q` is a query corresponding to a type class whose proof argument is `P`.

Evaluating `R` instantiates the proof term `P`, which is then used to refine the original ROCQ goal.

Example of execution We consider as an example of execution the following Rocq goal, where local instances are taken into account for the resolution.

$$\frac{\texttt{T} : \texttt{Type} \qquad x : \texttt{T} \qquad H : \texttt{Add T}}{?s : \texttt{plus}\ (x, 3)\ (x, 4) = \texttt{plus}\ (x, 4)\ (x, 3)}$$

The goal can be solved using the `comm` property over addition. What is interesting here is the type-class goal aiming to find the right implementation of `plus` in the database, i.e. an instance of `Add` applied to the type `(T * nat)`, where `T` appears in the hypothesis lists.

The Elpi goal is:

```
nabla t \ nabla x \ nabla h \ seal
  (goal
    [decl h `H` {{Add lp:t}}, decl x `x` t, decl t `T` {{Type}}] % the context
    (X0 t x h)
    {{Add (lp:t * nat)}}                                        % the conclusion
    (X1 t x h) [])
```

The context contains a premise which is a typeclass, namely `h`. It is compiled into the simple local rule `tc-Add t h`: the proof for the type-class `Add` applied to the local type `t` is the local hypothesis `h`. The conclusion is encoded as the query `tc-Add {{lp:t * nat}} P`.

To solve this goal we backchain the query in the program extended with `tc-Add t h` and obtain the proof `P := addProd lp:t nat lp:h addNat`.

3.4.3 User-Defined Rules

In addition to the automatic compilation of rules, users can extend the database with custom rules to implement specialized or more efficient resolution strategies in specific situations.

Using the `Elpi Accumulate` command, users can define ad hoc rules, such as the one shown below:

```
tc-Add {{lp:A * lp:A}} P :-
  P = {{let x := lp:A in let p := lp:Pa in addProd x x p p}},
  tc-Add A Pa.
```

This rule matches products whose two components are identical. When searching for an instance of the class `Add` applied to a type `A`, both components of the pair yield the same result, so the search is performed only once. Moreover, the resulting proof term shares memory between subproofs and their types, improving space and time efficiency.

In the simple rule presented above, we illustrate the expressive power of meta-programming in Elpi. Since Elpi is a Turing-complete language, users can arbitrarily customize the proof search by defining their own rules. In particular, there are no restrictions on these extensions: users are free to implement any custom rule that suits their needs.

Although the Rocq proof search mechanism can be extended through the `Hint` database, and in particular through the `Hint Extern` command, which allows the user to invoke Ltac tactics when goals match a given pattern, this customization is limited

to the resolution of individual proof obligations. Since `Hint Extern` is directly integrated into the built-in proof search engine, it naturally allows user-defined tactics to participate in resolution. However, the proof-search strategy itself remains fixed and cannot be customized. In contrast, our type class solver approach makes both the proof search strategy and the individual proof steps programmable. This provides substantially greater expressive power, for example by enabling search strategies and control mechanisms, such as non-local cuts, that are not available through `Hint Extern`.

3.5 Experimental results

3.5.1 Benchmarks

Using the instance compiler presented in the previous section and the ELPI interpreter to solve type-class goals, we evaluate our solver on existing libraries, in particular TLC [Cha21] and STDPP [KJJ+23]. Both libraries use type-class inference as a central implementation mechanism. Studying these libraries helps us understand the practical needs of ROCQ users. Moreover, this evaluation can reveal limitations of our implementation and suggest directions for future work. An important goal of this evaluation is to compare our solver with the native ROCQ solver.

As a first benchmark, we consider a set of classes and instances from STDPP. We focus on the `Inj` type class, defined as follows:

```
Class Inj {A B} (f : A → B) :=
  inj x y : f x = f y → x = y.
```

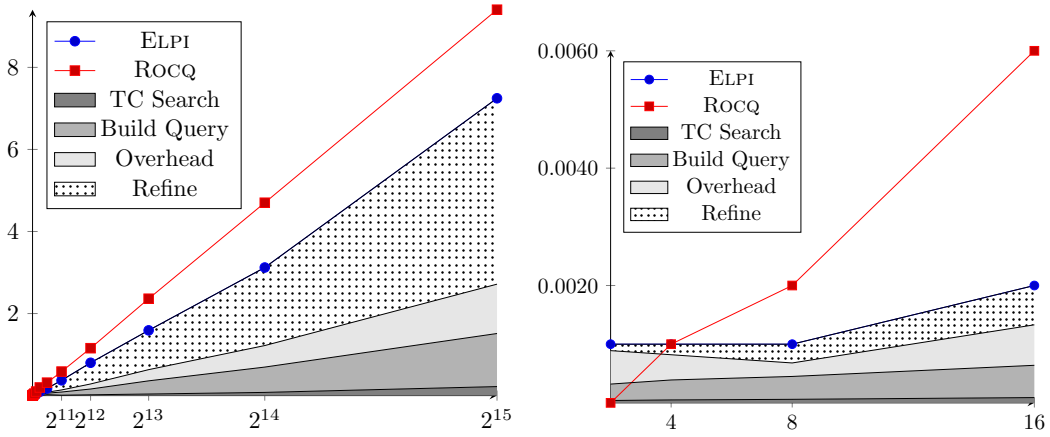
The `Inj` type class expresses that a function f is injective: for all x and y, if $f\ x = f\ y$, then $x = y$.

The library provides 13 instances of this class. These include, for example, instances for the identity function and injections of sum types. The instance targeted by our benchmark is function composition. The composition (written $\circ$) of two functions `f` and `g` is injective if both functions are injective.

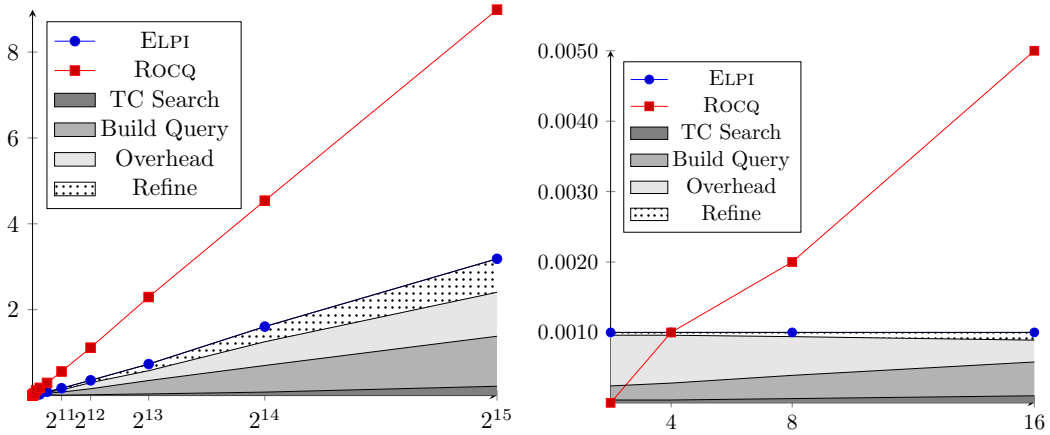
To evaluate performance, we measure the time required by the instance solver to solve goals of increasing size. This allows us to understand where time is spent and to compare ELPI with ROCQ. The benchmark goals are generated by the following function. It builds a complete binary tree where internal nodes are compositions and leaves are a fixed injective function f:

$$g_n = \begin{cases} f & \text{if } n = 0, \\ g_{n-1} \circ g_{n-1} & \text{if } n > 0. \end{cases}$$

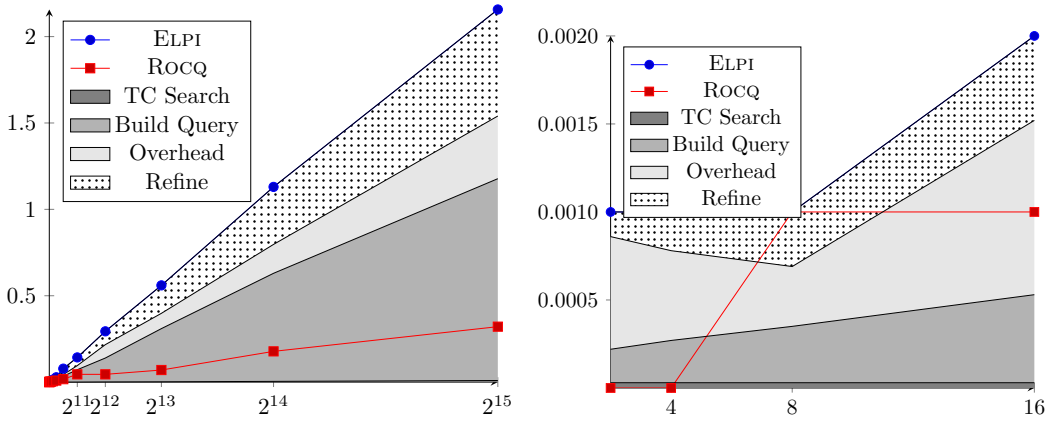
For a given n, we submit the goal `Inj` g_n to the solver. The size of this goal grows exponentially with n. Figure 3.7 shows the solving time for values of n ranging from 0 to 15. The x-axis represents the number of nodes, while the y-axis represents the time in seconds. The plots compare the time spent by the ROCQ and ELPI solvers. For each benchmark configuration, we show both a plot containing the full range of problem sizes and a zoomed-in view of the smaller instances.



(a) With typechecking



(b) Without typechecking



(c) With memory sharing

Figure 3.7: Benchmarks of the Elpi solver. The x-axis of each plot represents the problem size, expressed as the number of nodes. The y-axis represents the solving time, expressed in seconds.

For ELPI, we split the total execution time into four components: type-class resolution, query construction (the translation of ROCQ terms into ELPI), communication between ELPI and ROCQ, and refinement of the final proof against the original goal.

For ROCQ, query construction and communication do not appear as separate costs because they are internal to the implementation. However, separating the cost of instance search from type checking is more difficult, since these operations are interleaved. In particular, partial type checking is performed after each instance selection. Therefore, we cannot easily measure these two costs independently.

The benchmarks can be reproduced by cloning the GitHub repositories `https://github.com/FissoreD/coq` and `https://github.com/FissoreD/coq-elpi`, checking out the `phd` branch in both repositories, and compiling them. Once compilation is complete, the benchmark script `bench_inj.py` should be executed from the `app/tc` subdirectory. The script contains brief documentation describing its usage and the available command-line options.

The results in figure 3.7a show that instance search itself is very efficient. For goals with 2^{15} nodes, it takes approximately 0.2 seconds out of a total execution time of 7 seconds. In contrast, query construction and communication costs increase with the size of the goal. These costs are due to the interaction between ROCQ and ELPI and do not depend on the ELPI interpreter itself. They therefore have a significant impact on small instances, as shown in the right-hand plot of figure 3.7a. However, starting from instances containing approximately 8 nodes, the ELPI-based resolution process is already faster.

The main cost in the plot comes from the call to the `refine` tactic, which, as explained at the end of section 2.4.4, type checks the goals against the generated proof term and, if this operation succeeds, transmits the proof back to ROCQ. This step represents approximately 77% of the total execution time.

We consider two approaches to reduce this cost. First, we can use the more lightweight implementation of refinement, called `refine.no_check` (see again the end of section 2.4.4), which only elaborates the solution before sending it to ROCQ, without invoking the expensive type checking procedure. Second, we can introduce ad hoc rules in the ELPI database for this specific problem.

The first approach, i.e., skipping type checking, reduces the time spent on proof refinement by approximately a factor of 6, as shown in figure 3.7b. In most cases, avoiding type checking of proof terms does not introduce problems. The proof term should be correct because the rules in the database are compiled in a context where they have already been type checked. The main difficulty comes from universe constraints [ST14].

In the ELPI compiler, universes are removed from compiled rules in order to simplify unification. For example, the ROCQ term `Type@{i}` corresponds to the HOAS term `sort (typ i)`. During compilation, the universe level i is replaced by a unification variable. This allows terms at different universe levels to be matched at runtime.

However, removing universes from the database may introduce inconsistencies or cause some universe constraints to be omitted. These issues are usually detected during type checking. A possible solution, which we do not implement, would be to add universe constraints explicitly to the compiled rules. In this way, part of the type checking process would be performed during search. This approach is similar to the strategy used by the native ROCQ solver.

The second approach, shown in figure 3.7c, exploits the same strategy described in section 3.4.3. This is a highly ad hoc case study, but it is useful to compare the custom Elpi rule we introduce with the equivalent external `Hint` that would be written for Rocq.

```
:after "0"
tc-Inj T T {{(@compose lp:T lp:T lp:T lp:F lp:F)}}
{{let P := lp:I in @compose_inj lp:T lp:T lp:T lp:F lp:F P P}} :- !,
tc-Inj T T F I.

Hint Extern 0 (@Inj ?t ?t (@compose ?t ?t ?t ?f ?f)) =>
assert (@Inj t t f) as H by apply _;
exact (@compose_inj t t t f f H H) : typeclass_instances.
```

The results show that, once again, proof search itself is extremely fast: the time spent on this part of the computation is almost negligible in the plot. However, in this setting, all the other costs remain unchanged, and their combination makes the Elpi type-class solver approximately 7 times slower than the native Rocq solver.

Taken together, these experiments show that the Elpi interpreter itself has very low execution overhead. However, this low cost does not come for free: communication between Rocq and Elpi introduces an additional overhead that must be taken into account. Although the Elpi solver becomes competitive for goals containing approximately 8 nodes or more, this improvement is not sufficient, in its current state, to justify its use in libraries such as Stdpp or Tlc, where type-class goals are typically much smaller.

3.5.2 Modes: Elpi vs Rocq in Stdpp

We previously discussed discrepancies between Elpi and Rocq modes in section 3.3.1. Here, we highlight a family of implementation choices in the Stdpp library that make the Elpi mode system inadequate. As a running example, consider the following class.

```
#[mode="- !"] Class Union A M :=
  union: (A → A → option A) → M → M → M.
```

The class is parametrized by two types, `A` and `M`, representing respectively the type of keys and the type of collections. The mode declaration requires `M` to have a rigid applicative head, while no constraint is imposed on `A`. The class provides the overloadable symbol `union`, which computes the union of two collections. During the union operation, the filter function is used to determine which element to keep if a key appears in both collections.

We now consider an instance of `Union` from Stdpp, together with a lemma that relies on it.

```
Instance ounion {A} : Union A (option A) := ...
Lemma union_Nr A (f: A → A → option A) mx : union f mx None = mx.
```

The `ounion` instance implement the class `Union` for the type option, the `union` function behaves as follows: it returns `None` if both arguments are `None`; it uses the higher-order function to determine which element to keep if the arguments are `Some x` and `Some y`; otherwise, it returns the non-`None` argument.

In the lemma `union_Nr`, an implicit type-class constraint must be resolved. The goal is `Union A (option ?X)`, where `?X` is an existential variable corresponding to the implicit

argument of the `None` constructor. At this point, the typechecker lacks sufficient information to instantiate `?X`. The expected behavior is that type-class resolution both selects an instance and determines the value of `?X`.

In ROCQ, the goal is solved using the instance `ounion`. In ELPI, however, the same instance cannot be applied because of mode restrictions. The input mode is declared on the second argument of the class, in particular, matching the query `tc-Union {{A}} {{option lp:X}}` against the clause head for `ounion` fails, since ELPI does not assign variables in input positions during matching, that is it fails to match the query input term `{{option lp:X}}` against the term `{{option A}}`.

Another subtle point arises when additional instances are present. If, before the lemma `union_None_r`, the database is extended with the following instance,

```
Instance oounion {A} : Union A (option (option A)).
```

then, under ROCQ modes, it becomes unclear which instance is selected to solve the same goal produced by lemma `union_Nr`. Indeed, both instances match the constraint `Union A (option ?X)`. Depending on their priorities, `?X` may be instantiated differently, that is either `A` or `option A` depending on which instance among `ounion` and `oounion` is used. As a result, developers may be unaware of which instance has been chosen, potentially leading to type errors later in the development.

In contrast, in ELPI, even with the additional instance `oounion`, type-class resolution still fails to make progress. In particular, the query `tc-Union {{A}} {{option lp:X}}` has no solution. To overcome this limitation, the type of the argument `M` must be provided explicitly in the lemma.

```
Lemma union_Nr A (f : A → A → option A) mx : union f mx (@None A) = mx.
```

3.5.3 Unification problems

As explained in section 3.1, the ELPI solver can be used as an alternative – not a replacement – to the ROCQ solver. In a development, one can enable or disable either solver depending on the needs.

When running the ELPI solver on existing libraries, one may observe failures in situations where the ROCQ solver succeeds, and vice versa, although the latter case is rare. In our setting, we focused on making the target library compile with the ELPI solver.

When a type-class resolution fails in ELPI, the cause can be investigated by inspecting the execution traces of both ELPI (using the ELPI trace browser [Tas25, Sec. 3.5]) and ROCQ (using the `Set Typeclasses Debug` option). Our analysis shows that most discrepancies arise from unification issues. We distinguish two main categories encountered in our study: failures related to $\beta\eta$-reduction, and failures related to δ-reduction (see [ZS15]).

A typical $\beta\eta$-unification example is the term `fun x y => ?F y x` unified with `map`[‡], where `?F` is a unification variable. In ROCQ, this succeeds by instantiating `?F` with `fun x y => map y x`. In contrast, ELPI represents these terms as HOAS expressions, namely `fun _ _ x\ fun _ _ y\ app[F, y, x]` and `global (const «map»)`. Since these terms have different rigid heads, unification fails.

[‡]The standard list map function

This behavior is expected from the Elpi unification algorithm, but it is problematic in the context of instance resolution, where equivalent problems – expressed differently – should be solvable. For example, using Elpi-style λ-abstraction and application, the term `(x\ y\ F y x)` unifies with `map`, assigning `F` to `x\ y\ map y x`. To address this, we extend the instance compiler to detect potential $\beta\eta$-unification issues. Such terms are replaced with a fresh unification variable v, simplifying unification in Elpi. The variable v is then linked back to the original Rocq term, so that instantiations in Elpi are reflected in Rocq. This approach led to our contribution presented in [FT24], described in detail in chapter 4, which provides an automatic solution to $\beta\eta$-unification issues.

A concrete example of $\beta\eta$-unification failure in the Elpi solver arises from the compiled rule for the instance `Pimpl`, introduced at the end of section 3.4. Consider the goal `Provable (impl (atom a) (atom a))` for some atom `a`. This goal has the solution `Pimp (atom a) (atom a) (fun x => x)`. However, the compiled rule fails to find this solution due to $\beta\eta$-unification.

After applying the rule for `Pimpl`, the sub-query becomes

```
tc-Provable {{atom a}} {{lp:Q lp:p}}
```

in a context extended with the hypothesis h = `tc-Provable {{atom a}} p`. Solving the sub-query requires unifying it with h. However, the HOAS term corresponding to `{{lp:Q lp:p}}` is `app[Q, p]`, which cannot be unified with `p`, whose HOAS representation is simply `p`.

The second class of issues concerns δ-unification, which is not handled by the Elpi unification mechanism. In Rocq, definitions can be unfolded during unification. For example, the function *id* unifies with `fun x => x` after unfolding its definition. In contrast, Elpi – and more generally Prolog-like systems – does not support definitional unfolding during unification.

An example of failure due to missing δ-unification is given by the following code, from the library Tlc:

```
Class Extens A := ...
Instance Extens_f1 A1 A2: Extens (∀ x1 : A1, A2 x1) := ...
Definition map A B := A → option B.
```

The Rocq solver can solve the goal `Extens (map A B)` using the instance `Extens_f1`. However, Elpi fails because `map A B` does not syntactically match a term starting with a quantifier.

Providing a general and automatic solution for δ-unification in Elpi is non-trivial. In some cases, the absence of δ-unification is actually desirable, since unfolding definitions can make unification unpredictable and harder to debug. Moreover, it may introduce performance issues [ZS15].

To address δ-unification issues, the user has two options. First, one can declare `map` as a notation for `A → option B`, since notations are unfolded by the elaborator, creating no ambiguity in the unification algorithm. Alternatively, one can add an ad-hoc rule to the solver:

```
Elpi Accumulate TC.Solver lp:{{
  tc-Extens {{map lp:A lp:B}} R :-
    tc-Extens {{lp:A → option lp:B}} R.
}}.
```

This rule manually implements unfolding: when the first argument of `Extens` is `map`, the solver recursively processes the unfolded definition.

3.6 Discussion

The previous sections highlighted the flexibility of the ELPI language and its suitability for compiling ROCQ terms into ELPI rules. In particular, ELPI provides a natural framework for proof automation: its PROLOG-like search engine acts as an inference mechanism, provided that the program contains the derivation rules required to solve a given query. Although the type-class engine in ROCQ exhibits a similar behavior, our goal was not to faithfully reproduce the ROCQ solver. Instead, we aimed to leverage the distinctive features of ELPI whenever possible.

A key advantage inherited from ELPI is the control the user have over the search process. This control is achieved through several mechanisms, including rule priorities, predicate modes, the unify/matching unification algorithm, and the cut operator. All these aspects have been discussed in previous sections, with the exception of the cut operator.

We have deliberately postponed the discussion of cut, as it has no direct counterpart in the ROCQ type-class solver. As explained in section 2.3.2, the cut operator allows the user to commit to a solution once certain conditions (e.g., rule guards) are satisfied. To fully exploit this feature, the ROCQ language would require a syntax extension capable of indicating where cut should be introduced during compilation to ELPI rules. Discussions with other researchers suggest that cut may provide an effective approach to addressing a well-known issue in type-class resolution: determinacy.

In logic programming, determinacy is the property that a predicate produces at most one solution for any given call. The cut operator is a central mechanism for enforcing this property. Transposed to type-class resolution, determinacy would ensure that a class declared as deterministic remains so, regardless of the rules added to the database.

In ROCQ, the flag `Typeclasses Unique Solutions` enforces a related behavior: it computes all possible solutions to a type-class query and returns a result only if exactly one solution exists; otherwise, it raises an error. However, as noted in the ROCQ manual [Roc], this approach is computationally expensive, since it requires exploring the entire search tree.

In contrast, determinacy has been extensively studied in logic programming (e.g., [DW89, LK05, HSC96]) and can be verified statically using abstract interpretation. This approach incurs no runtime cost during goal execution. Moreover, since determinacy is a general property of logic programs rather than being specific to type-class resolution, we extend ELPI with a determinacy checker implemented at the compiler level. As a result, all applications built on ELPI, including the type-class solver, can benefit from this analysis.

We presented this work in [FT26], where we describe the implementation of a determinacy checker for ELPI that accounts for both the cut operator and higher-order features

such as local rule insertion. A detailed discussion is provided in chapter 5. Finally, we formalize the first-order fragment of the checker within Elpi in the Rocq system; this formalization is described in chapter 6.

Chapter 4

Supporting $\beta\eta$-unification in meta-programs

In the previous chapter, and in particular in section 3.5.3, we have talked about the unification issues that we encounter in the Elpi class solver when unifying certain Rocq terms. Unification modulo δ-reduction is not supported by Elpi, but the $\beta\eta$-conversion rules are part of the Elpi unification algorithm. In this chapter, we propose a framework that simplifies the unification of HOAS representations of object-language terms in the meta-language. The main goal of this chapter can be summarized as follows: *how to reuse the higher-order unification procedure of the meta language in order to simulate a higher-order logic program for the object language.*

The example we have given in section 3.5.3 providing a source of $\beta\eta$-unification problem is due to the instance `Pimpl`.

```
tc-Provable {{impl lp:F lp:B}} {{Pimpl lp:F lp:B lp:Q}} :-
  pi p \ tc-Provable F p => tc-Provable B {{lp:Q lp:p}}.
```

The variable `Q` in Rocq is an object living in the function space, while in the HOAS representation of Elpi `Q` is a first-order variable of type `term`. As shown in section 3.5.3, the Rocq statement `Provable (impl (atom a) (atom a))` can be solved by the Rocq type-class solver using the instance `Pimpl`. On the contrary, Elpi fails: the rule above would be used on the query `tc-Provable {{impl (atom a) (atom a)}} S` but the sub-query `tc-Provable {{atom a}} {{lp:Q lp:p}}` will fail because it cannot be unified with the local rule `tc-Provable {{atom a}} p`.

The root cause of the problems we outlined in this example is a mismatch between the equational theories of the meta-language and the object language, which limits the effectiveness of the meta-language unification procedure. The equational theory of Elpi includes $\beta\eta$-equivalence and its unification procedure can solve higher-order problems in the pattern fragment (definition 2.3.3). Although the equational theory of Rocq is much richer, automatic proof search procedures typically rely, for efficiency and predictability, on a unification procedure restricted to $\beta\eta$-equivalence and the higher-order pattern fragment. Outside this fragment, Rocq's unification algorithm employs heuristics. For example, the unification problem `P (Q 3) = f (g 3)` succeeds with the substitution $\sigma = \{\texttt{P} \mapsto \texttt{f}, \texttt{Q} \mapsto \texttt{g}\}$. However, this problem admits many incomparable solutions, such as

$\sigma = \{\texttt{P} \mapsto \lambda x.\texttt{f (g 3)}, \texttt{Q} \mapsto \lambda x.100\}$, illustrating the non-deterministic search space explored by higher-order unification algorithms such as Huet's [Hue75].

As a first approximation, a more sophisticated encoding of ROCQ terms would produce the following compilation of `Pimpl`:

```
tc-Provable {{impl lp:F lp:B}} {{Pimpl lp:F lp:B lp:Q}} :-
  link Qm Q, % a constraint relating the local variable Qm with Q
  pi p \ tc-Provable F p => tc-Provable B (Qm p).
```

In the body of the rule, we have added a new higher-order ELPI variable `Qm` which has the type `term → term`. In the recursive call to `tc-Provable` we pass as argument the variable `Qm` applied, using ELPI application, to `p`, allowing the unification problem `Qm p = p` to get a solution, namely, `Qm = x\ x`.

The role of the `link` predicate is to relate the ELPI variable `Qm` back to the ROCQ variable `Q`. For instance, when `Qm` is assigned, then `Q` is assigned to the HOAS term `fun `dummy_name` T (x\ x)` where `T` is the type of the bound variable `x`. With this encoding, the query `tc-Provable {{impl (atom a) (atom a)}} S` has a solution.

The purpose of this compilation is to detect, at compile time, all subterms that may cause unification difficulties up to $\beta\eta$-conversion. Each such subterm t is replaced by a unification variable v. This transformation simplifies unification in the meta-language, while an additional premise is introduced to relate t and v.

Intuitively, this transformation exposes object-level structure as meta-level structure, allowing the meta-language unification algorithm to operate directly on it.

Our compilation strategy is described as a general framework based on two languages: $\mathcal{O}$ for the object language, and $\mathcal{M}$ for the meta-language. We explain how $\mathcal{O}$ is encoded into $\mathcal{M}$, and we show that the higher-order unification procedure of the meta-language, denoted $\simeq_m$, can simulate a higher-order unification procedure $\simeq_o$ for the object language, including $\beta\eta$-conversion.

The following sections are inspired from our paper [FT24]. In particular, section 4.1 states the problem formally and gives the intuition behind our solution; section 4.2 sets up a basic simulation of first-order logic programs, section 4.3 and section 4.4 extend it to higher-order logic programs in the pattern fragment while section 4.6 goes beyond the pattern fragment. Section 4.7 discusses the implementation in ELPI.

4.1 Problem statement and solution

Although the problem was first observed while working on ROCQ's Calculus of Inductive Constructions, we introduce a simplified setting to study it more easily. In this setting, we consider a language $\mathcal{O}$ with a rich equational theory, and a language $\mathcal{M}$ with a simpler one.

Both $\mathcal{O}$ and $\mathcal{M}$ are encoded in the ELPI language. The syntax of terms is shown in figure 4.1. Terms of $\mathcal{O}$ are called as $\texttt{tm}_o$, while the HOAS encoding of these terms in $\mathcal{M}$ are written as $\texttt{tm}_m$. These correspond, respectively, to object-level terms and meta-level terms.

The main difference between these two representations is in the way variables are encoded. In both cases, a variable is linked to a memory address. However, in $\mathcal{O}$, variables

```
kind tm_o  type.
type app_o  list tm_o  → tm_o .
type lam_o  (tm_o  → tm_o ) → tm_o .
type con_o  string → tm_o .
type uva_o  addr → tm_o .
```

```
kind tm_m  type.
type app_m  list tm_m  → tm_m .
type lam_m  (tm_m  → tm_m ) → tm_m .
type con_m  string → tm_m .
type uva_m  addr → list tm_m  → tm_m .
```

Figure 4.1: The $\mathcal{O}$ and $\mathcal{M}$ languages

are first-order objects and they do not carry any notion of scope. On the other hand, variables in $\mathcal{M}$ are equipped with an explicit scope, represented as a list of terms.

This scope is intended to capture the application of meta-level variables to terms. By contrast, the constructor $\texttt{app}_m$ encodes the application of $\mathcal{O}$ terms at the meta-language level.

This distinction can be seen in the example introduced in the previous section for the rule `tc-Provable`. The term `(Qm p)` is represented as $\texttt{uva}_m$ `Qm [p]`, while the term `{{lp:Q lp:p}}` is encoded as $\texttt{app}_m$ `[Q, p]`.

When we write $\mathcal{M}$ terms outside code blocks we follow the usual λ-calculus notation, reserving f, g, a, b for constants, x, y, z for bound variables and X, Y, Z, F, G, H for unification variables. However, we need to distinguish between the "application" of a unification variable to its scope and the application of a term to a list of arguments. We write the scope of unification variables in subscript while we use juxtaposition for regular application. Here are a few examples:

$f\ a$	$\texttt{app}_m$ `[`$\texttt{con}_m$ `"f", `$\texttt{con}_m$ `"a"]`
$\lambda x.\lambda y.F_{xy}$	$\texttt{lam}_m$ `x\ `$\texttt{lam}_m$ `y\ `$\texttt{uva}_m$ `F [x, y]`
$\lambda x.F_x\ a$	$\texttt{lam}_m$ `x\ `$\texttt{app}_m$ `[`$\texttt{uva}_m$ `F [x], `$\texttt{con}_m$ `"a"]`
$\lambda x.F_x\ x$	$\texttt{lam}_m$ `x\ `$\texttt{app}_m$ `[`$\texttt{uva}_m$ `F [x], x]`

When it is clear from the context, we shall use the same syntax for $\mathcal{O}$ terms (although we never use subscripts for unification variables). We use s, s_1, ... for terms in $\mathcal{O}$ and t, t_1, ... for terms in $\mathcal{M}$.

4.1.1 Equational theories and unification

To reproduce the unification issue in Elpi when dealing with Rocq terms, we extend our setting with two main routines: one for unification, and one for testing equality between terms. The second routine is required to check that the substitution produced by the unification process indeed makes the two terms equal.

Term equality: $=_o$ and $=_m$ For both languages, we extend the equational theory from ground terms to the full language by adding reflexivity for unification variables. That is, a unification variable is equal to itself, but different from any other term. The corresponding implementation is given in figures 4.2 and 4.3.

The first four rules are shared by both equalities and define the standard congruence on terms. Because we use an HOAS encoding, these rules also account for α-equivalence. In addition, $=_o$ includes rules for η- and β-equivalence. This is because, in the object language, the term $\texttt{app}_o$ `[`$\texttt{lam}_o$ `x\ x, `$\texttt{con}_o$ `"3"]` and $\texttt{con}_o$ `"3"` are definitionally equal.

```
pred (=o) tmo , tmo →.
cono C =o cono C.
appo A =o appo B :- forall2 (=o) A B.
lamo F =o lamo G :- pi x\ x =o x => F x =o G x.        % rλλ
uvao N =o uvao N.
lamo F =o T :-                                        % rηl
  pi x\ beta T [x] (T' x), x =o x => F x =o T' x.
T =o lamo F :-                                        % rηr
  pi x\ beta T [x] (T' x), x =o x => T' x =o F x.
appo [lamo X|L] =o T :- beta (lamo X) L R, R =o T.    % rβl
T =o appo [lamo X|L] :- beta (lamo X) L R, T =o R.    % rβr
```

Figure 4.2: Equality implementation for $\mathcal{O}$

```
pred (=m) tmm , tmm →.
conm C =m conm C.
appm A =m appm B :- forall2 (=m) A B.
lamm F =m lamm G :- pi x\ x =m x => F x =m G x.
uvam N A =m uvam N B :- forall2 (=m) A B.
```

Figure 4.3: Equality implementation for $\mathcal{M}$

Both procedures use implication to assume that the fresh nominal constants introduced by `pi` are equal to themselves.

The main point in showing these equality tests is to remark that $=_m$ is much weaker than $=_o$, and to identify the four rules that need special treatment in the simulation of $\simeq_o$ in $\mathcal{M}$. The implementation of the predicate `beta` essentially computes the weak-head normal form of a term, its implementation is similar to the rule for the predicate `whd` in section 2.3.4.

Substitution: ρs and σt We reuse the notations for substitution from section 2.3.2: we write $\sigma = \{\ X \mapsto t\ \}$ for the substitution that assigns the term t to the variable X, we write σt for the application of the substitution to a term t, and $\sigma X = \{\ \sigma t \mid t \in X\ \}$ when X is a set of terms. We write $\sigma \subseteq \sigma'$ for substitution extension as in definition 2.3.1. We will use ρ for representing substitutions in $\mathcal{O}$, and σ for the ones in $\mathcal{M}$. In this section, we consider the substitution for $\mathcal{O}$ and $\mathcal{M}$ identical. We defer to section 4.2.1 a more precise description pointing out their differences.

Term unification: $\simeq_o$ vs. $\simeq_m$ As the signature of unify and matching in section 2.3.2, $\mathcal{M}$'s unification signature is:

```
pred (≃m) tmm , tmm , subst → subst.
```

We write $(t_1 \simeq_m t_2)\ \sigma \rightsquigarrow \sigma'$ when σt_1 and σt_2 unify with substitution σ'. σ' is a substitution extension of σ (see definition 2.3.1). We write $t_1 \simeq_m t_2 \rightsquigarrow \sigma'$ when the initial substitution σ is empty. We write $\mathcal{L}$ as the set of terms that are in the pattern fragment (definition 2.3.3).

The specification of a "good" unification procedure restricted to a domain $\mathcal{D}$ is the following:

Proposition 4.1.1 (Good unification)**.** *A good unification* $\simeq$ *for equality* $=$ *in the domain* $\mathcal{D}$ *satisfies the following properties:*

$$\text{Completeness}: \{\ t_1,\ t_2\ \} \subseteq \mathcal{D} \to t_1 \simeq t_2 \rightsquigarrow \rho \to \rho t_1 = \rho t_2 \tag{4.1}$$

$$\text{Correctness}: \{\ t_1,\ t_2\ \} \subseteq \mathcal{D} \to \rho t_1 = \rho t_2 \to \exists \rho', t_1 \simeq t_2 \rightsquigarrow \rho' \wedge \text{mgu}(\rho t_1, \rho t_2) = \rho' \tag{4.2}$$

when mgu *ensure that* ρ' *is the most general unifier between* t_1 *and* $_2$ *under the initial substitution* ρ*.*

The meta and (resp. object) language of choice are expected to provide an implementation of $\simeq_m$ (resp. $\simeq_o$) that is a good unification for $=_m$ (resp. $=_o$) in $\mathcal{L}$.

4.1.2 The problem: logic-program simulation

We simulate the execution, called *run*, of a sequence of unification problems, denoted $\mathbb{P}$, from the object language $\mathcal{O}$ into the meta-language $\mathcal{M}$. The sequence $\mathbb{P}$ is given as a list of *step*s. Each step $\mathbb{P}_p$ corresponds to the unification of two terms, $\mathbb{P}_{p_l}$ and $\mathbb{P}_{p_r}$. We write $\mathbb{P}_p = \{\ \mathbb{P}_{p_l}, \mathbb{P}_{p_r}\ \}$. A term s is said to belong to $\mathbb{P}$ if it appears either on the left-hand side or on the right-hand side of some unification step in $\mathbb{P}$.

The composition of these steps starting from the empty substitution ρ_0 produces the final substitution ρ_n, which is the result of the logic program execution.

$$\begin{aligned}
&\text{step}_o(\mathbb{P}, p, \rho) \rightsquigarrow \rho' \stackrel{def}{=\!=} (\mathbb{P}_{p_l} \simeq_o \mathbb{P}_{p_r})\ \rho \rightsquigarrow \rho' \\
&\text{run}_o(\mathbb{P}, n) \rightsquigarrow \rho_n \stackrel{def}{=\!=} \bigwedge_{p=1}^{n} \text{step}_o(\mathbb{P}, p, \rho_{p-1}) \rightsquigarrow \rho_p
\end{aligned}$$

In order to simulate the execution of run_o in $\mathcal{M}$, we start by converting each $\mathcal{O}$-term $s \in \mathbb{P}$ to a corresponding $\mathcal{M}$-term $t \in \mathbb{Q}$. We write this translation as $\langle s \rangle \rightsquigarrow (t, m, l)$. The implementation of the compiler is detailed in sections 4.2, 4.4 and 4.6, here we just point out that it additionally produces a variable map m and a list of links l. The final goal of this simulation in $\mathcal{M}$ is to generate a substitution which is a valid substitution for $\mathcal{O}$. Therefore, we need a variable map that connects unification variables in $\mathcal{M}$ to variables in $\mathcal{O}$ and is used to "decompile" the substitution. Decompilation is noted $\langle \sigma, m, l \rangle^{-1} \rightsquigarrow \rho$. Links are an accessory piece of information whose description is deferred to section 4.1.3.

Then we simulate each run in $\mathcal{O}$ with a run in $\mathcal{M}$ as follows:

$$\begin{aligned}
&\text{step}_m(\mathbb{Q}, p, \sigma, \mathbb{L}) \rightsquigarrow (\sigma'', \mathbb{L}') \stackrel{def}{=\!=} \\
&\qquad \sigma \mathbb{Q}_{p_l} \simeq_m \sigma \mathbb{Q}_{p_r} \rightsquigarrow \sigma' \wedge \text{progress}\ (\mathbb{L}, \sigma') \rightsquigarrow (\mathbb{L}', \sigma'') \\
&\text{run}_m(\mathbb{P}, n) \rightsquigarrow \rho_n \stackrel{def}{=\!=} \\
&\qquad \mathbb{Q} \times \mathbb{M} \times \mathbb{L}_0 = \{(t, m, l) \mid s \in \mathbb{P}, \langle s \rangle \rightsquigarrow (t, m, l)\} \\
&\qquad \bigwedge_{p=1}^{n} \text{step}_m(\mathbb{Q}, p, \sigma_{p-1}, \mathbb{L}_{p-1}) \rightsquigarrow (\sigma_p, \mathbb{L}_p) \\
&\qquad \langle \sigma_n, \mathbb{M}, \mathbb{L}_n \rangle^{-1} \rightsquigarrow \rho_n
\end{aligned}$$

By analogy with $\mathbb{P}$, we write $\mathbb{Q}_{p_l}$ and $\mathbb{Q}_{p_r}$ for the two $\mathcal{M}$ terms being unified at step p, and we write $\mathbb{Q}_p$ for the set $\{\ \mathbb{Q}_{p_l}, \mathbb{Q}_{p_r}\ \}$.

The step_m procedure performs the unification of the p^{th} problem in $\mathbb{Q}$ under a substitution σ and a list of links $\mathbb{L}$. It consists of two sub-steps: first, a call to the meta-language

unification procedure; second, a check of the progress condition on the set of links. This check compensates for the weaker equational theory supported by $\simeq_m$.

Since we simulate unification of $\mathcal{O}$ within $\mathcal{M}$, the purpose of run_m is to take a list of unification problems $\mathbb{P}$ and produce a substitution ρ_n for $\mathcal{O}$. This substitution is equivalent to the one returned by run_o when applied to $\mathbb{P}$.

The run_m procedure first compiles all terms in $\mathbb{P}$, then executes each step_m to compute a substitution σ_n, and finally decompiles σ_n to obtain ρ_n.

We claim:

Proposition 4.1.2 (Run equivalence)**.** $\forall \mathbb{P}, \forall n$, *if all term in* $\mathbb{P}$ *belong to* $\mathcal{L}$

$$\text{run}_o(\mathbb{P}, n) \rightsquigarrow \rho \wedge \text{run}_m(\mathbb{P}, n) \rightsquigarrow \rho' \to \forall s \in \mathbb{P}, \rho s =_o \rho' s$$

That is, the two executions give equivalent results if all terms in $\mathbb{P}$ are in the pattern fragment. Moreover:

Proposition 4.1.3 (Simulation fidelity)**.** *In the context of* run_o *and* run_m*, if all term in* $\mathbb{P}$ *belongs to* $\mathcal{L}$ *we have that* $\forall p \in 1 \ldots n$,

$$\text{step}_o(\mathbb{P}, p, \rho_{p-1}) \rightsquigarrow \rho_p \leftrightarrow \text{step}_m(\mathbb{Q}, p, \sigma_{p-1}, \mathbb{L}_{p-1}) \rightsquigarrow (\sigma_p, \mathbb{L}_p)$$

In particular, this property ensures that a *failure* in the $\mathcal{O}$ execution is reflected by a failure in $\mathcal{M}$ at the *same step*. We consider this aspect important in practice, as it guarantees a strong correspondence between execution traces. As a consequence, a user can debug a logic program written in $\mathcal{O}$ by inspecting its execution trace in $\mathcal{M}$. We also claim that run_m handles terms outside $\mathcal{L}$ in the following sense:

Proposition 4.1.4 (Fidelity recovery)**.** *In the context of* run_o *and* run_m*, if* $\rho_{p-1}\mathbb{P}_p \subseteq \mathcal{L}$ *(even if* $\mathbb{P}_p \notin \mathcal{L}$*) then*

$$\text{step}_o(\mathbb{P}, p, \rho_{p-1}) \rightsquigarrow \rho_p \leftrightarrow \text{step}_m(\mathbb{Q}, p, \sigma_{p-1}, \mathbb{L}_{p-1}) \rightsquigarrow (\sigma_p, \mathbb{L}_p)$$

In other words, if the two terms involved in a step re-enter $\mathcal{L}$, then step_m and step_o are again related, even if $\mathbb{P} \notin \mathcal{L}$ and hence proposition 4.1.3 does not apply. Indeed, the main difference between proposition 4.1.3 and proposition 4.1.4 is that the assumption of the former is purely static, it can be checked upfront. When this assumption is not satisfied, one can still simulate a logic program and have guarantees of fidelity if, at run time, decidability of higher-order unification is restored.

This property has practical relevance since in many logic programming implementations, including ELPI, the order in which unification problems are executed does matter. The simplest example is the sequence $F \simeq \lambda x.a$ and $F\,a \simeq a$: the second problem is not in $\mathcal{L}$ and has two unifiers, namely $\sigma_1 = \{\ F \mapsto \lambda x.x\ \}$ and $\sigma_2 = \{\ F \mapsto \lambda x.a\ \}$. If we execute the two unification problem in order, the first unification return σ_2, making the second problem re-enter $\mathcal{L}$.

4.1.3 The solution (in a nutshell)

The role of the compiler is to translate terms from the object language into terms of the meta-language.

In particular, a term s is compiled into a term t in which every "problematic" subterm e is replaced by a fresh unification variable h. Each such variable is equipped with an additional *link*, which encodes a suspended unification problem between h and e. In this way, the unification procedure $\simeq_m$ behaves well on t, since it no longer contradicts with $=_o$ in the way it would on the original problematic subterms.

We now give a more precise definition of what we call "problematic" and "well behaved". We use the symbol $\diamond$, which in modal logic means "possibly", since problematic terms are characterized by a certain uncertainty.

Definition 4.1.5 ($\diamond\beta$)**.** *$\diamond\beta$ is the set of terms of the form $F\,x_1 \dots x_n$ such that $x_1 \dots x_n$ are distinct names (of bound variables).*

An example of a $\diamond\beta$ term is the application $F\,x$. This term is problematic since the application node of its syntax tree cannot be used to justify a unification failure, i.e. by properly instantiating F, the term head constructor may become a λ, or a constant, or a name, or remain an application.

Definition 4.1.6 ($\diamond\eta$)**.** *$\diamond\eta$ is the set of terms $\lambda x.s$ such that $\exists \rho, \rho(\lambda x.s)$ is an η-redex.*

An example of a term s in $\diamond\eta$ is $\lambda x.\lambda y.F\,y\,x$ since the substitution $\rho = \{F \mapsto \lambda x.\lambda y.f\,y\,x\}$ makes $\rho s =_o \lambda x.\lambda y.f\,x\,y$, which is the eta-long form of f. This term is problematic since its leading λ abstraction cannot justify a unification failure against a constant f.

Definition 4.1.7 ($\diamond\mathcal{L}$)**.** *$\diamond\mathcal{L}$ is the set of terms of the form $F\,t_1 \dots t_n$ such that $t_1 \dots t_n$ are not distinct names.*

These terms are problematic for the very same reason terms in $\diamond\beta$ are, but they cannot be handled directly by the unification of the meta language, which is only required to handle terms in $\mathcal{L}$. Still, given $s \in \diamond\mathcal{L}$ there may exist a substitution ρ such that $\rho s \in \mathcal{L}$.

We write $\mathcal{P}(t)$ for the set of sub-terms of t, and we write $\mathcal{P}(X) = \bigcup_{t \in X} \mathcal{P}(t)$ when X is a set of terms.

Definition 4.1.8 (Well behaved set)**.** *Given a set of terms X,*

$$\mathcal{W}(X) \leftrightarrow \forall t \in \mathcal{P}(X), t \notin (\diamond\beta \ \cup \ \diamond\eta \ \cup \ \diamond\mathcal{L})$$

We write $\mathcal{W}(t)$ as a short for $\mathcal{W}(\{t\})$. We claim our compiler validates the following properties:

Proposition 4.1.9 ($\mathcal{W}$-enforcing)**.** *Given a term s in $\mathcal{O}$,*

$$\langle s \rangle \rightsquigarrow (t, m, l) \to \mathcal{W}(t)$$

In other words, proposition 4.1.9 states that any term s in $\mathcal{O}$ is compiled into a term t in $\mathcal{M}$ that satisfies $\mathcal{W}$, even when s itself does not satisfy $\mathcal{W}$.

Proposition 4.1.10. *Given two terms s_1 and s_2, if $\exists \rho, \rho s_1 =_o \rho s_2$, then*

$$\langle s_1 \rangle \rightsquigarrow (t_1, m_1, l_1) \wedge \langle s_2 \rangle \rightsquigarrow (t_2, m_2, l_2) \to t_1 \simeq_m t_2 \rightsquigarrow \sigma$$

This last property states that if two terms can be unified in the source language, then their compiled forms can also be unified in the target language. Note that a compiler that maps every term to a unification variable would satisfy proposition 4.1.10, but would clearly fail to meet proposition 4.1.3.

Proposition 4.1.11 ($\mathcal{W}$-preservation)**.** $\forall \mathbb{Q}\ \mathbb{L}\ p\ \sigma\ \sigma'$,

$$\mathcal{W}(\sigma\mathbb{Q}) \wedge \sigma\mathbb{Q}_{p_l} \simeq_m \sigma\mathbb{Q}_{p_r} \rightsquigarrow \sigma' \to \mathcal{W}(\sigma'\mathbb{Q})$$
$$\mathcal{W}(\sigma\mathbb{Q}) \wedge \text{progress}(\mathbb{L}, \sigma) \rightsquigarrow (\_, \sigma') \to \mathcal{W}(\sigma'\mathbb{Q})$$

Proposition 4.1.11 is central for proving propositions 4.1.2 and 4.1.3. Informally, it states that the problematic subterms set aside by the compiler are not reintroduced, neither by $\simeq_m$ nor by the progress relation. As a consequence, each step_m, whose definition relies on these two notions, produces terms that still belong to $\mathcal{W}$.

In the next sections of this chapter, namely sections 4.3, 4.4 and 4.6, we explain how the compiler recognizes terms in $\diamond\beta$, $\diamond\eta$, and $\diamond\mathcal{L}$, respectively. We also focus on how progress handles links, and how it preserves $\mathcal{W}$ while ensuring propositions 4.1.2 to 4.1.4.

We prove proposition 4.1.3 in sections 4.4 to 4.6 and proposition 4.1.2 only in section 4.3, where links are not present yet. In order to prove proposition 4.1.2 in the presence of links one needs to refine proposition 4.1.3 by relating the two substitutions ρ_p and σ_p: they are equivalent only by using decompilation to take into account terms in $\mathbb{L}$. Then, by induction on the number of steps, proposition 4.1.3 implies proposition 4.1.2. We discuss proposition 4.1.4 in section 4.6.

4.2 Basic compilation and simulation

In this section, we present our encoding in ELPI for memory management, and explain how variables in $\mathcal{O}$ and $\mathcal{M}$ are assigned, allocated, and mapped from one language to the other. At the end of the section, we introduce a first simple compiler translating terms from $\mathcal{O}$ to $\mathcal{M}$.

4.2.1 Memory map ($\mathbb{M}$) and substitution (ρ and σ)

Unification variables are identified by a (unique) memory address. The memory and its associated operations are described below:

```
typeabbrev (memory A) (list (option A)).
pred set? addr, memory A → A.
pred unset? addr, memory A →.
pred assign addr, memory A, A → memory A.
pred new memory A → addr, memory A.
```

If a memory cell is `none`, then the corresponding unification variable is unset. The operation `assign` sets an unset cell to a given value, while `new` selects the first unused address and initializes it with `none`.

Since each unification variable in $\mathcal{M}$ appears together with a scope, its assignment must be abstracted over that scope. This allows the same assignment to be instantiated in different scopes. This abstraction is captured by the `inctx` container, in particular by its binding constructor `abs`.

```
kind inctx type → type.
type abs (tm_m  → inctx A) → inctx A.
type val A → inctx A.
typeabbrev assignment (inctx tm_m ).
typeabbrev subst (memory assignment).
```

For example, in a context where y is a bound variable and X a unification variable, the unification $X_y \simeq_m \lambda z.y$ yields the assignment $\{X_y \mapsto \lambda z.y\}$, which corresponds in Elpi concrete syntax to `abs y\ val (lam_m  z\ y)`.

Intuitively, the `abs` node represents abstraction in the meta-language, while `lam`$_m$ encodes object-level abstraction in the HOAS of $\mathcal{M}$, in the same way `uva`$_m$ represents application of the meta-language, and `app`$_m$ encodes object-level applications in the HOAS of $\mathcal{M}$.

In contrast, assignments for variables in $\mathcal{O}$ are represented as plain terms. In particular, `fsubst` is an abbreviation for `memory tm`$_o$. In this setting, the unification $X\,y \simeq_o \lambda z.y$ produces the assignment $\{X \mapsto \lambda yz.y\}$, which corresponds to `lam`$_o$ `y\ lam`$_o$ `z\ y` in Elpi concrete syntax.

The compiler defines a mapping between variables of the two languages, so as to preserve the correspondence between variables in $\mathcal{O}$ and variables in $\mathcal{M}$. This mapping is represented in the following code snippet.

```
kind ovariable type.
type ovar addr → ovariable.
kind mvariable type.
type mvar addr → arity → mvariable.
kind mapping type.
type (↦) ovariable → mvariable → mapping.
typeabbrev mmap (list mapping).
```

The variable mapping store is called `mmap` and consists of a list of `mapping`. Each mapping relates variables of the object language (`ovariable`) with variables of the meta-language (`mvariable`). We associate a number to each `mvariable` in order to record its arity.

Invariant 4.2.1 (Unification-variable arity)**.** *Each variable `A` in $\mathcal{M}$ has a (unique) arity `N` and each occurrence `(uva`$_m$ `A L)` is such that `L` has length `N`.*

```
pred m-alloc ovariable, mmap, subst → mvariable, mmap, subst.
m-alloc Ov M S Mv M S :- mem M (Ov ↦ Mv), !.
m-alloc Ov M S Mv [Ov ↦ Mv|M] S1 :- Mv = mvar N Ag, new S N S1.
```

Figure 4.4: Malloc code

The predicate `m-alloc` allocates a fresh $\mathcal{M}$ variable while preserving the arity information carried by its argument `Ag`. In particular, `Ag` should be understood as an input-output parameter: the predicate does not choose an arbitrary arity, but creates a variable whose arity is fixed by the current compilation phase, see for example figure 4.5a.
When a single `ovariable` occurs multiple times with different numbers of arguments, the compiler generates multiple mappings for it, on a first approximation, and then ensures the mapping is bijective by introducing η-`link`; this detail is discussed in section 4.5.

It is worth examining the code of the application of substitution to terms in $\mathcal{M}$. This is called `deref` and implements definition 2.3.2. Notice how assignments are moved to the

current scope, i.e. the `abs`-bound variables are renamed with the names in the scope of the unification variable occurrence.

```
pred deref subst, tm_m  → tm_m .
deref _ (con_m  C) (con_m  C).
deref S (app_m  A) (app_m  B) :- map (deref S) A B.
deref S (lam_m  F) (lam_m  G) :-
  pi x\ deref S x x => deref S (F x) (G x).
deref S (uva_m  N L) R :- set? N S A,
  move A L T, deref S T R.
deref S (uva_m  N A) (uva_m  N B) :- unset? N S,
  map (deref S) A B.

pred move assignment,
        list tm_m  → tm_m .
move (abs Bo) [H|L] R :-
  move (Bo H) L R.
move (val A)  [] A.
```

Note that `move` relies crucially on invariant 4.2.1, which ensures that all occurrences of a unification variable are applied to argument lists of the same length. As a consequence, these occurrences share the same simple type at the meta-level, and the number of `abs` nodes in the resulting assignment matches this length.

4.2.2 Links ($\mathbb{L}$)

As mentioned in section 4.1.3, the compiler replaces terms in $\diamond\eta$, $\diamond\beta$, and $\diamond\mathcal{L}$ with fresh variables linked to the problematic terms. Terms in $\diamond\beta$ do not need a link since $\mathcal{M}$ variables faithfully represent the problematic term thanks to their scope.

```
kind baselink type.
type (=η)  tm_m  → tm_m  → baselink.
type (=L) tm_m  → tm_m  → baselink.
typeabbrev links (list (inctx baselink)).
```

The right-hand side of a link, the problematic term, can occur under binders. To accommodate this situation, the compiler wraps `baselink` using the `inctx` container.

Invariant 4.2.2 (Link left hand side)**.** *The left-hand side of a suspended link is a variable.*

New links are suspended by construction. If the left-hand side is assigned during a step, then the link is considered for progress and possibly eliminated. This is discussed in section 4.4 and section 4.6.

In the examples we represent links as equations under a context. The equality sign is subscripted with the kind of `baselink`. For example $x \vdash A_x =_{\mathcal{L}} F_x\ a$ corresponds to:

```
abs x\ val (uva_m  A [x] =L app_m  [uva_m  F [x], con_m  "a"])
```

```
pred comp tm_o , mmap, links, subst → tm_m , mmap, links, subst.
comp (con_o  C) M1 L1 S1 (con_m  C) M1 L1 S1.          % r_c
comp (lam_o  F) M1 L1 S1 (lam_m  G) M2 L2 S2 :-        % r_λ
  comp-lam F M1 L1 S1 G M2 L2 S2.
comp (uva_o  A) M1 L1 S1 (uva_m  B []) M2 L1 S2 :-     % r_v
  m-alloc (ovar A) M1 S1 (mvar B 0) M2 S2.
comp (app_o  A) M1 L1 S1 (app_m  B) M2 L2 S2 :-        % r_@
  fold6 comp A M1 L1 S1 B M2 L2 S2.
```

(a) Recursive compilation of terms

```
pred comp-lam (tm_o  → tm_o ), mmap, links, subst →
              (tm_m  → tm_m ), mmap, links, subst.
comp-lam F M1 L1 S1 G M2 L3 S2 :-
  pi x y\ (pi M L S\ comp x M L S y M L S) =>
    comp (F x) M1 L1 S1 (G y) M2 (L2 y) S2,
  close-links L2 L3.
```

(b) Compilation of λ-abstractions

```
pred close-links (tm_m  → links) → links.
close-links (v\[X v|L v]) [abs X|R] :- close-links L R.
close-links (_\[]) [].
```

(c) Close links

```
pred compile tm_o , mmap, links, subst → tm_m , mmap, links, subst.
compile F M1 L1 S1 G M2 L2 S2 :-
  beta-normal F F', comp F' M1 L1 S1 G M2 L2 S2.
```

(d) Entry point

Figure 4.5: First-order compiler

4.2.3 Compilation

The simple compiler described in this section, and shown in figure 4.5, serves as a basis for the extensions presented in sections 4.3, 4.4 and 4.6. Its entry point (4.5d) first performs β-normalisation of the input term s in $\mathcal{O}$, and then compiles s into its corresponding HOAS representation in $\mathcal{M}$.

Together with the term s, the compiler is initially given a memory map, a list of links, and a substitution. These three components act as accumulators and are progressively updated during the compilation phase.

The auxiliary predicate `comp`, shown in figure 4.5a, performs the recursive traversal of the $\mathcal{O}$ term. Its current implementation is partial and will be extended in the following sections to handle problematic subterms. At this stage, the list of links is left unchanged. Rules r_c and $r_@$ in figure 4.5a implement the straightforward translation between the two term representations.

Rule r_v handles variables in the object language. A variable v_o in $\mathcal{O}$ is mapped to a variable in $\mathcal{M}$ using the memory map. If v_o is not yet present in the variable mapping, a fresh entry is created and the substitution is updated with a pointer for this new variable. This pointer is initially set to `none`. If, instead, v_o already occurs in the mapping and is associated with a meta-language variable v_m in $\mathcal{M}$, then v_m is simply returned. All these operations are delegated to the `m-alloc` procedure, which performs the actual allocation.

The λ-case is handled by rule r_λ, which invokes the auxiliary function `comp-mlam`. This procedure recursively calls `comp` in an extended context containing two local variables, `x` and `y`, representing respectively abstractions at the object and meta-level.

Finally, the auxiliary function `close-links` recursively wraps the abstractions in the links in an additional `abs` node binding. In this way, links generated deep inside the compiled term can be lifted outside their original binding context.

Lemma 4.2.3 ($\mathcal{W}$-partial-enforcement)**.** $\forall s, \langle s \rangle \rightsquigarrow (t, m, l)$*, if* $\mathcal{W}(s)$ *then* $\mathcal{W}(t)$*.*

Proof. By the definition of `comp`, the compilation procedure is a simple mapper from terms in $\mathcal{O}$ to terms in $\mathcal{M}$. By hypothesis, $\mathcal{W}(s)$, therefore $\mathcal{W}(t)$. □

4.2.4 Execution

A step in $\mathcal{M}$ consists in unifying two terms and reconsidering all links for progress. If either of these tasks fails, we consider the entire step to fail. It is at this granularity that we can relate steps in the two languages.

```
pred step_m  tm_m , tm_m , links, subst → links, subst.
step_m  T1 T2 L1 S1 L2 S3 :-
  (T1 ≃_m T2) S1 S2,
  progress L1 S2 L2 S3.
```

Reconsidering links is a fixpoint process because the progress of a link can update the substitution, which may then enable another link to progress.

```
pred progress links, subst → links, subst.
progress L S L3 S3 :-
  progress1 L L1 S S1,
```

```
occur-check-links L1,
scope-check L1 L2 S1 S2
if (L = L2, S = S2) (L3 = L2, S3 = S2)
   (progress L2 S2 L3 S3).
```

Progress In the base compilation scheme, `progress1` is the identity function on both the links and the substitution, so the fixpoint trivially terminates. Sections 4.4 and 4.6 add rules to `progress1` and explain why they do not hinder termination.

Occur check Since compilation moves problematic terms out of the sight of $\simeq_m$, that procedure can only perform a partial occur check. For example, the unification problem $X \simeq_m f\ Y$ cannot generate a cyclic substitution alone, but should be disallowed if $\mathbb{L}$ contains a link like $\vdash Y =_\eta \lambda z.X_z$: we don't know yet if Y will feature a lambda in head position, but we surely know it contains X. The procedure `occur-check-links` is in charge of performing this check that is needed in order to guarantee proposition 4.1.3.

Scope check The predicate `scope-check` replaces each link of the form $\Gamma \vdash X_{x_1 \dots x_n} =_\eta t$ with a link $x_1 \dots x_n \vdash X_{x_1 \dots x_n} =_\eta t'$ whenever $\{x_1 \dots x_n\} \subseteq \Gamma$. The term $t' = \lambda x.Y_{x_1 \dots x_n\ x}$ is obtained after executing $\lambda x.Y_{x_1 \dots x_n\ x} \simeq_m t$ using a fresh Y. This unification ensures that t' cannot contain variables in $\Gamma / \{x_1 \dots x_n\}$, and if it fails then progress hence step_m fails. In turn this grants fidelity in the case where the lhs of an η`-link` is pruned of a name that occurs (rigidly) in t: links represent suspended unification problems *that may succeed*.

4.2.5 Substitution decompilation

Decompiling the substitution involves three steps.

First and foremost, problematic terms stored in $\mathbb{L}$ have to be moved back into the game: a suspended link must be turned into a valid assignment. This operation is possible thanks to invariant 4.2.2, and that no link causes an occur-check (section 4.2.4) and the fact that $\mathbb{L}$ is duplicate-free (see section 4.5).

The second step allocates in the memory of $\mathcal{O}$ new variables used to implement the pruning of higher-order variables. For example, $F\,x\,y = F\,x\,z$ requires allocating a variable G in order to express the assignment $F_{ab} \mapsto G_a$.

The final step is to decompile each assignment in the $\mathcal{M}$ substitution. The `val` node carries a term that is easy to decompile since $\mathbb{M}$ is a bijection. Since the $\texttt{lam}_o$ node carries no additional information (other than the function body), each `abs` node can be decompiled to a $\texttt{lam}_o$ one.

However, if the $\texttt{lam}_o$ node was carrying more information, as is the case for Rocq where it holds the type of the bound variable, one would need to store this piece of information in the memory map, where we store the arity (that is a very simple function type).

Proposition 4.2.4 (Compilation round trip)**.** *If* $\langle s \rangle \rightsquigarrow (t, m, l)$ *and* $l \in \mathbb{L}$ *and* $m \in \mathbb{M}$ *and* $\sigma = \{A \mapsto t\}$ *and* $X \mapsto A^n \in \mathbb{M}$ *then* $\langle \sigma, \mathbb{M}, \mathbb{L} \rangle^{-1} \rightsquigarrow \rho$ *and* $\rho X =_o \rho s$.

The proposition above states that the translation from $\mathcal{O}$ to $\mathcal{M}$ is not lossy: it can be reversed. The statement is made somewhat convoluted by the fact that we hide the procedure for decompiling a term and instead state the property in terms of decompiling substitutions.

4.2.6 Definition of $\simeq_o$ and its properties

We already have all the ingredients to show how to implement the simulation of $\simeq_o$ in the meta-language.

```
pred (≃o) tmo , tmo  → fsubst.
(A ≃o B) F :-
  compile A [] [] [] A' M1 L1 S1,
  compile B M1 L1 S1 B' M2 L2 S2,
  stepm  A' B' L2 S2 L3 S3,
  decompile M2 L3 S3 [] F.
```

So far the compiler is very basic. It does not really enforce that the terms passed to step_m are in $\mathcal{W}$, and indeed makes no use of the higher-order capabilities of the meta language (all generated variables have an empty scope). Still, we can prove that $\simeq_o$ is a good "first-order" unification algorithm if the input already happens to be in $\mathcal{W}$. Later, when the compiler will ensure proposition 4.1.9, the proof will be refined to cover for the new cases.

Theorem 4.2.5 (Properties of $\simeq_o$ in $\mathcal{W}$)**.** *The implementation of $\simeq_o$ above is a good unification for $=_o$ (as per proposition 4.1.1) in the domain $\mathcal{W}$.*

Proof sketch. In this setting, $=_m$ is as strong as $=_o$ on ground terms. What we have to show is that whenever two different $\mathcal{O}$ terms can be made equal by a substitution ρ, we can produce this ρ by finding a σ via $\simeq_m$ on the corresponding $\mathcal{M}$ terms and by decompiling it. If we look at the syntax of $\mathcal{O}$, the only interesting case is $\texttt{uva}_o$ `X` $\simeq_o$ `s`. In this case, after compilation, we have $\texttt{uva}_m$ `Y []` $\simeq_m$ `t` that succeeds with $\sigma = \{Y \mapsto t\}$ and σ is decompiled to $\rho = \{X \mapsto s\}$ by proposition 4.2.4. □

Theorem 4.2.6 (Fidelity in $\mathcal{W}$)**.** *Proposition 4.1.2 and proposition 4.1.3 hold if $\mathcal{W}(\mathbb{P})$.*

Proof sketch. Since `progress1` is a no-op, step_o and step_m coincide. By theorem 4.2.5, the unification relation $\simeq_m$ is equivalent to $\simeq_o$ on terms in $\mathcal{W}$. □

4.2.7 Notational conventions

In the following sections we use the following notation to represent the behavior of the compiler. $\mathbb{P}$ represent the input unification problems in $\mathcal{O}$ while $\mathbb{Q}$ their corresponding compiled version with memory mapping $\mathbb{M}$ and links $\mathbb{L}$. For example:

$$\begin{array}{lllllllll}
\mathbb{P} = \{ & p_1 & \simeq_o & p_2 & & p_3 & \simeq_o & p_4 & \} \\
\mathbb{Q} = \{ & t_1 & \simeq_m & t_2 & & t_3 & \simeq_o & t_4 & \} \\
\mathbb{M} = \{ & X_1 \mapsto A_1^n & & X_2 \mapsto A_2^m & & & & & \} \\
\mathbb{L} = \{ & \Gamma \vdash a =_\eta b & & & & & & & \}
\end{array}$$

We index each sub-problem, sub-mapping, and sub-link with its position starting from 1 and counting from left to right. For example, $\mathbb{Q}_2$ corresponds to the $\mathcal{M}$ problem $t_3 \simeq_m t_4$.

As we deal with each category of problematic terms we weaken the assumptions of theorem 4.2.6 using the following definitions:

Definition 4.2.7 ($\mathcal{W}_\beta$)**.** $\mathcal{W}_\beta(X) \leftrightarrow \forall t \in \mathcal{P}(X), t \notin (\diamond\eta \cup \diamond\mathcal{L})$

Definition 4.2.8 ($\mathcal{W}_{\beta\eta}$)**.** $\mathcal{W}_{\beta\eta}(X) \leftrightarrow \forall t \in \mathcal{P}(X), t \notin \diamond\mathcal{L}$

4.3 Handling of $\diamond\beta$

The basic compiler given in the previous section is unable to make the following higher-order unification problem succeed.

$$\begin{array}{lllll} \mathbb{P} = \{ & \lambda x.(f\ (X\ x)\ a) & \simeq_o & \lambda x.(f\ x\ a) & \} \\ \mathbb{Q} = \{ & \lambda x.(f\ (A\ x)\ a) & \simeq_m & \lambda x.(f\ x\ a) & \} \\ \mathbb{M} = \{ & X \mapsto A^0 & & & \} \end{array}$$

The unification problem $\mathbb{Q}_1$ fails while trying to unify $A\ x$ and x, which is equivalent to $\texttt{app}_m\ \ \texttt{[uva}_m\ \ \texttt{A [], x]}$ versus `x`. In order to exploit the higher-order unification algorithm of the meta language, we compile the $\mathcal{O}$-term $X\ x$ into the $\mathcal{M}$-term A_x.

We add the following rule before rule $r_@$ in figure 4.5a, where `pattern-fragment` is a predicate checking if a list of terms is a list of distinct names.

```
comp (app_o  [uva_o  A|Ag]) M1 L S1 (uva_m  B Bg) M2 L S2 :-
  pattern-fragment Ag, !,
    fold6 comp Ag M1 L S1 Bg M1 L S1,
    len Ag Arity,
    m-alloc (ovar A) M1 S1 (mvar B Arity) M2 S2.
```

Note that compiling `Ag` cannot create new mappings nor links, since `Ag` is made of bound variables, and the hypothetical rule in figure 4.5b loaded by `comp-mlam` grants this property.

Decompilation No change to `commit-link` since no link is added.

Progress No change to `progress` since no link is added.

Lemma 4.3.1 (Properties of $\simeq_o$ in $\mathcal{W}_\beta$)**.** *Given the updated compilation scheme, the $\simeq_o$ of section 4.2.6 is a good unification (as per proposition 4.1.1) for $=_o$ in the domain $\mathcal{W}_\beta$.*

Proof sketch. If we look at the $\mathcal{O}$ terms, there is one more interesting case, namely $\texttt{app}_o\ \ \texttt{[uva}_o\ \ \texttt{X|W]}\ \simeq_o\ \ \texttt{s}$ when `W` is a list of distinct names compiled to $\vec{w}$. In this case the $\mathcal{M}$ problem is $Y_{\vec{w}} \simeq_m t$ that succeeds with $\sigma = \{Y_{\vec{y}} \mapsto t[\vec{w}/\vec{y}]\}$, which in turn is decompiled to $\rho = \{Y \mapsto \lambda\vec{y}.s[\vec{w}/\vec{y}]\}$. Thanks to rule r_{β_l} in figure 4.2 we have $(\lambda\vec{y}.s[\vec{w}/\vec{y}])\ \vec{w} =_o s$. □

Lemma 4.3.2 ($\mathcal{W}$-partial-enforcement)**.** $\forall s, \langle s\rangle \rightsquigarrow (t, m, l)$*, if* $\mathcal{W}_\beta(s)$ *then* $\mathcal{W}(t)$*.*

Proof sketch. We need to extend the proof lemma 4.2.3, since we need to consider the case of terms in $\diamond\beta$. Terms in $\diamond\beta$ are captured by the new rule for `comp` at the beginning of this section. The compiled term is a variable and its arguments are distinct names, therefore the returned term is also in $\mathcal{W}_\beta$. □

In other words the compiler eliminates all subterms that are in $\diamond\beta$.

Theorem 4.3.3 (Fidelity in $\mathcal{W}_\beta$)**.** *Proposition 4.1.2 and proposition 4.1.3 hold if* $\mathcal{W}_\beta(\mathbb{P})$*.*

Proof sketch. Thanks to lemma 4.3.2, $\simeq_m$ is as powerful as $\simeq_o$ in $\mathcal{W}_\beta$, as well as in $\mathcal{W}$ by theorem 4.2.6. □

4.4 Handling of $\diamond\eta$

From section 2.1, a term $\lambda x.t\,x$ is said to be the η-redex of t if x does not occur free in t, and conversely we say that t is the η-contraction of $\lambda x.t\,x$. The equational theory of $\mathcal{O}$ identifies these terms, but the current compilation scheme does not, as shown by the following example: while $\lambda x.X\,x \simeq_o f$ does admit the solution $\rho = \{\ X \mapsto f\ \}$, the corresponding problem in $\mathbb{Q}$ does not.

$$\begin{array}{lrcl} \mathbb{P} = \{ & \lambda x.(X\,x) & \simeq_o & f \ \} \\ \mathbb{Q} = \{ & \lambda x.A_x & \simeq_m & f \ \} \\ \mathbb{M} = \{ & X \mapsto A^1 & & \ \} \end{array}$$

The reason is that $\texttt{lam}_m$ `x\ uva`$_m$ `A [x]` and $\texttt{con}_m$ `"f"` start with different term constructors.

In order to guarantee proposition 4.1.2, we detect lambda abstractions that can disappear by η-contraction (section 4.4.1), and we modify the compiler so that it generates fresh unification variables in their place and delegates the problematic term t to $\mathbb{L}$ (section 4.4.2) by replacing t with a unification variable. The compilation of the problem $\mathbb{P}$ above is refined to:

$$\begin{array}{lrcl} \mathbb{P} = \{ & \lambda x.(X\,x) & \simeq_o & f \ \} \\ \mathbb{Q} = \{ & A & \simeq_m & f \ \} \\ \mathbb{M} = \{ & X \mapsto B^1 & & \ \} \\ \mathbb{L} = \{ & \vdash A =_\eta \lambda x.B_x & & \ \} \end{array}$$

As per invariant 4.2.2 the η-`link` left-hand side is a variable while the right-hand side is a term in $\diamond\eta$ that has the following property:

Invariant 4.4.1 (η-`link` rhs)**.** *The rhs of any η-`link` has the shape $\lambda x.t$ and t is in $\mathcal{W}$.*

Each η-`link` is kept in the link store $\mathbb{L}$ during execution and is activated under specific conditions, in particular, when invariant 4.2.2 or invariant 4.4.1 are no more true, or the rhs is for sure a term that cannot be η-contracted. Activation is implemented in section 4.4.3 by extending the `progress1` predicate defined in section 4.2.4.

4.4.1 Detection of $\diamond\eta$

When compiling a term t, we need to determine if any subterm s of t that is of the form $\lambda x.r$ can be an η-redex, i.e. if there exists a substitution ρ such that $\rho(\lambda x.r) =_o r'$ and r' is not a λ-abstraction. The detection of lambda abstractions that can "disappear" is not as trivial as it may seems. Here a few examples:

$$\begin{array}{lll} \lambda x.f\,(A\,x) & \in \diamond\eta & \rho = \{\ A \mapsto \lambda x.x\ \} \\ \lambda x.f\,(A\,x)\,x & \in \diamond\eta & \rho = \{\ A \mapsto \lambda x.a\ \} \\ \lambda x.f\,x\,(A\,x) & \notin \diamond\eta & \\ \lambda x.\lambda y.f\,(A\,x)\,(B\,y\,x) & \in \diamond\eta & \rho = \{\ A \mapsto \lambda x.x\ ;\ B \mapsto \lambda y.\lambda x.y\ \} \end{array}$$

The first two examples are straightforward, and illustrate how a unification variable can either expose or hide a bound name in its scope, thereby turning the resulting term into an η-redex or not.

The third example shows that when a variable occurs outside the scope of a unification variable, it cannot be erased and may therefore prevent the term from being an η-redex.

The last example highlights the recursive nature of the check we need to implement. The term begins with a spine of two λ-abstractions, so the whole term belongs to $\diamond\eta$ if and only if the inner term $\lambda y.\, f\,(A\,x)\,(B\,y\,x)$ also belongs to $\diamond\eta$. If this is the case, it could η-contract to $f\,(A\,x)$, making $\lambda x.\, f\,(A\,x)$ a potential η-redex.

We now describe how terms in $\diamond\eta$ are detected.

Definition 4.4.2 (*may-contract-to*)**.** *A β-normal term s* may-contract-to *a name x if there exists a substitution ρ such that $\rho s =_o x$.*

Lemma 4.4.3. *A β-normal term $s = \lambda x_1 \dots x_n.t$* may-contract-to *$x$ only if one of the following three conditions holds:*

1. *$n = 0$ and $t = x$;*
2. *t is the application of x to a list of terms l and each l_i* may-contract-to *x_i (e.g. $\lambda x_1 \dots x_n.x\,x_1 \dots x_n =_o x$) ;*
3. *t is a unification variable with scope W, and for any $v \in \{x, x_1 \dots x_n\}$, there exists a $w \in W$, such that w* may-contract-to *v (if $n = 0$ this is equivalent to $x \in W$).*

Proof sketch. Since our terms are in β-normal form the only rule that can play a role is r_{η_l} in figure 4.2, and note that that rule uses `beta` to add an argument to an application or to the scope of a variable. If the term s is not exactly x (case 1) it can only be an η-redex of x, or a unification variable that can be assigned to x, or a combination of both. If s begins with a lambda, then the lambda can only disappear by η contraction. In that case, the term t under the spine of binders $x_1 \dots x_n$ can either be x applied to terms that *may-contract-to* these variables (case 2), or a unification variable that can be assigned to the application $x\,x_1 \dots x_n$ (case 3). □

Definition 4.4.4 (*occurs-rigidly*)**.** *A name x* occurs-rigidly *in a β-normal term t, if for any substitution ρ, x is a subterm of ρt*

In other words, x *occurs-rigidly* in t if it occurs in t outside of the scope of a unification variable X; otherwise, an instantiation of X can make x disappears from t. Note that η-contracting t cannot make x disappear, since x is not a locally bound variable inside t.

We can now describe the implementation of $\diamond\eta$ detection:

Definition 4.4.5 (*maybe-eta*)**.** *Given a β-normal term $s = \lambda x_1 \dots x_n.t$,* maybe-eta *$s$ holds if any of the following holds:*

1. *t is a constant c or a name y applied to the arguments $l_1 \dots l_m$ such that $m \geq n$ and for every i such that $m - n < i \leq m$ the term l_i* may-contract-to *x_i, and no x_i* occurs-rigidly *in $l_1 \dots l_{m-n}$ y;*
2. *t is a unification variable with scope W and for each x_i there exists a $w_j \in W$ such that w_j* may-contract-to *x_i.*

Lemma 4.4.6 ($\diamond\eta$ detection)**.** *If t is a β-normal term and $t \in \diamond\eta$ then* maybe-eta *t holds.*

```
comp (lam_o  F) M1 L1 S1 (uva_m  A Scope) M2 L3 S3 :-
  maybe-eta (lam_o  F) [], !,
  new S1 A S2,                          % allocates a fresh variable A in S1
  comp-lam F M1 L1 S2 G M2 L2 S3,
  get-scope (lam_m  G) Scope,            % returns the set of free names in lam_m
  L3 = [val (uva_m  A Scope =_η lam_m  G) | L2].
```

Figure 4.6: Compilation of $\diamond\eta$ terms

Proof sketch. Follows from definition 4.4.4 and lemma 4.4.3 □

Remark that the converse of lemma 4.4.6 does not hold: *maybe-eta* performs an over-approximation of terms that are in $\diamond\eta$. There exists, in fact, a term t satisfying the criteria (1) of definition 4.4.5 that is not in $\diamond\eta$, i.e. there exists no substitution ρ such that ρt is an η-redex. A simple counter example is $\lambda x.f\ (A\ x)\ (A\ x)$ since x does not *occur-rigidly* in the first argument of f, and the second argument of f *may-contract-to* x. In other words $A\ x$ may either use or discard x, but our analysis does not take into account that *the same term* cannot have two contrasting behaviors.

As we will see in the rest of this section, this is not a problem since it does not break proposition 4.1.2 nor proposition 4.1.3.

4.4.2 Compilation and decompilation

Compilation In order to take into account the compilation of $\diamond\eta$ terms, we insert the rule in figure 4.6 just before rule r_λ in figure 4.5a.

Whenever a term of the form `lam`$_o$ `F` is detected to belong to $\diamond\eta$, it is compiled into `lam`$_m$ `G` and replaced by a fresh variable `A`. This variable sees all names that are free in `lam`$_m$ `G`, and it is linked to `lam`$_m$ `G` through a η-`link`. Invariant 4.2.2 is satisfied for this link. Moreover:

Corollary 4.4.7. *The rhs of any η-`link` has exactly one lambda abstraction, hence the rule above respects invariant 4.4.1.*

Proof sketch. By contradiction, suppose that the rule above is applied and that the rhs of the link is $\lambda x.\lambda y.t$, where x and y occur in t. If *maybe-eta* $\lambda y.t$ holds then the recursive call to `comp` (made by `comp-mlam`) must have put a fresh variable in its place, so this case is impossible. Otherwise, if *maybe-eta* $\lambda y.t$ does not hold, then also *maybe-eta* $\lambda x.\lambda y.t$ does not hold either, contradicting the assumption that the rule was applied. □

Lemma 4.4.8 ($\mathcal{W}$-partial-enforcement)**.** $\forall s, \langle s\rangle \rightsquigarrow (t, m, l)$*, if* $\mathcal{W}_{\beta\eta}(s)$ *then* $\mathcal{W}(t)$*.*

Proof sketch. We need to extend the proof of lemma 4.4.8, since we need to consider the case of terms $\diamond\eta$. By lemma 4.4.6, we know that any $\diamond\eta$ term is captured by *maybe-eta*, therefore the rule in figure 4.6 is used. The output is a variable and its scope is a list of distinct variables, therefore, the new terms is $\mathcal{W}$. □

Decompilation The decompilation of a η-`link` is performed by unifying the lhs with the rhs. Note that this unification never fails, since the lhs is a flexible term not appearing in any other η-`link` (by definition 4.4.10).

4.4.3 Progress

η-`link`s are meant to delay the unification of "problematic" terms until we know for sure if their head lambdas can (or cannot) be η-contracted.

Definition 4.4.9 (η-*progress-lhs*)**.** *A link* $\Gamma \vdash X =_\eta t$ *is removed from* $\mathbb{L}$ *when* X *becomes instantiated in a substitution* σ*. Let* $y \in \Gamma$*. There are two cases:*

1. *if* $\sigma X = \lambda x.s$ *we call "run* $(X \simeq_m t)\ \sigma$*";*
2. *if* $\sigma X = a$ *or* $\sigma X = y$ *or* $\sigma X = f\ a_1 \dots a_n$ *we unify the* η*-redex of* X *with* t*, that is we call "run* $(\lambda x.X\ x \simeq_m t)\ \sigma$*".*

The idea behind η-*progress-lhs* is that the link is triggered whenever the suspended variable of the $\diamond\eta$ link is assigned a term. In this situation, we need to simulate either the r_{η_l} rule or the $r_{\lambda\lambda}$ rule from figure 4.2, depending on whether the head constructor of the term σX is a λ-abstraction.

In the first case, the head of the term σX is a λ-abstraction. Therefore, we simulate the $r_{\lambda\lambda}$ rule by calling the unification procedure on the two terms. In the second case, the term t must be an η-redex. To simulate the unification performed in the object language by r_{η_l}, we η-expand the term σX and then unify the two terms using $\simeq_m$.

Definition 4.4.10 (η-*progress-deduplicate*)**.** *A link* $\Gamma \vdash X_{\vec{s}} =_\eta T$ *is removed from* $\mathbb{L}$ *when another link* $\Delta \vdash X_{\vec{r}} =_\eta T'$ *is in* $\mathbb{L}$*. By invariant 4.2.1 the length of* $\vec{s}$ *and* $\vec{r}$ *is the same; hence we can move the term* T' *from* Δ *to* Γ *by renaming its bound variables, i.e.* $T'' = T'[\vec{r}/\vec{s}]$*. We then run* $T \simeq_m T''$ *(under the context* Γ*).*

Proof sketch. By lemma 4.4.8. □

Lemma 4.4.11. *Given a* η-`link` $\Gamma \vdash X =_\eta \lambda x.t$*, the unification done by* η-progress-lhs *is between terms in* $\mathcal{W}$*.*

Proof sketch. Let σ be a substitution such that $\mathcal{W}(\sigma X)$ (since $\simeq_m$ preserves $\mathcal{W}$). By invariant 4.4.1, we have $\mathcal{W}(t)$. If $\sigma X = \lambda x.r$, then r is unified with t and both terms are in $\mathcal{W}$. Otherwise, $\lambda x.X\ x$ is unified with $\lambda x.t$, and $X\ x$ and t are both in $\mathcal{W}$. □

Lemma 4.4.12. *The unification done by* η-progress-deduplicate *is between terms in* $\mathcal{W}$*.*

Proof. Given two η-`link`s $\Gamma_1 \vdash X =_\eta \lambda x.t$ and $\Gamma_2 \vdash Y =_\eta \lambda x.t'$ the unification is performed between t and t'. By lemma 4.4.8 both terms are in $\mathcal{W}$. □

Lemma 4.4.13. *The progress of* η-`link` *guarantees proposition 4.1.11*

Proof sketch. By lemmas 4.4.11 and 4.4.12, every unification performed by the activation of a η-`link` is between terms in $\mathcal{W}$. □

Lemma 4.4.14. `progress` *terminates.*

Proof sketch. Rules definition 4.4.9 and definition 4.4.10 remove one link from $\mathbb{L}$, hence they cannot be applied indefinitely. Moreover each rule only relies on terminating operations such as $\simeq_m$, η-contraction, η-redex, relocation (a recursive copy of a finite term). □

Theorem 4.4.15 (Fidelity in $\mathcal{W}_{\beta\eta}$)**.** *Given a list of unification problems $\mathbb{P}$ such that $\mathcal{W}_{\beta\eta}(\mathbb{P})$, the introduction of η-`link` guarantees proposition 4.1.3.*

Proof sketch. We want to prove that step_o succeeds iff step_m succeeds, where step_m is made by a unification step u and a link progression p. Without loss of generality we consider a unification problem in $\mathcal{O}$ of the form $s_1 \simeq_o s_2$ where $s_1 \in \Diamond\eta$ and is compiled to a variable X with the accessory link $\Gamma \vdash X =_\eta \lambda x.t_1$. The unification u always succeeds, since X is a fresh variable, we only have to consider the link progression p. Two cases should be analyzed:

> $s_2 \in \Diamond\eta$, in this case, the unification problem becomes $X \simeq_m Y$ with the extra link $\vdash Y =_\eta \lambda x.t_2$. After the unification of X and Y, *η-progress-deduplicate* is fired and, once the lambdas are crossed, t_1 and t_2 are unified. This unification succeeds iff $s_1 \simeq_o s_2$ succeeds by theorem 4.3.3.
>
> $s_2 \notin \Diamond\eta$. In this case, s_2 is compiled to t_2, and the unification step u assigns t_2 to X, thereby triggering *η-progress-lhs*. If $s_1 \simeq_o s_2$ succeeds, then either r_{η_l} or $r_{\lambda\lambda}$ from figure 4.2 is applied. In the former case, the progression p unifies $\lambda x.t_1$ with the η-redex of t_2, namely $\lambda x.t_2\ x$. The link progression therefore attempts to unify t_1 with $t_2\ x$. Since both terms belong to $\mathcal{W}_{\beta\eta}$, they unify in $\mathcal{M}$ by theorem 4.3.3. In the latter case, t_2 has the form $\lambda x.t_2'$, where $t_2' \in \mathcal{W}_{\beta\eta}$. Hence, the link progression unifies t_1 with t_2', and again the unification succeeds by theorem 4.3.3.
>
> Conversely, if $s_1 \simeq_o s_2$ fails, then neither r_{η_l} nor $r_{\lambda\lambda}$ succeeds. Depending on the shape of t_2, *η-progress-lhs* attempts to unify t_1 either with t_2 or with its η-redex. In both cases, the unification is performed between terms in $\mathcal{W}_{\beta\eta}$. Since $s_1 \simeq_o s_2$ fails by hypothesis, the corresponding higher-order unification also fails by theorem 4.3.3.

□

Corollary 4.4.16 (Properties of $\simeq_o$ in $\mathcal{W}_{\beta\eta}$)**.** *Given the updated compilation scheme, procedure $\simeq_o$ of section 4.2.6 is a good unification (as per proposition 4.1.1) for $=_o$ in the domain $\mathcal{W}_{\beta\eta}$.*

Example of *η-progress-lhs* The example at the beginning of section 4.4, once $\sigma = \{\ A \mapsto f\ \}$, triggers *$\eta$-progress-lhs* since the link becomes $\vdash f =_\eta \lambda x.B_x$ and the lhs is a constant. This rule runs $\lambda x.f\ x \simeq_m \lambda x.B_x$, resulting in $\sigma = \{\ A \mapsto f\ ;\ B_x \mapsto f\ \}$. Since X is mapped to B and B_x is decompiled into $\lambda x.f\ x$, by the definition of η-`link` decompilation σ is decompiled to $\rho = \{\ X \mapsto \lambda x.f\ x\ \}$.

4.5 Making $\mathbb{M}$ a bijection

We want to allow for partial application of functions. As a consequence the same unification variable X can be used multiple times with different arities. When it is the case the memory map $\mathbb{M}$ generated by the compiler is not a bijection. For example:

$$\begin{array}{lllllllll}
\mathbb{P} = \{ & \lambda x.(X\ x) & \simeq_o & f & X & \simeq_o & \lambda x.a & \} \\
\mathbb{Q} = \{ & A & \simeq_m & f & C & \simeq_m & \lambda x.a & \} \\
\mathbb{M} = \{ & X \mapsto C^0 & X \mapsto B^1 & & & & & \} \\
\mathbb{L} = \{ & x \vdash A =_\eta \lambda y.B_y & & & & & & \}
\end{array}$$

in the unification problems $\mathbb{P}$ above, X is used with arity 1 in $\mathbb{P}_1$ and with arity 0 in $\mathbb{P}_2$. In order to preserve invariant 4.2.1 the compiler did generate two entries in $\mathbb{M}$ for X, namely B and C. However, there is no connection between these two variables and any incompatible assignments to them would not be detected, in turn breaking proposition 4.1.3. To address this issue, we post-process $\mathbb{M}$ to ensure the following property:

Proposition 4.5.1 ($\mathbb{M}$ is a bijection)**.** *After compilation, $\mathbb{M}$ defines a bijective correspondence between $\mathcal{O}$-variables in $\mathbb{P}$ and $\mathcal{M}$-variables in $\mathbb{Q}$. In particular, for every entry $X \mapsto A^n \in \mathbb{M}$, there is no distinct variable $X' \neq X$ such that $X' \mapsto A^n \in \mathbb{M}$, and no distinct variable $A'^n \neq A$ such that $X \mapsto A'^n \in \mathbb{M}$.*

Note that running $\simeq_m$ may require allocating new variables in $\mathcal{M}$ as explained in section 4.2.5. The property above ignores these variables since they are fresh and have no corresponding variable in $\mathcal{O}$.

The core procedure of this post-processing step is *align-arity* that is iterated by *map-deduplication*:

Definition 4.5.2 (*align-arity*)**.** *Given two mappings $m_1 : X \mapsto A^m$ and $m_2 : X \mapsto C^n$ where $m < n$ and $d = n - m$,* align-arity m_1 m_2 *generates the following d links, one for each i such that $0 \leq i < d$,*

$$x_0 \dots x_{m+i} \vdash B^i_{x_0 \dots x_{m+i}} =_\eta \lambda x_{m+i+1}.B^{i+1}_{x_0 \dots x_{m+i+1}}$$

where B^i is a fresh variable of arity $m + i$, and $B^0 = A$ as well as $B^d = C$.

The intuition is that we η-expand the occurrence of the variable with lower arity to match the higher arity. Since each η-`link` can add exactly one lambda, we need as many links as the difference between the two arities.

Definition 4.5.3 (*map-deduplication*)**.** *Forall mappings $m_1, m_2 \in \mathbb{M}$ such that $m_1 : X \mapsto A^m$ and $m_2 : X \mapsto C^n$ and $m < n$ we remove m_1 from $\mathbb{M}$ and add to $\mathbb{L}$ the result of* align-arity m_1 m_2.

Theorem 4.5.4 (Fidelity with *map-deduplication*)**.** *Given a list of unification problems $\mathbb{P}$, such that $\mathcal{W}_{\beta\eta}(\mathbb{P})$, if $\mathbb{P}$ contains the same $\mathcal{O}$-variable used at different arities, then* map-deduplication *guarantees proposition 4.1.3*

Proof sketch. Let X be a $\mathcal{O}$-variable appearing in two different subterms s_1 and s_2. In s_1 variable X is applied to the list of terms $x_1 \dots x_m$ while, in s_2, it is applied to $y_1 \dots y_n$. Without loss of generality we take $m < n$. After *map-deduplication* we have two distinct $\mathcal{M}$-variables for X, say A and C, related by a chain $\vec{c}$ of η-`link`s of length $n - m$. We prove that if A and C are assigned respectively to the $\mathcal{W}$ terms $\lambda x_1 \dots x_p.t_1$ and $\lambda x_1 \dots x_q.t_2$, then a failure occurs iff these terms do not unify. By *η-progress-lhs*, the assignment of A activates p links of $\vec{c}$. In particular, when $p \geq n$, then the entire chain collapses and

unification is performed between $\lambda x_n \dots x_p.t_1$ and $\lambda x_1 \dots x_q.t_2$. By theorem 4.3.3 and since we work with $\mathcal{W}$ terms, this unification succeeds iff it does in $\mathcal{O}$. Otherwise if $p < n$ then we have to consider two cases. 1) If t_1 is a rigid term then t_2 must be the η-redex of t_1, implying $q = 0$. If it is not the case, then *η-progress-lhs* would eventually cause a failure. 2) If t_1 is a unification variable then `scope-check` guarantees that t_2 does not use $x_1 \dots x_p$ as free variables, causing otherwise a unification failure. After this check, $m - n - p$ links of $\vec{c}$ remain suspended, as we expect: decompilation is charged to wake all links in $\mathbb{L}$ and eventually relate t_1 with t_2. □

Example of *map-deduplication* If we take back the example given at the beginning of this section, *map-deduplication* produces the following result:

$$\begin{array}{lllllll}
\mathbb{P} = \{ & \lambda x.(X\ x) & \simeq_o & f & X & \simeq_o & \lambda x.a & \} \\
\mathbb{Q} = \{ & A & \simeq_m & f & C & \simeq_m & \lambda x.a & \} \\
\mathbb{M} = \{ & X \mapsto B^1 & & & & & & \} \\
\mathbb{L} = \{ & \vdash C =_\eta \lambda x.B_x & & x \vdash A =_\eta \lambda y.B_y & & & & \}
\end{array}$$

Note that $X \mapsto C^0$ disappears from $\mathbb{M}$ in favour of the auxiliary η`-link` $\vdash C =_\eta \lambda x.B_x$. The resolution of $\mathbb{Q}_1$ assigns f to A. This wakes up $\mathbb{L}_2$ by *η-progress-lhs*, assigning $f\ x$ to B_x. The resolution of $\mathbb{Q}_2$ instantiates C to $\lambda x.a$, which, in turn triggers $\mathbb{L}_1$ by *η-progress-lhs*. In turn, this performs $\lambda x.a \simeq_m \lambda x.(f\ x)$ that fails as expected.

4.6 Handling of $\Diamond\mathcal{L}$

As observed in [Nad01] it is worth handling terms in $\Diamond\mathcal{L}$ since, in practice, these terms often re-enter $\mathcal{L}$ at runtime.

In the following example problem $\mathbb{P}_2$, namely $X\ a \simeq_o a$, admits two different solutions: $\rho_1 = \{\ X \mapsto \lambda x.x\ \}$ and $\rho_2 = \{\ X \mapsto \lambda x.a\ \}$. The unification algorithm alone cannot chose the right one, since no solution is more general than the other, but for the first problem (P_1 below) the choice is obvious, and in turn this choice makes $\mathbb{P}_2 \subseteq \mathcal{L}$:

$$\begin{array}{lllllll}
\mathbb{P} = \{ & X & \simeq_o & \lambda x.a & (X\ a) & \simeq_o & a & \} \\
\mathbb{Q} = \{ & A & \simeq_m & \lambda x.a & (A\ a) & \simeq_m & a & \} \\
\mathbb{M} = \{ & X \mapsto A^0 & & & & & & \}
\end{array}$$

In order to support this scenario we have to improve a little our compiler and progress routines. In particular $\mathbb{Q}_1$ generates $\sigma = \{\ A \mapsto \lambda x.a\ \}$ making $\sigma\mathbb{Q}_2$ equal to $(\lambda x.a)\ a \simeq_m a$ that $\simeq_m$ cannot solve since it lacks rules r_{β_l} and r_{β_r} in figure 4.2.

To address this problem the compiler must recognize $\Diamond\mathcal{L}$ terms and replace them with fresh variables and generate a new kind of links that we call $\mathcal{L}$`-link`.

In addition to invariant 4.2.2, the term on the rhs of a $\mathcal{L}$`-link` has the following property:

Invariant 4.6.1 ($\mathcal{L}$`-link` rhs)**.** *The rhs of any $\mathcal{L}$`-link` has the shape $X_{s_1 \dots s_n}\ t_1 \dots t_m$ where X is a unification variable in $\mathcal{L}$ (with scope $s_1 \dots s_n$) and $t_1 \dots t_m$ is a list of terms such that $m > 0$ and t_1 is either a variable occurring in $s_1 \dots s_n$ or a term other than a variable.*

Note that the shape of such a rhs is $\texttt{app}_m\ \ \texttt{[uva}_m\ \ \texttt{X S | L]}$, where `S` is $\texttt{[s}_1\texttt{, ..., s}_n\texttt{]}$ and `L` is $\texttt{[t}_1\texttt{, ..., t}_m\texttt{]}$.

4.6.1 Compilation and decompilation

We insert this rule just before rule $r_@$ in figure 4.5a

```
comp (app_o  [uva_o  A|Ag]) M1 L1 S1 (uva_m  B Sc) M3 L3 S4 :- !,
  pattern-fragment-prefix Ag Pf Extra, new S1 B S2,
  len Pf Ar, m-alloc (ovar A) M1 S2 (mvar C Ar) M2 S3,
  fold6 comp Pf    M2 L1 S3 Pf1    M2 L1 S3,
  fold6 comp Extra M2 L1 S3 Extra1 M3 L2 S4,
  Beta = app_m  [uva_m  C Pf1 | Extra1],
  get-scope Beta Sc,                  % returns the set of free names of Beta
  L3 = [val (uva_m  B Sc =_L Beta) | L2].
```

The list `Ag` is split into two parts: `Pf`, which is the longest prefix of `Ag` such that `Pf` is a list of distinct names (it may be empty), and `Extra`, which is non-empty and satisfies `append Pf Extra Ag`.

The right-hand side of $\mathcal{L}$`-link` is the application of a fresh variable `C` whose scope includes all names in `Pf1` (the compilation of `Pf`).

The variable `B`, returned as the compiled term, is a fresh variable whose scope includes all free variables occurring in `Pf1` and `Extra1` (the compilation of `Extra`). This scope is constructed by the `get-scope` procedure, which returns the list of distinct names appearing in its first argument.

This construction enforces invariant 4.6.1.

Lemma 4.6.2 ($\mathcal{W}$-enforcement)**.** *$\forall s, \langle s \rangle \rightsquigarrow (t, m, l)$, then $\mathcal{W}(t)$.*

Proof sketch. By lemma 4.4.8, we already cover terms in $\Diamond\beta$ and $\Diamond\eta$. We now need to consider the case of a term s in the object language that is in $\Diamond\mathcal{L}$. This category of terms identifies variables that are applied to a list of non-distinct names; that is, the pattern fragment from definition 2.3.3 is not respected. The rule implemented above for the predicate `comp` handles this case by returning a variable in the meta-language whose scope is a list of distinct names, as ensured by the specification of `get-scope`. Therefore, the compiled term t is $\mathcal{W}$. □

Decompilation All $\mathcal{L}$`-link` should be solved before decompilation. If any $\mathcal{L}$`-link` remains in $\mathbb{L}$, decompilation fails.

4.6.2 Progress

We add the following rules to `progress1`. Let l be a $\mathcal{L}$`-link` of the form $\Gamma \vdash T =_{\mathcal{L}} X_{s_1 \ldots s_n}\, t_1 \ldots t_m$

Definition 4.6.3 ($\mathcal{L}$*-progress-refine*)**.** *Let σ be a substitution such that σt_1 is a name s not occurring in $s_1 \ldots s_n$. If $m = 1$, then l is removed and the lhs is unified with $X_{s_1 \ldots s_n}\ s$. If $m > 1$, then l is replaced by the link $\Gamma \vdash T =_{\mathcal{L}} Y_{s_1 \ldots s_n}\ s\, t_2 \ldots t_m$ (where Y is a fresh variable of arity $n+1$) and link $\Gamma \vdash X_{s_1 \ldots s_n} =_\eta \lambda x. Y_{s_1 \ldots s_n}\ x$ is added to $\mathbb{L}$.*

Definition 4.6.4 ($\mathcal{L}$*-progress-rhs*)**.** *Link l is removed from $\mathbb{L}$ if $X_{s_1 \dots s_n}$ is instantiated to a term t and $t\, t_1 \dots t_m$ β-reduces to a $t' \in \mathcal{L}$.*

Definition 4.6.5 ($\mathcal{L}$*-progress-fail*)**.** progress *fails when either*

there exists another link $l' \in \mathbb{L}$ with the same lhs as l; or

the lhs of l become rigid.

We relax this condition and accommodate for heuristics in section 4.6.3.

Finally we introduce the following η-`link` progress rule.

Definition 4.6.6 (η*-progress-rhs*)**.** *A link $\Gamma \vdash X =_\eta T$ is removed from $\mathbb{L}$ when either*

maybe-eta T *does not hold (anymore), in this case X is unified with T; or*

T η-contracts to a term T' not starting with the $\texttt{lam}_m$ constructor. In this case X is unified with T'

The idea behind this rule is to instantiate the lhs variable of a η-`link` whenever its rhs is for sure a $\mathcal{W}$ term.

Lemma 4.6.7. *`progress` terminates*

Proof sketch. Let $l = \Gamma \vdash T =_{\mathcal{L}} X_{s_1 \dots s_n}\, t_1 \dots t_m$ a $\mathcal{L}$-`link` in the store $\mathbb{L}$. The progression made by $\mathcal{L}$*-progress-rhs* terminates: l is removed from $\mathbb{L}$ after having performed terminating instructions. If l is not activated by $\mathcal{L}$*-progress-rhs*, then X is a variable. The activation of $\mathcal{L}$*-progress-refine* replaces l with the new $\mathcal{L}$-`link` $\Gamma \vdash T =_{\mathcal{L}} Y_{s_1 \dots s_n\ s}\, t_2 \dots t_m$. This progression can be triggered at most m times, since at each time the number of terms applied to X decreases. In the end l_m will be removed from $\mathbb{L}$ after the unification between lhs with rhs. We also note that each $\mathcal{L}$*-progress-refine* generates a η-`link`, yet, according to lemma 4.4.14, this does not affect termination. $\mathcal{L}$*-progress-fail* terminates since it stop progression with failure. Finally, η*-progress-rhs* performs terminating operations and, if its premises succeed, the considered η-`link` is removed from $\mathbb{L}$, ensuring termination. □

Theorem 4.6.8 (Fidelity in $\mathcal{O}$)**.** *The introduction of $\mathcal{L}$-`link` guarantees proposition 4.1.4 if $\mathbb{P} \notin \mathcal{L}$.*

Proof sketch. Let $\mathbb{P}_i$ be the first problem in $\mathbb{P}$ such that it is not in $\mathcal{L}$ and let σ be the substitution obtained solving $\mathbb{Q}_1 \dots \mathbb{Q}_{i-1}$. After the execution of step_m for the $i-1$ time, each variable X corresponding to a $\Diamond\eta$ subterm in the original $\mathcal{O}$ unification problem, is instantiated iff it is known for sure that X is no more in $\Diamond\eta$. If $\sigma\mathbb{Q}_i$ is in $\mathcal{L}$, then by definitions 4.6.3 and 4.6.4 the associated $\mathcal{L}$-`link` is solved and removed. In this case, all calls to $\simeq_m$ are between terms in $\mathcal{L}$ and by theorem 4.3.3 fidelity is guaranteed. If $\sigma\mathbb{Q}$ is still in $\Diamond\mathcal{L}$, then by definition 4.6.5 unification fails as the corresponding unification in $\mathcal{O}$ would (it is called outside its domain). □

4.6.3 Relaxing definition 4.6.5

Working with terms in $\mathcal{L}$ is sometime too restrictive [AP18] and we could find in the literature a few strategies to go beyond $\mathcal{L}$ without implementing Huet's algorithm [Hue75] which is a semi-decision procedure. Some implementations of λ-Prolog [NM88] such as Teyjus [Nad01] delay the resolution of $\diamond\mathcal{L}$ unification problems until the substitution makes them reenter $\mathcal{L}$. Other systems apply heuristics and commit to that solution. This is the case for the unification algorithm Rocq.

In this section we show how we can implement these strategies by simply adding (or removing) rules to the `progress` predicate. In the example below $\mathbb{P}_1$ is in $\diamond\mathcal{L}$.

$$\begin{array}{lllllllll}
\mathbb{P} = \{ & (X\,a) & \simeq_o & a & X & \simeq_o & \lambda x.Y & \} \\
\mathbb{Q} = \{ & A & \simeq_m & a & B & \simeq_m & \lambda x.C & \} \\
\mathbb{M} = \{ & Y \mapsto C^0 & X \mapsto B^0 & & & & & \} \\
\mathbb{L} = \{ & x \vdash A =_{\mathcal{L}} (B\,a) & & & & & & \}
\end{array}$$

If we want the object-language unification to delay the first unification problem (waiting for X to be be instantiated), we can relax definition 4.6.5. Instead of failing when the lhs of $\mathbb{L}_1$ becomes rigid (equal to a), we keep it in $\mathbb{L}$ until the head of its rhs also becomes rigid. In this case, since both the lhs and rhs have rigid heads, they can be unified. While this relaxed rule does not break proposition 4.1.3 per se, the `occur-check-links` procedure becomes incomplete since invariant 4.2.2 is broken. Also note that delaying unification outside $\mathcal{L}$ can leave $\mathcal{L}$`-link` for the decompilation phase. Therefore `commit-links` should be modified accordingly.

If instead we want $\simeq_o$ to follow the second strategy and pick an arbitrary solution we can modify progress by applying the desired heuristic instead of failing. For instance, in $X\,a\,b = Y\,b$, the last argument of the two terms is the same and unification can succeed by assigning $X\,a$ to Y. This heuristic is is used by Rocq's type-class resolution and vanilla tactics (like "apply"), through Rocq's "first-order" unification heuristic.

4.7 Actual implementation in Elpi

In this chapter we did study a compiler and a simulation loop on a minimal language. The actual implementation uses the Rocq-Elpi meta-language to both compile the sequence of problems (the rules) and execute them, that is Elpi plays the role of $\mathcal{M}$ as well.

The main difference is that we cannot implement run_m since it is the runtime of the programming language. In particular the runtime iterates $\simeq_m$, but step_m also needs to check links for progress. Luckily Elpi extends λ-Prolog with constraints (suspended goals) and Constraint Handling Rules (CHR) to operate on them (section 2.3.5). Similarly to [MP93] a constraint is a goal suspended on a list of variables that is resumed *as soon as one of these variable is assigned*, before any other existing goal is considered. In turn link activation grants proposition 4.1.3. We point out that [MP93] delays $\diamond\mathcal{L}$ unification problems while we also delay problems that are in $\diamond\eta$.

In the following snippet, we show how η`-link` is either suspended or allowed to make progress.

```
L =η R :- not (var L),        !, eta-progress-lhs L R.
```

```
L =η R :- not (maybe-eta R), !, eta-progress-rhs L R.
L =η R :- declare_constraint (L =η R) [L,R].
```

The first two clauses describe when progress can occur. If `L` is no longer a variable, the invariant 4.2.2 is violated; therefore, we trigger unification between `L` and `R`. Similarly, if `R` is no longer a $\Diamond\eta$ term, the invariant 4.4.1 does not hold anymore, and we again force unification of `L` and `R`.

If neither condition holds, no progress is possible, and the constraint is suspended again. Similar rules are implemented for $\mathcal{L}$`-link`.

```
rule (N1 ▷ G1 ?- uvar X LX1 =η T1)          % match
  /  (N2 ▷ G2 ?- uvar X LX2 =η T2)          % remove
  |  (relocate LX1 LX2 T2 T2')              % condition
 <=> (N1 ▷ G1 ?- T1 = T2').                 % new goal
```

In the snippet above, we illustrate the deduplication of η`-link`. The syntax `N ▷ G ?- P` denotes a λ-PROLOG sequent, that is a goal `P` under a set `G` of hypothetical rules (introduced by `=>`) and where the program context binds (via the `pi` operator) eigenvariables in `N`. Sequents are presented to CHR with their higher-order unification variables replaced by "frozen constants", that is A_{xyz} becomes `uvar` f_A `[x,y,z]` for a fresh constant f_A. Frozen constants are "defrost" when they are part of a new goal (see [GSCT19, section 4.3]). The first directive matches a constraint whose lhs is a variable `X`; the second matches and removes a constraint on the same variable; the third relocates `T2` and the latter crafts the new goal that unifies `T1` with `T2'` under `G1` and the set of eigenvariables `N1`.

4.8 Related work and conclusion

Object-language terms can be unified using different strategies.

The first approach that comes to mind consist in implementing $\simeq_o$ as a regular routine, i.e. write rules as follows

```
tc-Provable {{impl lp:F lp:B}} {{Pimpl lp:F lp:B lp:Q}} :-
  pi p \
    (tc-Provable {{fact lp:F}} p => tc-Provable B X),
    unify X {{lp:Q lp:p}}.
```

where `unify` is a regular predicate. This method fails to take advantage of the logic programming engine provided by the meta language since it removes all the data from the rule's heads and makes indexing degrade. Additionally, implementing a unification procedure in the meta language is likely to be significantly slower compared to the built-in one on the common domain of first order terms.

Another possibility is to avoid having application and abstraction nodes in the syntax tree, and use the meta language ones, as in:

```
tc-Provable ({{impl}} F B) ({{Pimpl}} F B Q) :-
  pi p \ tc-Provable ({{fact}} F) p => tc-Provable B (Q p).
```

However, this encoding has two big limitations. First it is not always feasible to adopt it for Rocq due to the fact that the type system of the meta language is too limited to accommodate the one of the object language, e.g. Rocq can typecheck variadic functions [Chl13].
Second, the encoding of Rocq terms provided by Elpi is primarily used for meta programming, i.e. to extend the Rocq system. Consequently, it must be able to manipulate terms that are not known in advance without relying on introspection primitives such as Prolog's `functor/3` and `arg/3`. To avoid that constants cannot be symbols of the meta language, but must rather live in an open world, akin to the `string` data type used in the $\texttt{con}_m$ constructor.

In the literature we could find a related encoding of the Calculus of Constructions (CC) [Fel93]. The goal of that work is to exhibit a logic program performing proof checking for CC and hence relate the proof system of intuitionistic higher-order logic (that animates λ-Prolog programs) with the one of CC. The encoding is hence tailored toward a different goal, for example it utilizes three relations to represent the equational theory of CC, and that choice alone makes things harder for us. Section 6 contains a discussion about the use of the unification procedure of the meta language in presence of non ground goals, but the authors are not interested in exploiting a decidable fragment of higher-order unification but are rather leaning towards an interactive system where the user is given control over the search procedure.

Another work that is, somewhat surprisingly, only superficially related to ours is the type-class engine built in Isabelle's meta language. In [Wen97] classes identify (simple) types, they are not higher order predicates as in Rocq, hence the solver does not require a higher-order unification procedure.

The approach proposed in this chapter addresses the issues discussed earlier. It exploits the unification mechanism of the meta-language, while handling problematic subterms separately when needed.

With this design, the encoding can benefit from indexing data structures and mode analysis for clause filtering. In particular, subterms are replaced by variables *only* when strictly necessary, so that most of the original structure is preserved and remains indexable.

Benchmarks on the Stdpp library [JKJ+18] show promising results. Our type-class solver introduces some overhead on small goals, but becomes consistently faster than the Rocq one for goals of size 32 nodes or more. The use of links also turns out to be useful in practice, as about 15% of the instances in Stdpp require them.

Another advantage is the flexibility of the approach, which allows different strategies and heuristics to deal with terms outside the pattern fragment.

Finally, the encoding remains sufficiently shallow to: (1) extend certain forms of static analysis from the meta-language, such as determinacy, to the object language; (2) benefit from future improvements in the logic programming engine of Elpi, for example tabled search.

We considered mechanizing our results in Abella [Gac08], in particular at an early stage of this work, but we were not able to complete it.

The main limitation is that the logic of Abella does not support quantification over predicates, nor the use of the cut operator. It is in principle possible to eliminate cut and higher-order combinators such as `map` or `forall2`, but this leads to a significant increase in verbosity, since these constructs are used pervasively.

More importantly, this transformation introduces a gap between the verified version and the actual implementation, which we found to hinder the development and exploration of the system.

Nevertheless, we established a test suite of about one hundred cases, which is available at https://github.com/FissoreD/ho-unif-for-free.

Chapter 5

Determinacy checking for Elpi

Over the years, Elpi has been adopted by many Rocq users. A commonly cited difficulty is the shift to a logic programming paradigm. While Rocq users are generally familiar with typed functional programming, they often have limited experience with backtracking and how to control it. When backtracking is not properly managed, it can lead to inefficient programs and make debugging more difficult.

This issue also appears in type class search, and it is a concern for developers working with type classes in Rocq. In some cases, a type class is expected to behave deterministically: the search for instances of this class should produce at most one solution when it succeeds. However, since there is no mechanism to verify statically this property, an inexperienced user may encounter long debugging traces caused by unintended non-determinism.

For instance, consider the following two instances of the same class:

```
Instance i1 A1 : Class1 A1 → DetClass A1.
Instance i2 A1 : Class2 A1 → DetClass A1.
```

If the class `DetClass` is intended to be deterministic, these two instances break that assumption. Under certain conditions, both `i1` and `i2` may, in fact, provide valid solutions, so a query to `DetClass` can return more than one result. A static checker could detect this situation and warn the user about the loss of determinism caused by the presence of multiple instances.

Moreover, one may want to keep both instances, but ensure that if the first one succeeds, the second is not considered. Expressing this kind of control over the search is not possible in Rocq. In contrast, Elpi allows this behavior by introducing a cut in the body of the first rule, which stops further backtracking and enforces the intended deterministic behavior.

More generally, unintended non-determinism can lead to undesirable effects. To reduce this issue, we propose a system of determinacy signatures for Elpi. This system allows programmers to specify when a predicate should behave like a function, that is, it produces at most one result.

This notion of determinism is referred to as *semidet* by Henderson et al. [HSC96], and as *operationally deterministic* by Nakamura [Nak86]. Other notions also exist. For example, Warren [DW89] defines predicates as *observably deterministic* when they produce the same result multiple times or diverge. In this work, we adopt operational determinacy,

as it closely matches the behavior of functions in mainstream functional programming languages.

Our determinacy checker covers ELPI's higher-order features, including higher-order predicates (for example map and fold, see section 5.1.1) and dynamic predicates that extend themselves at runtime (see section 5.1.2), and meta-programs that manipulate ELPI code (see section 5.1.3).

Our findings indicate that most ELPI programs are intended to be backtracking-free and require only minimal annotations for our static analyzer to verify determinacy.

The following sections are inspired by our paper [FT26]. In particular, section 5.1 motivates the need for determinacy in a logic programming language such as ELPI, while section 5.2 briefly reviews the syntax and semantics of ELPI, which are described in detail in sections 2.3.1 and 2.3.2. In sections 5.3 and 5.4, we present the core building blocks of determinacy checking, in particular the mutual-exclusion checker and the rule checker. Finally, in section 5.5, we establish the properties of our determinacy checker, while section 5.6 concludes with an experimental report.

5.1 Elpi and determinacy signatures by examples

As in most logic programming languages, ELPI programs are organized in rules. When rules are not *mutually exclusive*, i.e. multiple rules apply on the same query, the runtime continues the computation using the rule with highest priority. Moreover it stores the other rules as a *choice point* and can backtrack to that point to continue the computation. What makes backtracking tricky to ELPI users is that it is an opt-out feature of logic programming: it is left to the programmer to tell the runtime to commit to one rule and discard the choice point, rather than the other way around. If the user forgets to commit to a rule, his program still works but becomes much harder to debug and typically slower to run in case of failures.

The simplest example is the list-membership *relation* that can succeed more than once if the element we are looking for occurs multiple times in the list:

```
mem X [Y|_ ] :- X = Y.     % the head is the X we are looking for, or
mem X [_|YS] :- mem X YS. % recurse on the tail
```

The first rule for `mem` is the base case: it succeeds if the head of the list is equal to the `X` we are looking for. The second one discards the list's head and continues looking for `X` in the tail. A query such as `mem 2 [2,3,2]` can be solved in two different ways: either using rule 1 immediately, or using rule 2 twice to discard the first two items, and then conclude with rule 1.

ELPI users often have a background in functional programming and simply do expect `mem` to be a *function* testing membership, and not a relation. Moreover one needs to look, carefully, at how `mem` is implemented in order to see the peril. This misconception becomes a problem when they use `mem` in a larger piece of code and observe its execution.

```
main L :- code_before L, mem 1 L, code_after L.
```

For example, in the code above, one may see `code_after` being executed multiple times: each choice point left by `mem` enables the runtime to backtrack to that point in time and

continue from there. Even worse, if `code_after` is expensive and fails, one pays its cost multiple times.

It must be said that backtracking can be very useful in some situations and that a programmer with experience in logic programming can take advantage of that. We want the ELPI casual user to be able to write code without mastering all that, and simply declare that they expect *their code* to compute functions.

```
func main list int →. % the signature for main
```

The signature starts by a keyword declaring the determinacy of the predicate: `func` for deterministic predicates, that we also call functions, and `pred` for any predicate, including both functions and relations. Inputs are separated from outputs by the arrow symbol, and in the case of `main` there is no output.

```
main L :- code_before L, mem 1 L, code_after L.
%                        ^^^^^^ error: non-deterministic atom
```

Since `main` was declared as a function, our static analysis invites the user to discard the choice points left by `mem`, for example by inserting a `!` (a *cut* directive) after it. Assuming `code_after` is a function, this code is accepted, since the cut discards all choice points generated by `mem` and `code_before L`.

```
main L :- code_before L, mem 1 L, !, code_after L. % OK
```

Note that the programmer may also write a functional membership test as follows, and use it in functional code with no additional safeguard. In this case the cut discards the second rule as soon as `X = Y` succeeds.

```
func in int, list int →.
in X [Y|_ ] :- X = Y, !. % commit to this rule, discard the next one
in X [_|YS] :- in X YS.

main L :- code_before L, in 1 L, code_after L. % OK
```

Statically tracking the use of backtracking is a well-known technique in logic programming, usually referred to as determinacy analysis [DW89, SHC96]. However, the literature does not provide a detailed treatment of features that are specific to ELPI and not available in plain PROLOG, such as higher-order programming, dynamic programming, and meta-programming.

These features were introduced in section 2.3.4 and play an important role in determinacy. In the following three subsections, we revisit those examples and analyze them from the perspective of determinacy.

5.1.1 Higher-order programs

A notable higher-order program is the one turning relations into functions.

```
func once (pred) →. % a function taking a fully applied predicate
once P :- P, !.     % the ! commits to the first success
```

We say that the signature of the `once` predicate marks it as *unconditionally* a function: `once` leaves no choice points, no matter what it receives as the higher-order argument `P`.

An example of a *conditional* function is the predicate to map over a list:

```
func map (func A → B), list A → list B.
map _ [] [].
map F [X|XS] [Y|YS] :- F X Y, map F XS YS.
```

We say that the signature of `map` has a *precondition*, namely that `F` is a function, and a *postcondition*, namely that `map` is a function, i.e., the call `map F L L'` leaves no choice points. If the precondition is not met the postcondition is not guaranteed to hold and we say that `map` is *miscalled*. In this case we weaken its signature to*:

```
pred map (func A → B), list A → list B.
```

Our static analysis tracks miscalls to functions and treats them as relations when analyzing the surrounding code. This avoids code duplication, since only one definition of `map` is needed in the library. We illustrate this with four scenarios involving correct calls and miscalls to `map`.

```
% a function                        % a relation
func incr int → int.                pred get_pdiv int → int.
incr 1 2.                           get_pdiv 6 3. get_pdiv 6 2.

% four predicates calling map or miscalling map

func p list int → list int.
p L R :- map incr L R.              % OK

pred q list int → list int.
q L R :- map get_pdiv L R.          % OK

func r list int → list int.
r L R :- map get_pdiv L R.          % KO
           % ^^^^^^^^ error: non-deterministic atom

func r1 list int → list int.
r1 L R :- map get_pdiv L R, !.      % OK
```

In this example, `incr` is a function, while `get_pdiv` is a relation. In particular, `get_pdiv` associates more than one result to the input `6`, making it non-deterministic.

The predicate `p` is deterministic because it satisfies the precondition of `map`: the argument `incr` is a function, so the call does not introduce choice points.

The predicate `q` instead contains a miscall to `map`, since `get_pdiv` is a relation. As a result, the execution may leave choice points. However, this is accepted because `q` is declared as a relation (with `pred`), and therefore non-determinism is allowed.

The situation is different for `r`. Its body is the same as `q`, but `r` is declared as a function. For this reason, the static analysis reports an error: a deterministic predicate cannot contain a non-deterministic computation.

*We explain in detail this weakening operation in section 5.4

Finally, `r1` has the same signature as `r` and also miscalls `map`, but it explicitly removes choice points using a cut. This makes the overall behavior deterministic, and the definition is accepted by the analyzer. Similarly, an implementation such as `once (map get_pdiv L R)` would also be valid, since the `once` wrapper ensures that any potential choice points are discarded.

5.1.2 Dynamic programs

In section 2.3.4, we pointed out the interest of dynamic programming in a language like Elpi. In this paragraph, we point out why the predicate `copy` can be considered as a deterministic predicate.

```
func copy tm → tm.
copy (app A B) (app C D) :- copy A C, copy B D.
copy (lam F)   (lam G)   :- pi x\ copy x x => copy (F x) (G x).
```

`copy` is a function because no two rules can apply to the same input: `lam` and `app` are different constants, and each constant `x` generated dynamically is guaranteed to be fresh by the `pi` operator, hence each `copy x x` rule is mutually exclusive with the other rules.

5.1.3 Meta programs

The meta-program we have shown in section 2.3.4 is a good example of how our determinacy checker interacts well in program inspection and composition.

We take back the implementation for the predicates `comp` and `fuse`, but this time, we pay attention to their determinacy in the signatures.

```
func comp (func A → B), (func B → C), A → C.
comp F G X Z :- F X Y, G Y Z.

func fuse (func list A → list B), (func list B → list C) →
          (func list A → list C).
fuse (map F) (map G) (map (comp F G)).
```

The signature of `comp` informs the checker that `comp` behaves deterministically if its two higher-order arguments are functions. This piece of information is important when the checker analyses the rule for `fuse`.

The signature of `fuse` has two preconditions, both inputs should be functions, and two postconditions: `fuse` is a function and its output is also a function. The code of `fuse` inspects two programs: if both are mapping a list, respectively with `F` and `G`, then it generates a new program `map (comp F G)`. The static analysis is able to verify that this output is a function following this reasoning: by hypothesis, `map F` (resp `map G`) is a function, therefore, also `F` (resp `G`) must be, otherwise, we would have a miscall of `map`, contradicting the hypothesis claiming that the inputs of `fuse` are functions; from that we have that `comp F G` is a function and so is `map (comp F G)`.

5.2 Elpi's abstract syntax and semantics

Any static analysis aimed at ruling out non-determinism must take into account the operational semantics of the language. In particular, the order in which rules are applied,

as well as the order in which their premises are executed, directly affects the scope of the cut directive.

The ELPI syntax and semantics considered in this section are those presented in sections 2.3.1 and 2.3.2. To simplify the presentation, we treat `pi` and `=>` as a single operator. In this setting, the construct `pi` x. $r \Rightarrow a$ represents the introduction of a fresh symbol x, which is visible in both the rule r and the atom a. The local rule r is added to the scope of the atom a.

Using the combined form `pi` x. $r \Rightarrow a$ does not reduce the expressiveness of ELPI. The behavior of the `pi` operator can be simulated by introducing a dummy rule r in a, while the `=>` operator can be recovered by using a vacuous variable `x`.

We omit from the syntax the construction of data with binders, since it does not play a relevant role in the static analysis of determinacy.

We use the following notation: s ranges over symbol names (including predicates, data constructors, and variables), v over variables, k over predicate and data constructors, a and b over atoms, and r over rules.

In summary, the algorithm proceeds as follows:

1. it checks that each rule is mutually exclusive with the others in the program;
2. it verifies that the sequence of calls in the body consists of correctly used functional predicates;
3. it ensures that the preconditions of rules entail their postconditions.

5.3 Mutual Exclusion

Mutual exclusion is one of the two main ingredients of the static checker. The interesting aspects for this section are the cut operator and the matching/unify procedure depending on the modes of a predicate.

In the next definition we distinguish between global rules, i.e., rules that are part of the program $\mathcal{P}$, and local rules that are loaded via the `pi` operator. For example, the `copy` predicate from section 5.1.2 has two global rules and one local rule.

Definition 5.3.1 (Mutual exclusion (`mut_excl` $\mathbb{P}$ $\mathbb{R}$))**.** *Given a program $\mathcal{P}$ and a rule r, such that $r \notin \mathcal{P}$, we say that r is mutually exclusive with the rules in $\mathcal{P}$ if it has the highest priority and one of the following additional conditions holds:*
(a) there is a cut in the body of r, or
(b) r is a global rule and all rules r' in $\mathcal{P}$ have an input argument not unifying with the corresponding input in r
(c) r is a local rule introduced via `pi` x. $r \Rightarrow b$, x is an input of r and for all rules r' in the program, the corresponding input is not a variable

```
pred p int → int.        func q int → int.        func r int → int.
p 1 3.                   q 1 3.                   r X R :- p X R, !.
p 1 2.                   q 2 4.                   r X R :- q X R.
```

For example, the rules for predicate `p` above are not mutually exclusive, so choice points may remain after a query resolution, hence introducing non-determinism. For instance, `p 1 X` has two solutions, namely `X = 3` and `X = 2`.

Conversely, `q` satisfies mutual exclusion: no query can trigger both rules. Note that, since inputs are matched, a query like `q X Y` has no solution in Elpi.

Finally, both rules for `r` match any input. If the first rule succeeds, the cut removes the alternatives created by the call to `p` and also discards the other rule `r X R :- q X R.` If the first rule fails, the cut is not reached hence the second rule is not discarded. Thus, at most one rule can succeed for `q`.

The predicates `s` and `s1` below illustrate the role of `pi`.

```
func s int → int.                          func s1 int → int.
s X R :- pi y\ q y 3 => q X R.             s1 X R :- pi y\ q 1 5 => q X R.
                          % error: mutxecl violated  ^^^^^
```

The local rule `q y 3` is mutually exclusive with the rules for `q` above since `y` is fresh: each time the rule is loaded `y` is different from `1` and `2` and any other fresh constants generated before. On the contrary the local rule `q 1 5` is not mutually exclusive: the query `s1 1 R` has two results, namely `R = 5` and `R = 3`.

Note that condition *(c)* also rules out the following example, since the `pi`-quantified symbol `y` is not used in input position.

```
func s2 int → int.
s2 1 R :- pi y\ q 3 5 => s2 2 R.
s2 2 R :- pi y\ q 3 6 => q 3 R.
%                ^^^^^ error: mutxecl violated
```

Both rules for `s2` load a local rule for `q`. All these local rules are mutually exclusive with the global rules in the program, but the query `s2 2 R` has two results namely `R = 5` and `R = 6`.

5.3.1 Checking mutual exclusion: discrimination trees

This subsection describes the implementation of mutual exclusion checking in Elpi.

To support mutual exclusion checking in Elpi, we rely on the discrimination tree data structure [McC92], a standard mechanism for indexing rules in logic programming systems. Discrimination trees are widely used in Prolog-like systems to improve the efficiency of clause retrieval. Our implementation is partially inspired by the presentation in Pfenning's notes [†].

A discrimination tree is a trie-based structure. Inner nodes are labeled, while leaves store buckets of objects. Tries represent dictionaries by sharing common prefixes of keys. A *path* is defined as the sequence of labels encountered from the root to a leaf.

Two basic operations are supported: insertion and retrieval.

```
type retrieve trie → key → option elt.
type insert   trie → key → elt → unit.
```

To insert an element e with key k, the algorithm traverses the trie symbol by symbol, starting from the root. For each symbol, it follows the corresponding edge, creating a new node if necessary. Once all symbols have been consumed, the element e is stored in the corresponding leaf. Retrieval works similarly: the key is read symbol by symbol, and the

[†] https://www.cs.cmu.edu/~fp/courses/99-atp/lectures/lecture28.html

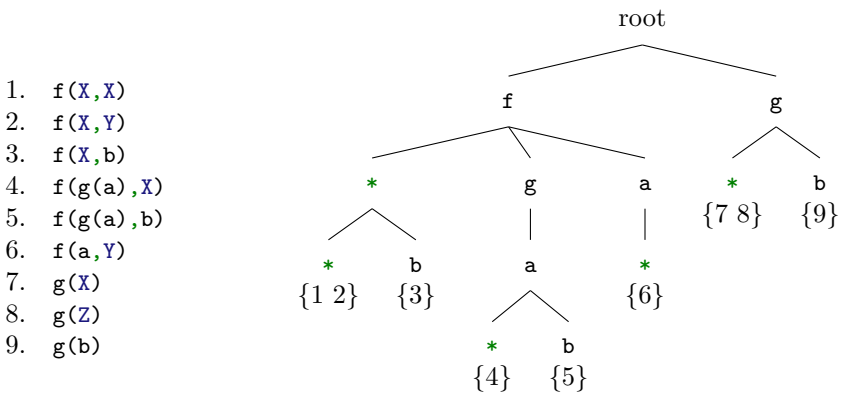


Figure 5.1: Example of a discrimination tree

tree is traversed accordingly. If the path exists, the elements stored in the corresponding leaf are returned.

In PROLOG systems, the indexed objects are the rules of the database, and the head of each rule serves as its key. In figure 5.1, taken from [McC92], the database (left) and its corresponding discrimination tree (right) are shown. Leaves are annotated with the rules stored in each bucket. The purpose of the tree is to enable fast clause retrieval.

Variables are represented approximately by wildcard symbols. In particular, they are encoded as generic symbols that match any term. For instance, in figure 5.1, rules 1 and 2 share the same path and are stored in the same bucket. Unlike the approach described in [McC92], retrieval in our setting must take modes into account. For example, if the predicate `g` has one input and one output argument, the query `g X Y` retrieves only rules 7 and 8. If both arguments are outputs, then rules 7, 8, and 9 are returned.

For efficiency, terms are encoded as sequences of integers, where each integer represents multiple pieces of information at the bit level. Modes are included in this encoding, allowing the system to determine whether a rigid term should be matched or unified during retrieval.

Unification in the discrimination tree is only an approximation of the full ELPI unification algorithm, which is more expensive. By construction, the set of rules returned by the tree is a superset of the rules that actually match the query. A second phase applies the full ELPI unification to filter out non-applicable rules.

We also optimize the encoding of query paths. In practice, query terms are often larger than the terms stored in the database, so encoding them entirely is not useful. Instead, we use a hit map that records which positions in the subterms can influence retrieval, and only encode them.

To avoid higher-order unification during retrieval, terms headed by a λ-abstraction are treated as variables. For example, `x\ X x` may unify with `id` or with `(x y\ true)`, but all such terms are encoded as variables in the tree. As a result, in our implementation of discrimination trees, the following program does not satisfy mutual exclusion:

```
func isfunc (func int → int) →.
isfunc id.
isfunc (x\ y\ x = y).
```

The argument of the second rule for the predicate `isfunc` contains a λ-abstraction, so the whole argument is encoded as a variable in the discrimination tree. To make the checker accept this program, a cut must be added to the body of the first rule, indicating that the second rule should not be considered once the first one succeeds.

Discrimination trees are effective for indexing rules in the ELPI database, making retrieval faster in many cases. In this chapter, however, their main role is to support mutual exclusion checking.

During compilation, an auxiliary discrimination tree is built. Each rule of a deterministic predicate is inserted into this structure. When a new rule is added, the tree is queried to retrieve the set of potentially overlapping rules r. By invariant, r is an ordered (possibly empty) list in which all rules, except possibly the last one, contain a cut. The mutual exclusion checker then verifies that either the new rule is inserted at the end of the list, or that it contains a cut. This ensures that any potentially overlapping rule is either the final alternative or explicitly commits to its choice.

5.4 Static analysis of determinacy signatures

The abstract syntax of signatures is given below. Predicate signatures $\mathbb{Ty}$ start with input arguments from the $\mathbb{S}_i$ non-terminal, continue with output arguments from the $\mathbb{S}_o$ non-terminal and end with a determinacy marker $\mathbb{D}$: `Fun` for functions and `Rel` for relations (respectively `func` and `pred` in the concrete syntax).

$$
\begin{array}{lll}
\mathbb{D} & ::= \texttt{Fun} \mid \texttt{Rel} & \text{determinacy} \\
\bigstar & ::= \star & \text{all data} \\
\mathbb{S}_i & ::= \mathbb{S}_o \mid \mathbb{S}_i \xrightarrow{\texttt{i}} \mathbb{S}_i \mid \bigstar \xrightarrow{\texttt{i}} \mathbb{S}_i & \text{signature start} \\
\mathbb{S}_o & ::= \mathbb{D} \mid \mathbb{S}_i \xrightarrow{\texttt{o}} \mathbb{S}_o \mid \bigstar \xrightarrow{\texttt{o}} \mathbb{S}_o & \text{signature stop} \\
\mathbb{Ty} & ::= \mathbb{S}_i & \text{signature}
\end{array}
$$

We collapse all data types into a single type $\bigstar$ and we disallow to store predicates into data, i.e., one cannot form a list of predicates. Note that the grammar of $\mathbb{Ty}$ forces all input arguments to come first. We reserve d for instances of type $\bigstar$, ρ and τ for signatures and $\mathcal{D}$ for instances of $\mathbb{D}$. For example the signature `func map (func A → B), list A → list B` corresponds to $(\star \xrightarrow{\texttt{i}} \star \xrightarrow{\texttt{o}} \texttt{Fun}) \xrightarrow{\texttt{i}} \star \xrightarrow{\texttt{i}} \star \xrightarrow{\texttt{o}} \texttt{Fun}$.

We denote booleans $\mathbb{B} = \{\top, \bot\}$; contexts Γ being partial maps from symbols to signatures. We use $\varnothing$ for the empty list and $x :: xs$ for prepending the element x to a list xs. We reserve Γ for contexts.

5.4.1 Operations on signatures

We compare signatures using the standard subtyping relation of functional languages. In particular, we compare inputs contravariantly and outputs covariantly. It is similar to [HP25], except for the absence of the `Any` terminal. `Fun` is smaller than `Rel` since functions generate a "smaller number of results" than relations.

$$
\begin{array}{llll}
\star \subseteq \star & \texttt{Fun} \subseteq \texttt{Fun} \quad \texttt{Rel} \subseteq \texttt{Rel} \quad \texttt{Fun} \subseteq \texttt{Rel} & & \text{base cases} \\
\rho \xrightarrow{\texttt{i}} \tau \subseteq \rho' \xrightarrow{\texttt{i}} \tau' & \text{iff} & \rho' \subseteq \rho \wedge \tau \subseteq \tau' & \text{contravariance} \\
\rho \xrightarrow{\texttt{o}} \tau \subseteq \rho' \xrightarrow{\texttt{o}} \tau' & \text{iff} & \rho \subseteq \rho' \wedge \tau \subseteq \tau' & \text{covariance}
\end{array}
$$

The static analysis needs often to keep the stronger or weaker signature among two. We hence derive from $\subseteq$ a `min` and `max` operators defined below.

$$\begin{array}{ll}
\texttt{min}\ \star\ \star = \star & \texttt{max}\ \star\ \star = \star \\
\texttt{min Rel Rel} = \texttt{Rel} & \texttt{max Rel Rel} = \texttt{Rel} \\
\texttt{min Rel Fun} = \texttt{Fun} & \texttt{max Rel Fun} = \texttt{Rel} \\
\texttt{min Fun Rel} = \texttt{Fun} & \texttt{max Fun Rel} = \texttt{Rel} \\
\texttt{min Fun Fun} = \texttt{Fun} & \texttt{max Fun Fun} = \texttt{Fun} \\
\texttt{min}\ (\rho \xrightarrow{\texttt{i}} \tau)\ (\rho' \xrightarrow{\texttt{i}} \tau') = \texttt{max}\ \rho\ \rho' \xrightarrow{\texttt{i}} \texttt{min}\ \tau\ \tau' & \texttt{max}\ (\rho \xrightarrow{\texttt{i}} \tau)\ (\rho' \xrightarrow{\texttt{i}} \tau') = \texttt{min}\ \rho\ \rho' \xrightarrow{\texttt{i}} \texttt{max}\ \tau\ \tau' \\
\texttt{min}\ (\rho \xrightarrow{\texttt{o}} \tau)\ (\rho' \xrightarrow{\texttt{o}} \tau') = \texttt{min}\ \rho\ \rho' \xrightarrow{\texttt{o}} \texttt{min}\ \tau\ \tau' & \texttt{max}\ (\rho \xrightarrow{\texttt{o}} \tau)\ (\rho' \xrightarrow{\texttt{o}} \tau') = \texttt{max}\ \rho\ \rho' \xrightarrow{\texttt{o}} \texttt{max}\ \tau\ \tau'
\end{array}$$

When a predicate is miscalled we need to weaken its signature.

We define `strong` and `weak` as follows:

$$\begin{array}{ll}
\texttt{strong}\ \star = \star & \texttt{weak}\ \star = \star \\
\texttt{strong Rel} = \texttt{Fun} \quad \texttt{strong Fun} = \texttt{Fun} & \texttt{weak Fun} = \texttt{Rel} \quad \texttt{weak Rel} = \texttt{Rel} \\
\texttt{strong}\ (\rho \xrightarrow{\texttt{i}} \tau) = \texttt{weak}\ \rho \xrightarrow{\texttt{i}} \texttt{strong}\ \tau & \texttt{weak}\ (\rho \xrightarrow{\texttt{i}} \tau) = \texttt{strong}\ \rho \xrightarrow{\texttt{i}} \texttt{weak}\ \tau \\
\texttt{strong}\ (\rho \xrightarrow{\texttt{o}} \tau) = \texttt{strong}\ \rho \xrightarrow{\texttt{o}} \texttt{strong}\ \tau & \texttt{weak}\ (\rho \xrightarrow{\texttt{o}} \tau) = \texttt{weak}\ \rho \xrightarrow{\texttt{o}} \texttt{weak}\ \tau
\end{array}$$

The set of signatures that only differ by a leaf in $\mathcal{D}$ form a lattice w.r.t. the order relation $\subseteq$, the join operator `max` and meet operator `min`. The infimum of the lattice is computed by `strong` and the supremum is computed by `weak`.

In section 5.1.1 we have weakened the signature of `map` from $\tau = (\star \xrightarrow{\texttt{i}} \star \xrightarrow{\texttt{o}} \texttt{Fun}) \xrightarrow{\texttt{i}} \star \xrightarrow{\texttt{i}} \star \xrightarrow{\texttt{o}} \texttt{Fun}$ to $(\star \xrightarrow{\texttt{i}} \star \xrightarrow{\texttt{o}} \texttt{Fun}) \xrightarrow{\texttt{i}} \star \xrightarrow{\texttt{i}} \star \xrightarrow{\texttt{o}} \texttt{Rel}$.

One could define the supremum of τ as $\tau' = (\star \xrightarrow{\texttt{i}} \star \xrightarrow{\texttt{o}} \texttt{Rel}) \xrightarrow{\texttt{i}} \star \xrightarrow{\texttt{i}} \star \xrightarrow{\texttt{o}} \texttt{Rel}$, which intuitively amounts to considering all `Fun` as `Rel`, i.e., dropping all `Fun` in case of error. However, this would contradict the lattice property: $\texttt{max}\ \tau\ (\texttt{weak}\ \tau) = \texttt{weak}\ \tau$, since we would have $\texttt{max}\ \tau\ \tau' = \tau'$, but $\tau \notin \tau'$. Moreover, since we compare input signatures using $\subseteq$, this alternative definition of `weak` (resp. `strong`) would not make the checker accept more programs.

5.4.2 The idea

The static analyzer accumulates knowledge on variables in a context Γ initially only containing the signature of predicates. The checking algorithm considers one rule at a time, following their priorities. It begins by updating Γ using the preconditions on the *inputs* of the head, then it scans all body atoms and finally checks the postconditions: the determinacy of the body and the determinacy of the *outputs* of the head.

For example, consider the predicates `comp` and `compose` below: `comp` is a function that takes as input two functions `F` and `G`, and calls them in sequence, whereas `compose` builds a new function from the two received in input.

```
func comp (func int -> int), (func int -> int), int -> int.
comp F G X Z :- F X Y, G Y Z.

func compose (func int -> int), (func int -> int) -> (func int -> int).
compose F G (comp F G).
```

At first, the checker assumes that the inputs of `comp` satisfy the preconditions given in its signature. In particular, `F` and `G` are marked as functions in Γ. Then the algorithm scans the atoms in the body. The determinacy of the whole rule is computed as the `max` of the determinacies of its premises. Since both `F X Y` and `G Y Z` have determinacy `Fun`, their `max` is also `Fun`. Finally, the analyzer verifies that the determinacy of the body is consistent with the one declared in the signature.

For `compose`, two conditions must be checked: the term `comp F G` must be a function, and `compose` itself must also be a function. The algorithm assumes again that `F` and `G` are functions. The term `comp F G` is a partial application, and it returns a function when its arguments are functions. Since this holds for `F` and `G`, the preconditions of `comp` are satisfied, and the result is a function. Finally, the body of `compose` is empty, so it is deterministic. Therefore, `compose` behaves deterministically.

We now move to an example that involves a miscall to a predicate, that is, a rule with a non-deterministic body atom.

```
func r (func int → int), (pred int → int), int → int.
r F G X Y :- map G [X] [Z], !, compose F F H, H Z Y.
```

After analyzing the head of the rule, the context asserts that `F` (resp. `G`) is a function (resp. a relation). In the body of `r`, the first atom is non-deterministic, since `G` violates the preconditions of `map`. The signature of `map G [X] [Z]` is hence `weak`ened to `Rel`. Even if this atom could generate multiple alternatives, the cut commits to the first one restoring the determinism of the body of `r` up to that point. The call to `compose` satisfies its preconditions, so its postconditions apply: the entire atom `compose F F H` is considered deterministic *and* `H` is marked as a function in Γ. The final atom `H Z Y` is also deterministic because of the knowledge about `H` accumulated so far in Γ.

5.4.3 The algorithm

We present the static analyzer using derivation rules, they are listed in figures 5.2 and 5.3. Each procedure has an associated entails-like symbol, e.g., context $\vdash^{\text{flag}}_{\text{name}}$ intake $\rightsquigarrow$ result. We use the equal sign to "mode" the rule, i.e., separate what is given to the rule from what is generated by it. The static analyzer is composed of seven procedures for a total of 24 rules.

The entry point is **check rule** (noted $\vdash_{\mathsf{r}}$, figure 5.2a). This procedure takes a program and a context of signatures, it verifies if a rule validates the signature ascribed to its predicate. The procedure stores in Γ all the knowledge available on variables and verifies that the precondition made by the signature entails the postcondition.

In particular, the **assume head input** procedure (noted $\vdash^{\mathsf{i}}_{\mathsf{hd}}$, figure 5.3a) loads into Γ_1 the information assumed from the head inputs. This procedure is applied under the assumption that the predicate is well-called. Hence, all variables that occur (recursively) in input positions are assumed to satisfy the determinacy specified in the signature.

On the other hand, the **check head output** procedure (noted $\vdash^{\mathsf{o}}_{\mathsf{hd}}$, figure 5.3b) verifies that Γ_{n+1} entails the required postcondition. More precisely, the inferred type of the terms appearing in output positions must be smaller or equal (with respect to $\subseteq$) than the type declared in the signature. In addition, the determinacy of the body must also be smaller or equal than the expected one. For instance, if a predicate is declared as deterministic,

$$\frac{\Gamma \left|^{\mathsf{i}}_{\mathsf{hd}}\ c \rightsquigarrow \mathcal{D}_1,\ \Gamma_1 \quad \forall i \in 1 \ldots n \text{ do } \mathcal{P},\ \Gamma_i,\ \mathcal{D}_i \left|_{\mathsf{a}}\ b_i \rightsquigarrow \mathcal{D}_{i+1},\ \Gamma_{i+1} \quad \Gamma_{n+1} \right|^{\mathsf{o}}_{\mathsf{hd}}\ c : \mathcal{D}_{n+1}}{\mathcal{P},\ \Gamma \left|_{\mathsf{r}}\ c\ \texttt{:-}\ b_1 \ldots b_n} (\mathsf{r})$$

(a) Rules for **check rule**: its type is $\mathbb{P},\ \mathbb{\Gamma} \left|_{\mathsf{r}}\ \mathbb{R}\right.$

$$\frac{x \text{ fresh in } \Gamma \qquad \mathcal{P},\ \Gamma + \{x \mapsto \star\} \left|_{\mathsf{r}}\ r\right.}{\texttt{mut\_excl}\ \mathcal{P}\ \Gamma\ x\ r \qquad (r :: \mathcal{P}),\ \Gamma + \{x \mapsto \star\},\ \mathcal{D} \left|_{\mathsf{a}}\ a \rightsquigarrow \mathcal{D}',\ \Gamma'\right.} (\mathsf{a}_\forall)$$

$$\mathcal{P},\ \Gamma,\ \mathcal{D} \left|_{\mathsf{a}}\ \texttt{pi}\ x.\ r \Rightarrow a \rightsquigarrow \mathcal{D}',\ \Gamma'\right.$$

$$\frac{}{\mathcal{P},\ \Gamma,\ \_ \left|_{\mathsf{a}}\ \text{cut} \rightsquigarrow \texttt{Fun},\ \Gamma\right.} (\mathsf{a}_!) \qquad \frac{\Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ c \rightsquigarrow \texttt{Rel},\ \bot\right.}{\mathcal{P},\ \Gamma,\ \_ \left|_{\mathsf{a}}\ c \rightsquigarrow \texttt{Rel},\ \Gamma\right.} (\mathsf{a}_\bot)$$

$$\frac{\Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ c \rightsquigarrow \mathcal{D}',\ \top \qquad \Gamma \left|^{\mathsf{o}}_{\mathsf{a}}\ c : \mathcal{D}' \rightsquigarrow \Gamma'\right.}{\mathcal{P},\ \Gamma,\ \mathcal{D} \left|_{\mathsf{a}}\ c \rightsquigarrow \texttt{max}\ \mathcal{D}\ \mathcal{D}',\ \Gamma'\right.} (\mathsf{a}_c)$$

(b) Rules for **check atom**: its type is $\mathbb{P},\ \mathbb{\Gamma},\ \mathbb{D} \left|_{\mathsf{a}}\ \mathbb{A} \rightsquigarrow \mathbb{D},\ \mathbb{\Gamma}\right.$

$$\frac{\Gamma\ s = \tau}{\Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ s \rightsquigarrow \tau,\ \top\right.} (\mathsf{tm}^{\mathsf{i}}_{k}) \qquad \frac{\Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ f \rightsquigarrow \star \xrightarrow{\mathsf{i}} \tau,\ b\right.}{\Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ f\ d \rightsquigarrow \tau,\ b\right.} (\mathsf{tm}^{\mathsf{i}}_{\star})$$

$$\frac{\Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ f \rightsquigarrow \_ \xrightarrow{\mathsf{o}} \tau,\ b\right.}{\Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ f\ t \rightsquigarrow \tau,\ b\right.} (\mathsf{tm}^{\mathsf{i}}_{o})$$

$$\frac{\Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ f \rightsquigarrow \rho \xrightarrow{\mathsf{i}} \tau,\ \top \qquad \Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ t \rightsquigarrow \rho',\ \top \qquad \rho' \subseteq \rho\right.}{\Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ f\ t \rightsquigarrow \tau,\ \top\right.} (\mathsf{tm}^{\mathsf{i}}_{\top})$$

$$\frac{\Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ f \rightsquigarrow \rho \xrightarrow{\mathsf{i}} \tau,\ b_1 \qquad \Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ t \rightsquigarrow \rho',\ b_2 \qquad b_1 = \bot \vee b_2 = \bot \vee \rho' \not\subseteq \rho\right.}{\Gamma \left|^{\mathsf{i}}_{\mathsf{tm}}\ f\ t \rightsquigarrow \texttt{weak}\ \tau,\ \bot\right.} (\mathsf{tm}^{\mathsf{i}}_{\bot})$$

(c) Rules for **check term input**: its type is $\mathbb{\Gamma} \left|^{\mathsf{i}}_{\mathsf{tm}}\ \mathbb{tm} \rightsquigarrow \mathbb{ty},\ \mathbb{B}\right.$

$$\frac{\tau' = \text{if } X \in \Gamma \text{ then } \Gamma\ X \text{ else } \tau}{\Gamma \left|^{\mathsf{o}}_{\mathsf{tm}}\ X : \tau \rightsquigarrow \Gamma + \{X \mapsto \texttt{min}\ \tau\ \tau'\}\right.} (\mathsf{tm}^{\mathsf{o}}_{X}) \qquad \frac{\Gamma\ k = \tau' \quad \tau' \subseteq \tau}{\Gamma \left|^{\mathsf{o}}_{\mathsf{tm}}\ k : \tau \rightsquigarrow \Gamma\right.} (\mathsf{tm}^{\mathsf{o}}_{k})$$

$$\frac{\Gamma \left|^{\mathsf{o}}_{\mathsf{tm}}\ f : \star \xrightarrow{\mathsf{i}} \tau \rightsquigarrow \Gamma'\right.}{\Gamma \left|^{\mathsf{o}}_{\mathsf{tm}}\ f\ d : \tau \rightsquigarrow \Gamma'\right.} (\mathsf{tm}^{\mathsf{o}}_{\star}) \qquad \frac{\Gamma \left|^{\mathsf{o}}_{\mathsf{tm}}\ f : \_ \xrightarrow{\mathsf{o}} \tau \rightsquigarrow \Gamma'\right.}{\Gamma \left|^{\mathsf{o}}_{\mathsf{tm}}\ f\ t : \tau \rightsquigarrow \Gamma'\right.} (\mathsf{tm}^{\mathsf{o}}_{o})$$

$$\frac{\Gamma \left|^{\mathsf{o}}_{\mathsf{tm}}\ f : \rho \xrightarrow{\mathsf{i}} \tau \rightsquigarrow \Gamma' \qquad \Gamma' \left|^{\mathsf{o}}_{\mathsf{tm}}\ t : \rho \rightsquigarrow \Gamma''\right.}{\Gamma \left|^{\mathsf{o}}_{\mathsf{tm}}\ f\ t : \tau \rightsquigarrow \Gamma''\right.} (\mathsf{tm}^{\mathsf{o}}_{i})$$

(d) Rules for **assume term output**: its type is $\mathbb{\Gamma} \left|^{\mathsf{o}}_{\mathsf{tm}}\ \mathbb{tm} : \mathbb{ty} \rightsquigarrow \mathbb{\Gamma}\right.$

Figure 5.2: Determinacy checker algorithm part 1

$$\frac{\Gamma\ k = \tau}{\Gamma \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. k \rightsquigarrow \tau,\ \Gamma}\ (\mathsf{hd}^{\ \mathsf{i}}_{k}) \qquad \frac{\Gamma \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. f \rightsquigarrow \_ \xrightarrow{\circ} \tau,\ \Gamma' \quad \vee \quad \Gamma \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. f \rightsquigarrow \star \xrightarrow{\mathsf{i}} \tau,\ \Gamma'}{\Gamma \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. f\ t \rightsquigarrow \tau,\ \Gamma'}\ (\mathsf{hd}^{\ \mathsf{i}}_{o})$$

$$\frac{\Gamma \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. f \rightsquigarrow \rho \xrightarrow{\mathsf{i}} \tau,\ \Gamma' \qquad \Gamma' \left|\frac{\mathsf{o}}{\mathsf{tm}}\right. t : \rho \rightsquigarrow \Gamma''}{\Gamma \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. f\ t \rightsquigarrow \tau,\ \Gamma''}\ (\mathsf{hd}^{\ \mathsf{i}}_{i})$$

(a) Rules for **assume head input**: its type is $\mathbb{\Gamma} \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. \mathbb{tm} \rightsquigarrow \mathbb{ty},\ \mathbb{\Gamma}$

$$\frac{\Gamma\ k = \tau' \qquad \tau \subseteq \tau'}{\Gamma \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. k : \tau}\ (\mathsf{hd}^{\circ}_{k}) \qquad \frac{\Gamma \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. f : \_ \xrightarrow{\mathsf{i}} \tau \quad \vee \quad \Gamma \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. f : \star \xrightarrow{\circ} \tau}{\Gamma \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. f\ t : \tau}\ (\mathsf{hd}^{\circ}_{i})$$

$$\frac{\Gamma \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. f : \rho \xrightarrow{\circ} \tau \qquad \Gamma \left|\frac{\mathsf{i}}{\mathsf{tm}}\right. t \rightsquigarrow \rho',\ \top \qquad \rho' \subseteq \rho}{\Gamma \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. f\ t : \tau}\ (\mathsf{hd}^{\circ}_{o})$$

(b) Rules for **check head output**: its type is $\mathbb{\Gamma} \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. \mathbb{tm} : \mathbb{ty}$

$$\frac{\Gamma\ s = \tau}{\Gamma \left|\frac{\mathsf{o}}{\mathsf{a}}\right. s : \tau \rightsquigarrow \Gamma}\ (\mathsf{a}^{\circ}_{k}) \qquad \frac{\Gamma \left|\frac{\mathsf{o}}{\mathsf{a}}\right. f : \rho \xrightarrow{\circ} \tau \rightsquigarrow \Gamma' \qquad \Gamma' \left|\frac{\mathsf{o}}{\mathsf{tm}}\right. t : \rho \rightsquigarrow \Gamma''}{\Gamma \left|\frac{\mathsf{o}}{\mathsf{a}}\right. f\ t : \tau \rightsquigarrow \Gamma''}\ (\mathsf{a}^{\circ}_{o})$$

$$\frac{\Gamma \left|\frac{\mathsf{o}}{\mathsf{a}}\right. f : \_ \xrightarrow{\mathsf{i}} \tau \rightsquigarrow \Gamma}{\Gamma \left|\frac{\mathsf{o}}{\mathsf{a}}\right. f\ t : \tau \rightsquigarrow \Gamma}\ (\mathsf{a}^{\circ}_{i})$$

(c) Rules for **assume atom output**: its type is $\mathbb{\Gamma} \left|\frac{\mathsf{o}}{\mathsf{a}}\right. \mathbb{tm} : \mathbb{ty} \rightsquigarrow \mathbb{\Gamma}$

Figure 5.3: Determinacy checker algorithm part 2

then the determinacy flag inferred for its body must be `Fun`. Conversely, if the predicate is declared as a relation, then the inferred determinacy flag for the body may be either `Fun` or `Rel`.

Each of the n body atoms b_i is processed by **check atom** (noted $\vdash_{\mathsf{a}}$, figure 5.2b). This procedure updates the context, step by step, and computes the determinacy of the whole body, producing $\mathcal{D}_{n+1}$.

The **check atom** procedure includes specific rules to handle cut ($a_!$ in figure 5.2b) and `pi` ($a_\forall$ in figure 5.2b), as well as rules for predicate calls (a_c) and miscalls ($a_\bot$).

The cut operator resets the current determinacy to `Fun`. This follows from its semantics: once a cut is executed, all alternative choice points are discarded, so no nondeterminism can arise.

The `pi` operator introduces a fresh symbol together with a local rule. This rule is checked to ensure that determinacy is preserved. In particular, it must satisfy **check rule** in the current context and must not violate mutual exclusion. Then, the underlying atom is analysed in the program extended with this local rule.

Rule a_c checks that the preconditions of the predicate c are satisfied. This is ensured by **check term input**, which verifies that the inputs meet the expected conditions. If this holds, the postcondition is assumed, producing an updated context Γ'. The determinacy is updated as the maximum between the current one and that declared for the predicate.

If the preconditions are not satisfied, the context remains unchanged and the determinacy is set to `Rel`. This value reflects that, in case of a miscalled predicate, no deterministic behaviour can be guaranteed.

The most relevant procedure is the one used to check whether the preconditions of a predicate call are satisfied, namely **check term input** (noted $\vdash^{\mathsf{i}}_{\mathsf{tm}}$, figure 5.2c). This procedure ignores data arguments and output arguments, and simply retrieves from the context the signature of constants and variables (tm^i_k in figure 5.2c).

Rule $tm^i_\top$ corresponds to the standard typing rule for application. When the input arguments match the expected signature, it computes the resulting type and reports no miscall (i.e. $\top$).

Otherwise, the head or the argument contain a miscall, or when the argument does not match the expected signature, rule $tm^i_\bot$ is applied. In this case, a miscall is reported and the resulting signature is obtained by weakening the signature inferred from the head. Intuitively, the predicate call is treated as a relation, and the types of the output arguments are weakened accordingly.

The procedure to update the context when an atom is found to be a good call is **assume atom output** (noted $\vdash^{\mathsf{o}}_{\mathsf{a}}$, figure 5.3c). It is an iterator over output arguments: other than ignoring input arguments ($\mathsf{a}^{\mathsf{o}}_i$) and fetching the signature of symbols in the context ($\mathsf{a}^{\mathsf{o}}_k$), it calls **assume term output** on output arguments.

The **assume term output** procedure (noted $\vdash^{\mathsf{o}}_{\mathsf{tm}}$, figure 5.2d) updates the context when it encounters variables (rule tm^o_X). If a variable already has a signature in the context, the stronger one is kept.

Data and output arguments are ignored, while input arguments are traversed recursively. In this way, the signatures of variables occurring in input positions are also updated in the context.

5.5 Determinism guarantees

In order to express *operational determinacy* we need to use the operational semantics for Elpi. The semantics is given in section 2.3.2.

The properties of runE relevant to the current presentation are:

1. cut behaves as per section 2.3.3
2. `pi` $x.\ r \Rightarrow a$ generates a fresh constant for x available in r and a
3. `pi` $x.\ r \Rightarrow a$ locally loads r in a giving it the highest priority
4. input arguments are matched against the query as per definition 2.3.5

Definition 5.5.1 (Operationally deterministic execution ($\texttt{deterministic}_{\texttt{a}}\ \mathbb{P}\ \mathbb{A}$))**.** *We say that a query atom b is operationally deterministic in a program $\mathcal{P}$ iff*

$$\forall \sigma, \text{runE}\ ((\sigma,\ (\mathcal{P}, b) :: \varnothing) :: \varnothing) \rightsquigarrow \text{Some}\ (\_, a)\ \rightarrow\ a = \varnothing$$

In other words a deterministic query leaves no alternatives when it succeeds. We now link the static check to the deterministic execution of a query.

Definition 5.5.2 (Checked program ($\texttt{checked}_{\texttt{p}}\ \mathbb{\Gamma}\ \mathbb{P}$))**.** *A program $\mathcal{P}$ is checked against a context of signatures Γ iff it validates these rules:*

$$\frac{}{\textit{checked}_{\mathsf{p}}\ \Gamma\ \varnothing} \qquad \frac{\textit{checked}_{\mathsf{p}}\ \Gamma\ \mathcal{P} \qquad \textit{mut\_excl}\ \mathcal{P}\ r \qquad \mathcal{P},\ \Gamma \vdash_{\mathsf{r}} r}{\textit{checked}_{\mathsf{p}}\ \Gamma\ (r :: \mathcal{P})}$$

The empty program is a valid candidate for determinacy checking. Rules are processed one by one, in declaration order, thereby respecting the priority they have in the operational interpretation of the program. This ordering is essential for checking mutual exclusion. In particular, each rule r is checked against the remaining rules in the program, denoted by $\mathcal{P}$. A mutual exclusion error is reported whenever there exists a rule $r' \in \mathcal{P}$ whose head overlaps with the head of r, and the rule r is not protected by a cut.

Definition 5.5.3 (Checked query atom ($\texttt{checked}_{\texttt{a}}\ \mathbb{\Gamma}\ \mathbb{A}$))**.** *An atom b is checked against a context of signatures Γ iff*

$$\varnothing,\ \Gamma,\ \textit{Fun} \vdash_{\mathsf{a}} b \rightsquigarrow \textit{Fun},\ \_$$

Theorem 5.5.4 (Static analysis)**.** *Given program $\mathcal{P}$, a context of signatures Γ and a query atom b, $\textit{checked}_{\mathsf{p}}\ \Gamma\ \mathcal{P} \wedge \textit{checked}_{\mathsf{a}}\ \Gamma\ b \Rightarrow \textit{deterministic}_{\mathsf{a}}\ \mathcal{P}\ b$.*

The proof of theorem 5.5.4 has been mechanized in the Rocq proof assistant for the first-order fragment, and is available at `https://github.com/FissoreD/elpi-formalization`. The first-order formalization is discussed in detail in chapter 6. The mechanization of the higher-order part is still ongoing.

Software	LOC	% `Fun`	cut added	cut needed (bugs)
ELPI	1448	97%	1	1 (100%)
ROCQ-ELPI	13164	79%	91	78 (85%)
HIERARCHY BUILDER	5326	94%	33	30 (90%)
ALGEBRA TACTICS	1799	–	1	1 (100%)
TROCQ	2918	–	4	4 (100%)
BRICK (public)	2500	–	11	11 (100%)
BRICK (private)	7500	–	6	6 (100%)
Total	34655	85%	147	131 (89%)

Table 5.1: Experience report

5.6 Experience report

The static analyzer we describe was integrated in the ELPI language version 3.0. It consists of about 200 lines of OCAML code for the implementation of `mut_excl` and about 800 lines of code for the implementation of **check rule**, along with all the related auxiliary functions. We have ported some ROCQ libraries written using ELPI: ROCQ-ELPI, HIERARCHY BUILDER, ALGEBRA TACTICS, TROCQ, and BRICK.

We can divide the porting of these libraries into two categories: libraries in which we have traversed all the predicates and carefully changed `pred` markers to `func` (ELPI, ROCQ-ELPI, and HIERARCHY BUILDER), and libraries that have been ported to just pass the static analyzer. In particular, the latter class of libraries measures the minimal cost for adapting code to the changes we made to the ELPI and ROCQ-ELPI standard libraries. A typical example of code that requires a fix is one that loads a local rule without a cut, such as `copy`.

We summarize these results in table 5.1. In the ELPI standard library, we annotated 272 predicates out of 281 as functions, with 37 occurrences of higher-order arguments, as in `map, filter, fold`, etc. We identified 1 error, i.e., a missing cut. In ROCQ-ELPI, we have 928 functions out of 1181, with 62 higher-order predicates. We added 91 cuts to please the static analyzer, 78 of which were clearly bugs in the code. The remaining 13 cuts are due to `mut_excl`, which is not powerful enough to discriminate among rules. The algorithm could be improved to better distinguish mutual exclusion by extending definition 5.3.1, as in [LGBH05], but the problem is, in general, undecidable. An example of this issue is given below:

```
func fact int → int.
fact 0 1.
fact N R :- N > 0, N' is N - 1, fact N' R', R is N * R'.
```

The two rules for `fact` are in mutual exclusion: the input cannot be *0* and *greater than 0* at the same time. But a complete algorithm identifying all of these scenarios does not exists, and we require an explicit cut in the first rule.

In HIERARCHY BUILDER, we have 460 functions out of 488, with 33 higher-order arguments, and we have corrected 30 bugs in the code.

Overall, we can conclude that 85% of the predicates were functions.

Authors	System	Mode analysis	Dynamic	H.O.	Miscalls
[DW89]	SB-Prolog	required	✗	✗	✗
[LGBH05]	Ciao	required	✗	✗	✗
[KK11]	RedAlert	required	✗	✗	✗
[SHC96]	Mercury	required	✗	✓‡	✗†
[HP25]	Curry	no need	✗	✓	✓
Fissore and Tassi	Elpi	no need	✓	✓	✓

Table 5.2: Comparison with related work

In Algebra Tactics, we added 1 cut over 1799 lines of codes; in Trocq, we added 4 cuts over 2918 lines of code. None of these libraries were using `copy` as a relation, i.e. to generate all possible copies of a term. The BRiCk application has some public code we ported ourselves and some closed source code ported by the BlueRock company, that we thank for sharing these numbers. In the public code we added 11 cuts over 2500 lines of code, while in the private one they added 11 cuts over 7500 lines.

5.7 Related work and concluding remarks

We summarize several related works in table 5.2. One key difference between our work and [DW89, LGBH05, SHC96, KK11, HP25] is that they focus on *closed* predicates, that do not evolve during execution, while λ-Prolog, hence Elpi, allows rules to be dynamically loaded. For closed predicates, programs can be transformed into *super-homogeneous* form by merging all rules into one with disjunctions. This simplifies determinacy inference and may allow to compile the code to a faster binary. However, this transformation is not feasible in our dynamic setting (section 5.1.2).

Regarding modes, Elpi's matching procedure over input arguments allows us to dispense with explicit mode analysis – an essential component in [DW89, LGBH05, SHC96, KK11].

We differ from RedAlert [KK11] in that we pick an operational semantics that explicitly represents choice points as a list of alternatives, and that hence is very close to the actual behavior of Elpi's runtime. They pick a denotational semantics over the super-homogeneous form that in turn lets them formulate the static analyzer as an abstract interpreter. It is unclear if a denotational semantics can be written without transforming the program in super-homogeneous form.

Some higher-order features of Elpi are also present in Mercury [SHC96], although in a more restricted form (‡). In Mercury, input modes generally require terms to be ground (with some limited relaxation for data). However, the predicate `once` in Elpi (see section 5.1.1) is much more useful when the input relation can be non-ground. For example, `once (map get_pdiv [6] L)` is a valid use case in Elpi, but it cannot be encoded in Mercury.

We allow higher-order predicates to be miscalled, while Mercury does not. Instead, it requires higher-order functions to be annotated with explicit mode and determinacy signatures for each possible behavior. As a result, Mercury ascribes 11 signatures to

the `foldr` predicate and any call that does not match exactly one of these signatures results in a compilation failure (†). ELPI targets less technically inclined users, more familiar with functional programming languages than logic ones, hence we strive for simplicity, defining only one determinacy class (in MERCURY they are 8) and immediately classifying wrong calls as non-deterministic. As a result the signature for `foldr` is just `func foldr (func L, A → A), list L, A → A`, familiar to our audience.

The "determinism types" of CURRY [HP25] are much more similar to our signatures, and CURRY supports miscalled functions as well. In the work of Hanus and Prott the `Any` type stands for non-determinism and like a black hole it absorbs the determinism of the surrounding sub-terms. This is a convenient way to capture all miscalls and treat them uniformly. In our presentation, we have no equivalent; instead, we use `weak` to change the signature of a term when a miscall is detected (such as, $tm^i_\perp$ in 5.2c or atom calls $a_\perp$ in 5.2b). The advantage is that `weak` only changes the leaves ($\mathbb{D}$) of the signature while leaving its structure intact. Although we do not analyze data in this chapter, our set of rules can be complemented to perform full type checking (in addition to determinacy checking) in one pass.

In CURRY the `allValues` builtin can be used to turn a relation into a function. PROLOG, and also ELPI, provide a similar `findall` builtin, but in addition to that they also let one commit to the first result by either using `once`, or by placing a cut after the non-deterministic call. It is this "after the fact" commitment that make the analysis a bit more complex. In fact, in the body of a deterministic predicate we can have calls to relations (or miscalled functions) and still consider the rules valid with respect to determinacy (see $a_!$ in 5.2b). The notion of *functional context* in [DW89] is closely related to this difficulty.

Another difference with [HP25] is that, when a variable appears as the output of a miscalled predicate, its signature may change over time. For example, consider

```
compose rel1 rel2 R, R = copy, R T1 T2
```

where `compose` and `copy` are defined in sections 5.1.2 and 5.1.3, and `rel1` and `rel2` are two relations.

The miscall to `compose` initially assigns a relational interpretation to `R`. However, the subsequent assignment `R = copy` refines this information and makes `R` a function. As a result, the call `R T1 T2` is treated as a function.

Our experience report confirms most of the ELPI code out there is made of functional predicates and that the static analyzer accepts the vast majority of it requiring no changes. The main improvement we will consider is about *mode subtyping* for higher-order arguments. In the definition of $\subseteq$ we currently accept only arrows that have the same input/output mode. However, it seems possible to relax this condition. For example we could allow passing to `map` a function of type $\star \overset{\circ}{\to} \star \overset{\circ}{\to}$ `Fun`, since the modes of higher-order arguments seem to play no role in the determinacy of code that receives and calls them.

Chapter 6

A Mechanized First-Order Static Determinacy Checker

In the previous chapter we explained the implementation of the determinacy checker algorithm that we have included in ELPI since version 3. The checker helps users control backtracking: the user asserts which predicates should be considered deterministic and then the checker verifies that these predicates produce at most one solution for each call, i.e. they commit to their first solution, leaving no *choice points*.

In this chapter we present our work consisting of the formalization (in Rocq) of the determinacy checker we implemented in Elpi. This formalization covers the first-order part of the language, i.e. it does not consider higher-order variables or dynamic rule loading, such as the `pi` and `=>` operators. The idea of the formalization is to implement a *det_check* predicate that tests whether a program satisfies determinacy, i.e. whether every predicate declared deterministic produces at most one solution per call. Finally, we show that if a program satisfies this det_check, then any call to a deterministic predicate in the Elpi semantics produces at most one solution.

In order to proceed in this direction, we consider one main aspect. Determinacy is greatly influenced by the presence of cut in the code. This operator impacts the execution of the code and, therefore, it is important to correctly identify its scope while reasoning about determinacy.

The operational semantics of Elpi is, however, not well suited for reasoning about the scope of cuts. Because the syntax of Elpi programs does not explicitly record the structure of the search space, it is difficult to determine precisely which alternatives are discarded by a given cut.

To address this issue, we introduce a semantics based on And-Or trees, in which the history of the computation is represented in a structured way: given a goal, each applicable rule generates an Or-branch, which is in turn followed by an And-branch for each premise. This enables an explicit characterization of the cut operator as a function that prunes selected branches of the tree. The tree-based semantics allows us to formulate introduction and elimination rules for disjunction in the presence of cut, which, as expected, differ from those of classical logic. For example, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since the branch corresponding to q_2 may be pruned during the evaluation of q_1.

Once we have defined this semantics, we prove its equivalence with the ELPI semantics. Finally, we use the tree-based semantics to prove the correctness of the determinacy analysis from [FT26].

Our mechanization is axiom-free and is available at https://github.com/FissoreD/elpi-formalization.

The remainder of this work is organized as follows. We first present in section 6.1 the syntax we use in our ROCQ development to represent terms, atoms, rules, and programs (it is a subset of the syntax shown in section 2.3.1). In section 6.2, we provide the formal representation of the ELPI semantics restricted to the first-order fragment, called runS (it is a subset of section 2.3.2). After the formalization of the runS, we formalize the And-Or tree semantics, called runT, and show some properties about the impact of the cut operator on disjunctions. In section 6.4, we prove the equivalence of the two semantics. Finally, in section 6.5, we prove the correctness of the determinacy analysis with respect to the And-Or tree semantics..

6.1 Preliminaries: syntax

The description of a program is given by a pair $\mathbb{P} := (\text{list } \mathbb{R} \times \text{sigT})$ and is shared by both semantics. A program consists of a list of rules together with a signature environment sig. We delay the discussion of signatures to section 6.5 where we describe the static analysis that concerns them.

A rule is a pair $\mathbb{R} := (\mathbb{Tm} \times \text{list } \mathbb{A})$, where the first element represents its head and the second its list of premises. Each premise is an atom, i.e. either the cut operator or a term call. Terms include predicate symbols, data symbols, variables, and term application. The concrete definition of these two algebraic types is given below:

$$\mathbb{Tm} := \text{Symb } \mathbb{N} \mid \text{Pred } \mathbb{N} \mid \text{Var } \mathbb{N} \mid \text{App } \mathbb{Tm}\ \mathbb{Tm}$$
$$\mathbb{A} := \text{cut} \mid \text{call } \mathbb{Tm}$$

The syntax tree allows variables to be applied and predicates to be mixed with data. These higher-order features play no role in the current chapter, which focuses on first-order logic programming, but they allow future extensions to full λPROLOG to be based on the same mechanization.

Variables, predicates and data symbols are identified by natural numbers. We represent substitutions (of type Σ) as finitely supported maps from variables to terms, using the finmap library [mat26]. The empty substitution is written ε, while the composition of two substitutions σ_1 and σ_2 with disjoint supports as $\sigma_1 + \sigma_2$.

The backchaining operation applies all rules to a query term; it is central to both semantics and it has the following type:

$$\text{backchain} : \mathbb{P} \to \mathcal{F}_\nu \to \mathbb{Tm} \to \Sigma \to (\mathcal{F}_\nu \times \text{list } (\Sigma \times \text{list } \mathbb{A}))$$

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the backchain function takes a program, the set of variable names used so far ($\mathcal{F}_\nu$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More

precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

The function *unify* takes two terms t_1 and t_2, together with a substitution σ, as input, and optionally returns a substitution σ' that extends σ. Therefore, if t_1 and t_2 unify under σ, then $\sigma'\ t_1 = \sigma'\ t_2$, where $\sigma\ t$ denotes substitution application and $=$ denotes syntactic equality.

To keep the mechanization axiom-free, we implement a unification algorithm, which we briefly describe in section 6.5 and prove sound and complete. Since the two semantics and the static analysis use the same unification algorithm, the equivalence proof is completely agnostic to the particular implementation chosen for that algorithm.

Notational conventions In the following section we note $\mathbb{B}$ for the boolean type. Following the Mathematical Components library, on which we depend, we use `x.1` and `x.2` to denote the first and second projection applied to the pair `x`.

6.2 Stack-based semantics

The stack-based semantics we present is very close to what is implemented by the Elpi interpreter and is a subset of section 2.3.2. This semantics is similar to the one proposed by Pusch in [Pus96], which is in turn closely related to the semantics given by Debray and Mishra in [DM88, Section 4.3]. Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that are also present in the Warren Abstract Machine [War83, AK91].

The execution state is a stack of alternatives. Each alternative is a list of goals. Intuitively it amounts to a disjunctive normal form: each stack item is a self contained computation, coupling a substitution with all the goals to be solved.

Goals pair an atom with a list of the *cut-to* alternatives, making the two datatypes mutually recursive, but these alternatives have a very different role. They have to be understood as pointers to lower levels of the stack itself. They are used to implement the cut, i.e. to prune the stack of alternatives.

$$\text{alts} := \text{list}\ (\Sigma \times \text{goals}) \qquad \text{goals} := \text{list}\ (\mathbb{A} \times \text{alts})$$

The stack-based semantics is described by the runS inductive predicates.

$$\text{runS} : \overrightarrow{\mathcal{F}_\nu} \to \text{alts} \to \text{option}\ (\Sigma \times \text{alts}) \to \texttt{Prop}$$

The runS predicates relates a set of variables and a list of alternatives to optional result. None stands for failure, i.e., no solution, whereas Some (σ, a) represents success with σ being the solution and a as the list of not-yet-explored alternatives.

The rule system for runS is given in figure 6.1 and is made by 4 cases. We distinguish two main cases. If we run out of alternatives we fail: rule FailS stops the execution and return None. If the list of alternatives is $(\sigma_0, h) :: a$, then we look at the list of goals h (under the substitution σ_0). If h is empty, rule StopS stops the execution with a success σ_0 leaving a as the unexplored alternatives. If h is not empty we look at its head goal (t, ca) where t is the atom and ca is its cut-to alternatives. If t is a cut, rule CutS continues to evaluate the tail of h under with alternatives ca. Finally, when t is a call, rule CallS

$$\mathcal{B}_S\ p\ v\ t\ \sigma\ a\ g = \text{let } (v', rs) := \text{backchain } p\ v\ t\ \sigma \text{ in } \left(v', \left[\sigma', ([[(p, q, a) \mid q \in b] +\!\!+ g) \;\middle|\; (\sigma', b) \in rs\right]\right)$$

$$\frac{}{\text{runS } v\ ((\sigma,\ [\,])::a)\ (\text{Some } (\sigma,\ a))}\ \text{StopS}$$

$$\frac{\text{runS } v\ ((\sigma,\ g)::\text{ca})\ r}{\text{runS } v\ ((\sigma,\ (\text{cut},\ \text{ca})::g)::\_)\ r}\ \text{CutS}$$

$$\frac{\mathcal{B}_S\ p\ v\ t\ \sigma\ a\ g = (v',\ \text{bs}) \qquad \text{runS } v'\ (\text{bs} +\!\!+ a)\ r}{\text{runS } v\ ((\sigma,\ (\text{call } t,\ \_)::g)::a)\ r}\ \text{CallS}$$

$$\frac{}{\text{runS } \_\ [\,]\ \text{None}}\ \text{FailS}$$

Figure 6.1: Rule system for runS

backchains: for each rule it pairs each premise with the current stack of alternatives, and prepends the obtained goals to the tail of h. Note that if backchain returns the empty list, the second premise of rule CallS amounts to exploring the next item in the current stack.

6.3 And-Or Tree semantics

An And-Or tree is a stratified, variable-width tree, defined as follows:

$$\mathcal{T} := \text{KO} \mid \text{OK} \mid \text{Unexplored } \mathbb{A} \mid \text{Or } (\text{option } \mathcal{T})\ \Sigma\ \mathcal{T} \mid \text{And } \mathcal{T}\ (\text{list } \mathbb{A})\ \mathcal{T}$$

It consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving one alternative in the Or-layer, but all subqueries in the corresponding And-layer. These invariants about the $\mathcal{T}$ data type are precisely described in section 6.4.1 where we introduce the valid_tree predicate.

The tree is built incrementally. The *Unexplored* node represents an unexplored branch (an atom). When the atom is a call, we use the backchaining procedure from section 6.1 to compute the two layers to attach to the tree: alternatives form the Or-layer, and the premises of each applicable rule form the corresponding And-layer. This expansion step is performed by the *step* procedure described in section 6.3.2.

In addition to the Unexplored node we have two distinguished nodes, KO and OK, which respectively represent the smallest failed and successful trees.

The Or node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing disjunction A. The left subtree A is optional and is absent once that branch has been fully explored. The right alternative is paired with a substitution σ, which becomes the current substitution when backtracking to B.

```
Fixpoint next σ t : Σ * tree :=
  match t with
  | Unexplored _ | KO | OK ⇒ (σ, t)
  | Or □ σ' B ⇒ next σ' B
  | Or ·A _ _ ⇒ next σ A
  | And A _ B ⇒ let (σ', pA) := next σ A in
                if pA == OK then next σ' B else (σ', pA)
  end.

Definition next_subst σ t := fst (next σ t).
Definition next_tree t := snd (next ε t).
Definition success t := next_tree t == OK.
Definition failed t := next_tree t == KO.
Definition incomplete t :=
  match next_tree t with Unexplored _ ⇒ ⊤ | _ ⇒ ⊥ end.
```

Figure 6.2: next routine and derived functions

The And node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to the first choice point in A. We call B_0 the *reset point* for B: it represents the continuation for all alternatives of A except the first. That is, when backtracking occurs within A, the subtree B is replaced by the atoms in B_0. An example is shown in section 6.3.3.

A graphical representation of a tree is shown in figure 6.3b. For compactness, we depict And and Or nodes as n-ary, with children associating to the right. The KO node acts as the terminating element of an Or list, while OK is the terminating element of an And list.

6.3.1 The semantics

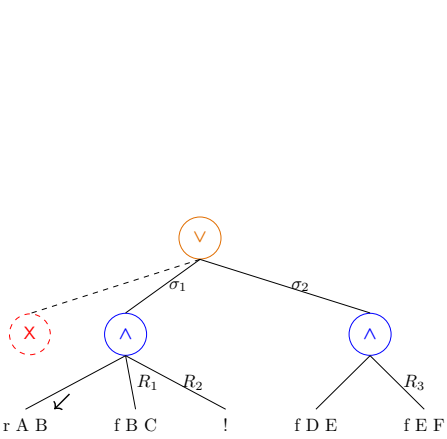
The tree is constructed incrementally by two recursive functions, step and prune. Both traverse the tree depth-first, left-to-right. The node to be explored in a tree is retrieved thanks to next_tree (in figure 6.2). When this node is OK we say the entire tree is a *success*, when it is KO we say the tree *failed*, otherwise the tree is *incomplete*. In figure 6.3 we mark with a black arrow the result of next_tree. The type of these two functions is given below.

$$\text{step} : \mathbb{P} \to \mathcal{F}_\nu \to \Sigma \to \mathcal{T} \to (\mathcal{F}_\nu \times \text{tag} \times \mathcal{T})$$
$$\text{prune} : \mathbb{B} \to \mathcal{T} \to \text{option } \mathcal{T}$$

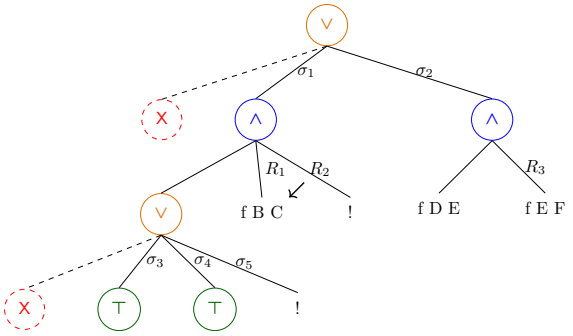
The step routine, among its outputs, also returns a *tag*, whose definition is described in the very following section.

To avoid the constraint that all functions must terminate in Rocq, we define the transitive closure of step and prune as the inductive relation runT. More precisely runT iterates calls to step on incomplete trees to expand Unexplored nodes. When a tree fails it calls prune to find the next alternative and continues from there, if one exists.

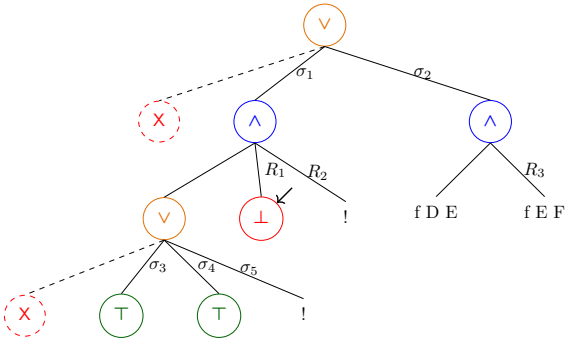
6.3.2 The step procedure



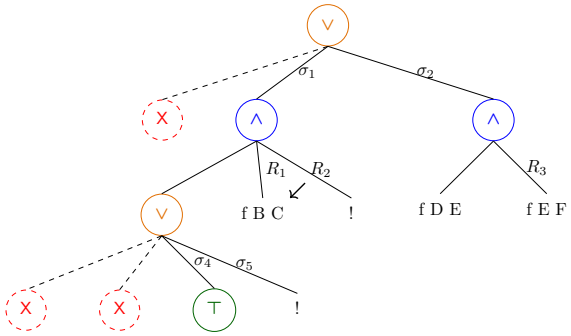
(b) Tree after step on `g 0 R`



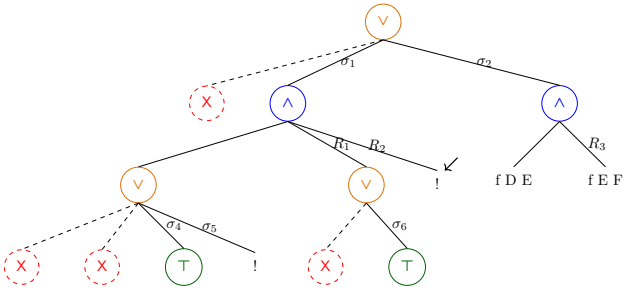
(c) step on figure 6.3b



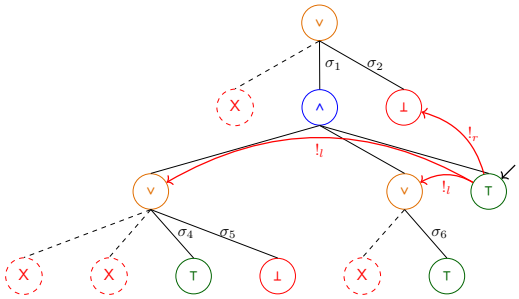
(d.1) step on figure 6.3c



(d.2) prune on figure 6.3d.1



(e) step on figure 6.3d.2



(f) step on figure 6.3e

Figure 6.3: Execution in the tree semantics. The substitutions $\sigma_1 \ldots \sigma_6$ are from section 2.3.3 and refer to interpretation on the program recalled in figure 6.4. R_1, R_2, and R_3 are reset points and correspond respectively to the lists of atoms [`f B C, !`], [`!`], and [`f D E`].

```
pred f int → int.
f 1 2.
f 2 3.

pred r int → int.
r 0 0.
r 0 1.
r 0 2 :- !.

pred g int → int.
g A C :- r A B, f B C, !.
g D F :- f D E, f E F.
```

Figure 6.4: Example recalling the program from figure 2.5

```
Definition step p v s A :=
  match A with
  | OK                  => (v, Success, OK)
  | KO                  => (v, Failed, A)
  | Unexplored cut      => (v, CutBrothers, OK)
  | Unexplored (call t) =>
      let: (v, l) := backchain u p v t s in
      (v, Expanded,
      if l is ((s0, x0) :: xs) then Or None s0 (big_or x0 xs) else KO)
  | Or A sB B =>
      if A is Some A then
        let: (v, tA, rA) := step p v s A in
        (v, if is_cb tA then Expanded else tA,
         Or (Some rA) sB (if is_cb tA then KO else B))
      else
        let: (v, tB, rB) := step p v sB B in
        (v, if is_cb tB then Expanded else tB, Or A sB rB)
  | And A B0 B =>
      if success A then
        let: (v, tB, rB) := step p v (next_subst s A) B in
        (v, tB, And (if is_cb tB then cutl A else A) B0 rB)
      else
        let: (v, tA, rA) := step p v s A in (v, tA, And rA B0 B)
  end.
```

Figure 6.5: The step procedure

The step procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a tag, and an updated tree.

The set of variables is assumed to contain all variables occurring in the tree. It is passed to backchain so that rules can be refreshed correctly.

The tag is implemented as follows:

$$\text{tag} := \text{Expanded} \mid \text{CutBrothers} \mid \text{Failed} \mid \text{Success}$$

It indicates which kind of step was performed. Failure and Success are returned for trees that satisfy, respectively, the failed and success tests. CutBrothers indicates the interpretation of a superficial cut: step was called on an And-layer and its next_tree is the cut atom. Finally, Expanded accounts for the interpretation of predicate calls and non-superficial cuts.

The step procedure evaluates atoms. In particular, it leaves the tree unchanged when the tree is *success* or *failed*.

Lemma 6.3.1 (succ_step_iff)**.** $\forall\ p\ v\ \sigma\ t,\ \text{success}\ t \leftrightarrow \text{step}\ p\ v\ \sigma\ t = (v,\ \text{Success},\ t)$

Lemma 6.3.2 (fail_step_iff). $\forall\ p\ v\ \sigma\ t,\ \text{failed}\ t \leftrightarrow \text{step}\ p\ v\ \sigma\ t = (v,\ \text{Failed},\ t)$

The step procedure expands Unexplored nodes by calling backchain, which returns a list l of alternatives. If l is empty, then the current call atom is replaced by KO. Otherwise, it is replaced by a right-skewed tree of Or nodes, where each leaf corresponds to a list of atoms $e \in l$. If e is empty, the leaf is OK; otherwise, it is a right-skewed tree of And nodes.

Figure 6.3 depicts the tree corresponding to the running example (section 2.3.3) as it undergoes calls to step and prune. We run query `q 0 R` against the program P in figure 2.5. Let c_0, c_1, and c_2 be the children of the root. By construction, c_0 is None, while c_1 and c_2 correspond respectively to the premises of rules r_1 and r_2 in P. Note that c_1 (resp. c_2) is associated with substitution σ_1 (resp. σ_2), whose values are the same as in section 2.3.3. Reset points are defined as follows: $R_1 :=$ `[f Y Z, !]`, $R_2 :=$ `[!]`, and $R_3 :=$ `f Y Z`. Intuitively, they record the lists of atoms used to generate the corresponding subtree. Other examples of step performing a tree expansion via backchain appear in figures 6.3c to 6.3e.

6.3.2.1 The interpretation of a cut

The step corresponding to a cut performs two actions: (i) the cut node is replaced by OK; and (ii) the subtrees in the scope of Cut are pruned. More precisely, (ii) prunes the left siblings of the Cut node (in the same And-layer, denoted $!_l$) and prunes the right uncles of Cut (in the one-level-up Or-layer, denoted $!_r$).

An example of the impact of cut is shown in figure 6.3f. The red arrows indicate the scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to each pointed subtree. Note that the evaluation of a cut keeps the path leading to the cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under substitution σ_5 (which also succeeds) is replaced by KO. This amounts to discarding the third rule of predicate `r`.

6.3.3 The prune procedure

When backchain returns an empty list, step produces a tree that *failed* and the computation must backtrack. The prune procedure is responsible for producing the new tree from which the computation can continue.

In figure 6.3d.1 we encounter a failure because no rule applies to the atom `f B C` under substitution σ_3, which maps `B` to `0`. The prune procedure prunes the path leading to this failure and resets the current sub-query to its original shape. More precisely, it kills the OK node under σ_3 (since that branch has been fully explored and produced no solution), and then replaces the KO node with the information stored in the reset point R_1. This restores the call `f B C`, which can then be reconsidered under substitution σ_4.

Furthermore, prune serves another important purpose. Its behavior depends on the boolean flag it receives as input: prune false t recursively removes all failures (if any) from t, whereas prune true t removes the first success in t. For example, the tree in figure 6.3f contains a successful path that maps `R` to `2`. Calling prune on this tree returns None, since there are no alternatives in the tree after the success.

Some interesting properties of prune are shown below; they illustrate how prune complements step with respect to failed, successful, and incomplete trees.

```
Definition omap f o := match o with Some b => Some (f b) | _ => o end.
Definition prune b A : option tree :=
  match A with
  | KO => None
  | OK => if b then None else Some OK
  | Unexplored _ => Some A
  | And A B0 B =>
      let build_B0 b := omap (λx. And x B0 (big_and B0)) (prune b A) in
      if success A then
        if prune b B is Some B' then Some (And A B0 B')
        else build_B0 true
      else if failed A then build_B0 false
      else Some (And A B0 B)
  | Or None sB B => omap (λx. Or None sB x) (prune b B)
  | Or (Some A) sB B =>
      if prune b A is Some A' then Some (Or (Some A') sB B)
      else omap (λx. Or None sB x) (prune false B)
end.
```

Figure 6.6: The prune function

Lemma 6.3.3 (incpl_prune)**.** $\forall\ b\ t,$ incomplete $t \to$ prune $b\ t =$ Some t

Lemma 6.3.4 (prune_Some)**.** $\forall\ b\ t\ t',$ prune $b\ t =$ Some $t' \to$ failed $t' =$ false

Lemma 6.3.5 (prune_None)**.** $\forall\ t,$ prune false $t =$ None $\to$ failed $t =$ true

6.3.4 The runT relation and its properties

The inductive relation runT has the following type:

$$\text{runT} : \mathbb{P} \to \vec{\mathcal{F}_\nu} \to \Sigma \to \mathbb{T} \to \text{sol} \to \mathbb{B} \to \vec{\mathcal{F}_\nu} \to \texttt{Prop}$$

It associates a program, a set of variables, an initial substitution σ, and a tree t with a solution r, a boolean b, and an updated set of variable names v'.

The type of sol is defined as follows:

$$\text{sol} := \text{Zero} \mid \text{One}\ \Sigma \mid \text{Many}\ \Sigma\ \mathcal{T}$$

Zero represents a failing execution. One σ represents an interpretation producing exactly one solution, i.e. no backtracking is possible from the given state. It also carries the substitution σ that makes the interpretation succeed. Finally, Many $\sigma\ t$ represents an execution in which the original state produces the solution σ, while t represents the remaining unexplored alternatives.

The boolean b records whether a superficial cut was evaluated during execution, and v' is the updated set of variables tracking freshly generated names.

The runT predicate has five cases, depicted as inference rules in figure 6.7.

Rules StopOT and StopMT say that if the tree t is success, then the substitution σ' is extracted from t (starting from σ), depending on the result of prune true t – aiming to

$$\frac{\text{success } t \qquad \text{next\_subst } \sigma\ t = \sigma' \qquad \text{prune true } t = \text{None}}{\text{runT } v\ \sigma\ t\ (\text{One } \sigma')\ \text{false } v}\ \text{StopOT}$$

$$\frac{\text{success } t \qquad \text{next\_subst } \sigma\ t = \sigma' \qquad \text{prune true } t = \text{Some } t'}{\text{runT } v\ \sigma\ t\ (\text{Many } \sigma'\ t')\ \text{false } v}\ \text{StopMT}$$

$$\frac{\text{incomplete } t \qquad \text{step } p\ v\ \sigma\ t = (v',\ \text{tg},\ t') \qquad \text{runT } v'\ \sigma\ t'\ r\ b\ v''}{\text{runT } v\ \sigma\ t\ r\ ((\text{tg}\ =_b \text{CutBrothers})\ \|\ b)\ v''}\ \text{StepT}$$

$$\frac{\text{failed } t \qquad \text{prune false } t = \text{Some } t' \qquad \text{runT } v\ \sigma\ t'\ r\ n\ v'}{\text{runT } v\ \sigma\ t\ r\ n\ v'}\ \text{BackT}$$

$$\frac{\text{prune false } t = \text{None}}{\text{runT } v\ \sigma\ t\ \text{Zero false } v}\ \text{FailT}$$

Figure 6.7: Rule system for runT

clean the tree from its successful path – we return One or Many, if there exist alternatives to be explored. Rule StepT applies when next_tree t is an atom, then step is invoked to evaluate t, and runT is called recursively on the updated tree. When next_tree t is a failure, rule BackT calls prune to clear the failed path; if the resulting cleaned tree exists, runT is called recursively on it. Finally, rule FailT accounts for trees with no solutions.

Lemma 6.3.6 (runT_det)**.**

$$\forall\ p\ v_0\ \sigma\ t\ r\ r'\ b\ b'\ v\ v',\ \text{runT } p\ v_0\ \sigma\ t\ r\ b\ v \to \text{runT } p\ v_0\ \sigma\ t\ r'\ b'\ v' \to r' = r \wedge v' = v \wedge b' = b$$

The proof is by induction on the first runT derivation and is immediate, because the rules are selected based on next_tree t. In particular, the hypotheses success t, incomplete t, and failed t are mutually exclusive. Moreover, by lemma 6.3.5, the hypothesis of FailT implies that t *failed*; hence FailT is exclusive with StopOT, StopMT and StepT, and it is also exclusive with BackT since they impose different requirements on the output of prune.

The boolean output of runT allows us to state interesting properties about the Or node in the presence of cut. Below, we report only a selection of the lemmas that were proved. To simplify their presentation, we introduce the following notation for Or nodes. When the left-hand side is None, we write the symbol $\square$. Otherwise, the notation $A \vee B_\sigma$ implicitly denotes $(\text{Some } A) \vee B_\sigma$.

Lemma 6.3.7 (or_intro_left)**.**

$$\forall\ p\ v\ v'\ \sigma\ \sigma'\ A\ A'\ \sigma_1\ B,\ \text{runT } p\ v\ \sigma\ A\ (\text{Many } \sigma'\ A')\ \text{true } v' \to \text{runT } p\ v\ \sigma\ (A \vee B_{\sigma_1})\ (\text{Many } \sigma'\ (A' \vee \text{KO}_{\sigma_1}))\ \text{false } v'$$

Lemma 6.3.8 (or_intro_right)**.**

$$\forall\ p\ v\ v'\ \sigma\ A\ \sigma_1\ B,\ \mathrm{runT}\ p\ v\ \sigma\ A\ \mathrm{Zero}\ \mathrm{true}\ v' \to \mathrm{runT}\ p\ v\ \sigma\ (A \vee B_{\sigma_1})\ \mathrm{Zero}\ \mathrm{false}\ v'$$

Lemma 6.3.9 (or_intro_right2)**.**

$$\forall\ p\ v_0\ v_1\ v_2\ \sigma\ A\ \sigma_1\ \sigma_2\ B\ b,\ \mathrm{runT}\ p\ v_0\ \sigma\ A\ \mathrm{Zero}\ \mathrm{false}\ v_1 \to \mathrm{runT}\ p\ v_1\ \sigma_1\ B\ (\mathrm{One}\ \sigma_2)\ b\ v_2 \to \mathrm{runT}\ p\ v_0\ \sigma\ (A \vee B_{\sigma_1})\ (\mathrm{One}\ \sigma_2)\ \mathrm{false}\ v_2$$

Lemmas 6.3.7 to 6.3.9 correspond to the introduction rules for disjunction. If A executes a cut, one *cannot* obtain $A \vee B$ from B: the execution of A, whether it ultimately succeeds or fails, makes the success of B irrelevant (the cut discards it). In the first lemma, B is forced to be KO; in the second one, the failure of A absorbs B. The last lemma is connected to the depth-first exploration of the tree: proving a disjunction from its right-hand side can only occur after the left-hand side has been disproved. Depicted as rules:

$$\frac{A \quad (A\ \text{cuts})}{A \vee B}\ (\vee_{il}) \qquad \frac{\neg A \quad (A\ \text{cuts})}{\neg(A \vee B)}\ (\vee_{ir1})$$

$$\frac{\neg A \qquad B \quad (A\ \text{does not cut})}{A \vee B}\ (\vee_{ir2})$$

Lemmas 6.3.10 and 6.3.11 are elimination rules for disjunction.

Lemma 6.3.10 (or_elim1)**.** $\forall\ p\ v\ v'\ \sigma\ \sigma'\ \sigma_1\ A\ A'\ B\ B'$,

$$\mathrm{runT}\ p\ v\ \sigma\ (A \vee B_{\sigma_1})\ (\mathrm{Many}\ \sigma'\ (A' \vee B'_{\sigma_1}))\ \mathrm{false}\ v' \to \exists\ b,\ \mathrm{runT}\ p\ v\ \sigma\ A\ (\mathrm{Many}\ \sigma'\ A')\ b\ v' \wedge B' = \mathrm{if}\ b\ \mathrm{then\ KO\ else}\ B$$

Lemma 6.3.11 (or_elim2)**.** $\forall\ p\ v\ v'\ \sigma\ \sigma'\ \sigma_1\ A\ B\ B'$,

$$\mathrm{runT}\ p\ v\ \sigma\ (A \vee B_{\sigma_1})\ (\mathrm{Many}\ \sigma'\ (\square\ \vee B'_{\sigma_1}))\ \mathrm{false}\ v' \to (\mathrm{runT}\ p\ v\ \sigma\ A\ (\mathrm{One}\ \sigma')\ \mathrm{false}\ v') \vee (\exists\ v_2\ b,\ \mathrm{runT}\ p\ v\ \sigma\ A\ \mathrm{Zero}\ \mathrm{false}\ v_2 \wedge \mathrm{runT}\ p\ v_2\ \sigma_1\ B\ (\mathrm{Many}\ \sigma'\ B')\ b\ v')$$

From $A \vee B$, when A is not inert (i.e. not already explored), we cannot deduce A or B independently. Instead, we can only derive a weakened conclusion of the form $\top \vee B'$, where B' provides no information in the case where the exploration of A performs a cut. This yields an elimination rule that is stronger than the usual one. If A does not cut, the elimination rule has the usual two premises, together with the additional constraint that whenever B holds, A must have failed. This again reflects the depth-first traversal of the proof tree.

$$\dfrac{A \vee B \qquad \overset{[A]}{C} \qquad (A \text{ cuts})}{C}\,(\vee_{e1})$$

$$\dfrac{A \vee B \qquad \overset{[A]}{C} \qquad \overset{[\neg A, B]}{C} \qquad (A \text{ does not cut})}{\mathrm{C}}\,(\vee_{e2})$$

It is worth recalling that the "A cuts" side-condition in the rules above is precisely the boolean result returned by runT. Without this information, it is impossible to formulate these rules: it is not merely a matter of whether A contains a syntactic occurrence of cut, but whether that cut is actually executed during the (dis)proof of A (in rules $(\vee_{ir1})$ and $(\vee_{ir2})$). Indeed, an atom preceding the cut may fail, so the cut is never reached.

As anticipated in the introduction, these lemmas contradict the commutativity of disjunction.

6.4 Equivalence of the two semantics

In order to relate the two semantics we have to relate the computations states, i.e., the And-Or tree and the stack of alternatives. We define a translation from trees to stacks, called tree_to_stack, but we do not explicitly provide the converse translation, an economy allowed by the proof technique we employ, inspired by [Ber98]. Before describing the translation of trees to stacks (in section 6.4.2) we need to define the notion of *valid tree*. The tree datatype indeed contains trees that cannot be generated by runT via step or prune, for example all the trees that do not correspond to a depth-first left-to-right tree construction strategy.

6.4.1 Valid trees

The class of valid trees is captured by the valid_tree function shown in Figure 6.8. While all base cases of tree are valid, we need to analyze the Or and And constructors more carefully.

For the Or constructor, we distinguish two cases depending on whether the left-hand side is present. If it is not, then the right-hand side must be a valid tree. Otherwise, the left-hand side must be a valid tree, and the right-hand side must either be the KO tree

```
Fixpoint valid_tree A :=
  match A with
  | Unexplored _ | OK | KO ⇒ ⊤
  | Or □ _ B ⇒ valid_tree B
  | Or ·A _ B ⇒ valid_tree A && ((B == KO) || base_or B)
  | And A B0 B ⇒ valid_tree A && if success A then valid_tree B
                                 else B == big_and B0
  end.
```

Figure 6.8: Valid tree

or be unexplored. The former case occurs when the right-hand side is cut away by the evaluation of a superficial cut in the left-hand side. In the latter case we use the *base_or* predicate to check that B is the right-skewed tree formed by a disjunction of conjunctions, generated after a successful backchain for a call.

For the And constructor, the left hand side is required to be a valid tree. The shape of the right hand side depends on whether the left hand side is a success. If it is not successful, then the right hand side must not be explored, i.e. B must be the right-skewed tree containing conjunctions of premise atoms. This is enforced using the *big_and* procedure, which generates a tree from the reset point B_0 (the same procedure used to transform each alternative output by backchain into the And-layer of the resulting tree). When the left hand side is successful, then the right hand side may have been explored, and consequently must be a valid tree.

We point out the two main invariants stating that valid trees are preserved by calls to step and prune.

Lemma 6.4.1 (valid_tree_step)**.**

$$\forall\ \sigma\ v\ A\ r,\ \text{valid\_tree}\ A \to \text{step}\ u\ p\ v\ \sigma\ A = r \to \text{valid\_tree}\ (\text{snd}\ r)$$

Lemma 6.4.2 (valid_tree_prune)**.**

$$\forall\ A\ B\ b,\ \text{valid\_tree}\ A \to \text{prune}\ b\ A = \text{Some}\ B \to \text{valid\_tree}\ B$$

Theorem 6.4.3 (valid_tree_run)**.** $\forall\ b\ \sigma\ \sigma'\ v\ v'\ A\ B$,

$$\text{valid\_tree}\ A \to \text{runT}\ u\ p\ v\ \sigma\ A\ (\text{Many}\ \sigma'\ B)\ b\ v' \to \text{valid\_tree}\ B$$

6.4.2 From trees to stacks

The tree_to_stack recursive function translates a tree to a stack.

$$\text{tree\_to\_stack} : \mathcal{T} \to \Sigma \to \text{alts} \to \text{alts}$$

The function tree_to_stack takes as input the tree t_0, a substitution σ for the input tree, and an auxiliary list of choice points *cp*. These choice points represent alternatives that appear at the same level as the current tree, such as, for example, the right siblings of an Or node.

The OK node represents one successful alternative; therefore it is translated into a singleton list whose only element has an empty list of goals. The KO node represents failure, hence it is mapped to the empty list of alternatives. Finally, atoms are mapped to a singleton list containing one alternative whose only goal is that atom, with an empty cut-to list. At this stage, the cut-to list cannot point to *cp*: atoms are not right-uncle subtrees, whereas *cp* represents alternatives that will actually be discarded if the atom turns out to be a cut. The correct cut-to list is therefore determined later, once the trees corresponding to sibling subtrees have been inspected.

The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for B. The alternatives of B are valid choice points for A and should be passed to the translation of A. The two lists of alternatives can then be concatenated. Moreover, every atom occurring one level below *cp* (i.e. in A or B) should add *cp* as its cut-to alternatives.

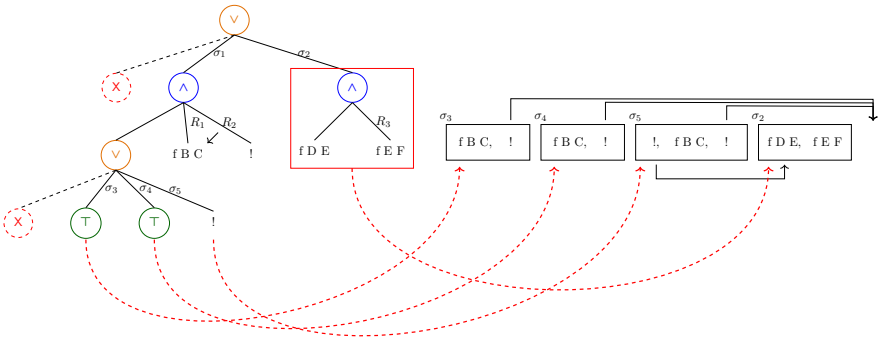


Figure 6.9: Example of tree_to_stack execution

The translation of $A \wedge_{B_0} B$ starts with the translation of A. If the translation of A is an empty list we return the empty list of alternatives without even touching B nor B_0, since an empty translation for A indicates failure and this in turn implies the failure of the entire conjunction. When the translation of A is a list $(\sigma_0, g) :: a$, the B node represents the continuation of the first alternative g, whereas B_0 is the continuation for each alternative in a.

6.4.2.1 Example of tree_to_stack execution

The translations of the trees in figure 6.3 are shown in figure 2.6. The letters in the figures relate each tree to the corresponding stack. For example, 6.3b is translated into 2.6b. We also note that 6.3d.1 and 6.3d.2 are both translated into 2.6d, showing that tree_to_stack is not injective: different trees can map to the same stack. Consequently, there are transitions in runT that are not visible in runS. As an example of a call to tree_to_stack, we take the tree and the stack in figure 6.9, which are respectively taken from figures 2.6c and 6.3c. The red arrows point from the head of an alternative in the tree to the head of the corresponding cell in the stack. We focus on the deepest Or node in the tree. This subtree is a disjunction with four children. The first is ignored, since it is None. The success node below σ_3 has `f B C` and the cut operator as continuation, corresponding to the first cell in the stack. We note that the cut operator points outside the stack, since the scope of this cut eliminates its right uncle. The success node below σ_4 has R_1 as continuation, where R_1 is the list of goals `f B C, !`; this is translated into the second cell of the stack. Finally, the cut operator under σ_5 also has R_1 as continuation. It corresponds to the third alternative. We can see that the first cut points to the translation of the subtree under σ_2, since it is one Or-layer above its right uncles.

6.4.3 Equivalence proof

The tree-based semantics exposes more information than needed, hence we hide it behind existentials.

Definition 6.4.4 (runT'). $\lambda\ p\ v\ \sigma\ t\ r.\ \exists\ b\ v',\ \mathrm{runT}\ p\ v\ \sigma\ t\ r\ b\ v'$

From figures 2.6 and 6.3, we observe that, upon backtracking, runT may perform more steps than runS before returning a solution or a failure. These silent transitions must be skipped when proving the equivalence between the two semantics. Therefore, we prove the following lemma.

Lemma 6.4.5 (pruneF_tree_to_stack)**.**

$$\forall\ t\ t'\ b,\ \text{valid\_tree}\ t \to \text{failed}\ t \to \text{prune}\ b\ t = \text{Some}\ t' \to \\ \forall\ l\ \sigma,\ \text{tree\_to\_stack}\ t\ \sigma\ l = \text{tree\_to\_stack}\ t'\ \sigma\ l$$

Proof. By induction on the tree t. □

The first direction of the equivalence states that the execution of a valid tree is matched by the execution of the stack obtained by translating the tree. Moreover, the unexplored alternatives returned as a tree correspond to those returned as a stack.

Theorem 6.4.6 (runT_to_runS)**.**

$$\forall\ p\ t\ \sigma\ r,\ \text{let}\ v := \text{vars}\ t\ \cup\ \text{vars}\ \sigma\ \text{in valid\_tree}\ t \to \text{runT}'\ p\ v\ \sigma\ t\ r \to \\ \begin{cases} \text{runS}\ p\ v\ (\text{tree\_to\_stack}\ t\ \sigma\ [\ ])\ \text{None} & \text{if } r = \text{Zero} \\ \text{runS}\ p\ v\ (\text{tree\_to\_stack}\ t\ \sigma\ [\ ])\ (\text{Some}\ (\sigma', [\ ])) & \text{if } r = \text{One}\ \sigma' \\ \text{runS}\ p\ v\ (\text{tree\_to\_stack}\ t\ \sigma\ [\ ])\ (\text{Some}\ (\sigma',\ \text{tree\_to\_stack}\ t'\ \sigma\ [\ ])) & \text{if } r = \text{Many}\ \sigma'\ t' \end{cases}$$

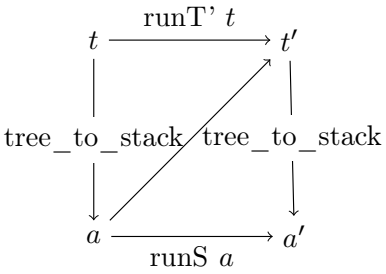
Proof. We proceed by induction on the execution of runT'. There are five cases to consider. The first (similar) cases arises from StopOT and StopMT; they deal with successful trees. A successful tree corresponds to a stack that start with an empty list, in both cases we conclude using StopS. The case for StepT requires treating two subcases: step evaluates either a cut or a call. Accordingly, we apply CutS or CallS, followed by the induction hypothesis. The case for BackT is silent: it represents the non-injective behavior of tree_to_stack; however, thanks to lemma 6.4.5 and the induction hypothesis, the conclusion is proved. The last case arises from the derivation FailT. It is easily proved using FailS and the fact that if prune returns None when trying to erase failure in t, then tree_to_stack applied to t returns the empty list. □

The converse direction is stated following [Ber98]: instead of using a function to translate a stack into a tree it states that any tree that translates to the given stack has an equivalent execution, i.e. gives the same solution and returns an unexplored tree that translates to the unexplored stack.

Theorem 6.4.7 (runS_to_runT)**.**

$$\forall\ p\ v\ a\ r,\ \text{runS}\ p\ v\ a\ r \to \forall\ \sigma\ t,\ \text{valid\_tree}\ t \to \text{tree\_to\_stack}\ t\ \sigma\ [\ ] = a \to \\ \begin{cases} \text{runT}'\ p\ v\ \sigma\ t\ \text{Zero} & \text{if } r = \text{None} \\ \text{runT}'\ p\ v\ \sigma\ t\ (\text{One}\ \sigma') & \text{if } r = \text{Some}\ (\sigma',\ [\ ]) \\ \exists\ t',\ \text{runT}'\ p\ v\ \sigma\ t\ (\text{Many}\ \sigma'\ t') \wedge \text{tree\_to\_stack}\ t'\ \sigma\ [\ ] = a' & \text{if } r = \text{Some}\ (\sigma',\ a') \end{cases}$$

Proof. The proof is based on the idea explained in [Ber98, Section 3.5.1] and depicted in the figure on the right: we have a tree t whose translation is a, we proceed by induction on the execution of runS. This gives us not only the output a' but that hypothesis that it exists a tree t' such that its translation to a stack is a'. □



Stating the converse direction of theorem 6.4.6 in a symmetric way would require a function stack_to_tree transforming a stack a into a tree t. Writing this function is far from trivial, since the structure of the tree is lost by $\mathcal{B}_S$ and runS that intuitively keep the state as a big disjunction of conjunctions by distributing conjunctions over disjunctions. Moreover, one would need to define a notion of valid stack that is equally intricate.

6.5 Determinacy analysis

The formalization of the two semantics in the Rocq proof assistant is an essential ingredient in demonstrating the correctness of the determinacy checker for the first-order fragment of the Elpi language. This theorem is proved in the tree semantics and can therefore be transferred to the stack semantics using theorem 6.4.6.

As explained in detail in chapter 5, a predicate behaves deterministically if two conditions are satisfied. *Mutual exclusion* guarantees that at most one rule can be successfully applied to any given query, whereas *rule checking* ensures that each rule does not create choice points, for example by calling non-deterministic predicates. In this chapter, we take both aspects into account.

Our mechanization implements the unification algorithm of Martelli and Montanari [MM82], which we prove to be terminating using a standard lexicographic ordering, as well as correct and complete.

In the following definitions and lemmas, we use the notation $x \cap y = \varnothing$ to denote that the sets x and y are disjoint. When the arguments are terms, as in $t_1 \cap t_2 = \varnothing$, we mean that the two terms do not share any variables.

The first notion we need to introduce is the application of an acyclic substitution:

Definition 6.5.1 (deref)**.**

$$\begin{aligned} \sigma\ (\text{Symb}\ n) &= \text{Symb}\ n & \qquad \sigma\ (\text{Var}\ n) &= t & \text{if } (n,t) \in \sigma \\ \sigma\ (\text{Pred}\ n) &= \text{Pred}\ n & \qquad \sigma\ (\text{Var}\ n) &= \text{Var}\ n & \text{otherwise} \\ \sigma\ (\text{App}\ t_1\ t_2) &= \text{App}\ (\sigma\ t_1)\ (\sigma\ t_2) \end{aligned}$$

Definition 6.5.2 (acyclic)**.** $\lambda\sigma.\text{dom}\ \sigma \cap \text{codom}\ \sigma = \varnothing$

Note that this definition of acyclicity entails that the substitution is necessarily self applied, i.e. $\sigma := \{X \mapsto f\ Y, Y \mapsto a\}$ is not acyclic, whereas the equivalent $\sigma' := \{X \mapsto f\ a, Y \mapsto a\}$ is. The algorithm by Martelli and Montanari generates substitutions that are acyclic in this sense.

Lemma 6.5.3 (unify_correct)**.**

$$\forall\ t_1\ t_2\ s\ s',\ \text{acyclic\_sigma}\ s \to \text{unify}\ t_1\ t_2\ s = \text{Some}\ s' \to s'\ t_1 = s'\ t_2$$

Lemma 6.5.4 (unify_complete)**.** $\forall\ t_1\ t_2\ s,\ \text{acyclic\_sigma}\ s \rightarrow$

$$(\exists\ s',\ \text{acyclic\_sigma}\ s' \wedge s'\ (s\ t_1) = s'\ (s\ t_2)) \rightarrow \exists\ s'',\ \text{unify}\ t_1\ t_2\ s = \text{Some}\ s''$$

The Elpi language adopts a restricted unification routine for input arguments, called matching [AK91, Zho12]. Similarly to unify, matching takes the query term t_1 and the pattern t_2 and returns an optional substitution σ' that does not assign any variable in the query arguments that are in input position.

To address this need, we extend the Martelli-Montanari unification procedure so that it takes a set of frozen variables, i.e. variables that cannot be assigned. The function matching has the following property:

Lemma 6.5.5 (matching_disj)**.**

$$\forall t_1\ t_2\ \sigma\ \sigma', \text{acyclic}\ \sigma \rightarrow \text{matching}\ t_1\ t_2\ \sigma = \text{Some}\ \sigma' \rightarrow \sigma'\ t_1 = \sigma\ t_2$$

In particular, we can see how the returned substitution σ', differently from lemma 6.5.3, does not affect variables in $\sigma\ t_2$.

The backchain function uses matching or unify on the arguments q depending on whether the argument is in *input* or *output* mode.

For the static checker, and in particular in order to validate rule mutual exclusion, we need to ensure that no two rules can be used on the same query in the absence of cut. We exploit the behavior of the matching over input arguments to guarantee this property: mutual exclusion ensures that every pair of rules for a deterministic predicate has an input term that does not unify.

An important property we derive from the matching and unification procedures is the following. Given three pairwise variable-disjoint terms h_1, h_2, and q, representing two rule heads in the database and a query, if both heads match the query q, then the two heads must unify.

Lemma 6.5.6 (matching_unify_transP)**.**

$$\forall\ h_1\ h_2\ q.\ h_1 \cap h_2 = \varnothing \rightarrow h_1 \cap q = \varnothing \rightarrow h_2 \cap q = \varnothing \rightarrow$$
$$\text{matching}\ h_1\ q\ \varnothing \rightarrow \text{matching}\ h_2\ q\ \varnothing \rightarrow \text{unify}\ h_1\ h_2\ \varnothing$$

Proof. By lemma 6.5.5, we now that the matching of h_1 (resp. h_2) and q gives a substitution σ_1 (resp. σ_2) where the variables in q are not assigned, i.e. its domain only contains variables in h_1 (resp h_2). Using lemma 6.5.4 we need to prove that there exists a substitution σ' such that $\sigma'\ h_1 = \sigma'\ h_2$. This substitution is the composition of σ_1 and σ_2, that is $\sigma' = \lambda x.\sigma_1(\sigma_2\ x)$. Since h_1 and h_2 are disjoint, also are the domain of σ_1 and σ_2, therefore $\sigma' h_1 = \sigma_2(\sigma_1\ h_1)$ and $\sigma' h_2 = \sigma_2(\sigma_1\ h_2)$. We can now conclude the proof: $\sigma'\ h_1 = \sigma_2(\sigma_1\ h_1) = \sigma_2\ q = q = \sigma_2\ h_2 = \sigma' h_2$. □

Finally, we also need to prove that the refreshing of variables performed at runtime does not affect unification. In particular, a refreshing renaming f for a term t is a function from variables to variables such that: (i) f is defined on all variables in t, i.e. every variable is renamed; and (ii) f is injective, i.e. it does not identify distinct variables. Refreshing therefore produces α-equivalent terms.

Definition 6.5.7 ($=_\alpha$)**.** $\lambda t_1\ t_2.\exists(r : \text{renaming\_for}\ t_2), t_1 = \text{rename}\ r\ t_2$

Lemma 6.5.8 (unif_ren_ac)**.** $\forall\ t_1\ t_2\ t_1'\ t_2', t_1 =_\alpha t_1' \wedge t_2 =_\alpha t_2' \rightarrow$

$$t_1 \cap t_2 = \varnothing \wedge t_1' \cap t_2' = \varnothing \rightarrow \text{unify}\ t_1\ t_2\ \varepsilon \rightarrow \text{unify}\ t_1'\ t_2'\ \varepsilon$$

In figure 2.5, the rules of the program are equipped with three signatures, one per predicate. Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, their types, and whether the predicate is deterministic. They indicate that `f` and `g` are expected to be functions, as specified by the `func` keyword, whereas `r` is a relation, as indicated by the `pred` keyword. The `->` symbol separates inputs from outputs; therefore, all three predicates are expected to take an `int` as input and return an `int` as output.

The signatures of the program are stored in the `sig` field of the program and are encoded as described in [FT26]: we distinguish data from predicates, and predicates are classified as deterministic or non-deterministic.

A program execution behaves deterministically if, given a program, a substitution, and an input tree for the runT inductive, the execution either fails or succeeds with no alternatives left to explore.

Definition 6.5.9 (is_det)**.**

$$\lambda\ p\ \sigma\ v\ t.\ \forall\ r,\ \text{runT}'\ p\ v\ \sigma\ t\ r \rightarrow r = \text{Zero} \vee \exists\ \sigma,\ r = (\text{One}\ \sigma)$$

Definition 6.5.10 (tm_is_det)**.** *Given a program p and a term t, we say that t is deterministic if its head is a predicate f which is declared as deterministic in p.*

The theorem we want to prove is the following one:

Theorem 6.5.11 (det_check_call)**.** $\forall\ p\ \sigma\ t\ v,$

$$\text{check\_program}\ p \rightarrow \text{tm\_is\_det}\ p\ t \rightarrow \text{is\_det}\ p\ \sigma\ v\ (\text{Unexplored}\ (\text{call}\ t))$$

Here, check_program denotes the function that checks the program with respect to mutual exclusion and rule checking.

The proof of this lemma proceeds by induction on the program execution. However, without further work, the induction hypothesis is too weak since it talks about Unexplored trees (i.e. atoms), whereas we need it on any tree.

To overcome this issue, we introduce the notion of deterministic trees, written det_tree. Intuitively, a deterministic tree is a tree whose interpretation does not generate choice points. The definition is shown in figure 6.10.

For a call to a term t, the tree is deterministic whenever t calls a deterministic predicate. The recursive cases correspond to the And and Or nodes.

A conjunction behaves deterministically whenever its right-hand side is a deterministic tree. Two cases must then be considered.

If prune (success A) A = None, then A produces no choice points: it either always fails or contains a single successful execution path. Since det_tree $p\ B$ holds, the entire conjunction is therefore deterministic.

```
Fixpoint det_tree (p: program) A :=
  match A with
  | Unexplored cut => true
  | Unexplored (call t) => tm_is_det p t
  | KO | OK => true
  | And A B0 B =>
      det_tree p B &&
      ((prune (success A) A == None) ||
       ((det_tree p A || (has_cut B && has_cuts B0)) && det_trees p B0))
  | Or None _ B => det_tree p B
  | Or (Some A) _ B =>
      det_tree p A && if has_cut A then det_tree p B else (B == KO)
  end.
```

Figure 6.10: The *det_tree* predicate

Otherwise, the left-hand side may introduce backtracking, potentially resuming the reset point, which must itself behave deterministically. In this case, either A is deterministic, or both B and the reset point contain a cut that prevents the non-deterministic behaviour arising from the evaluation of A.

For Or trees, the interesting case is when the left-hand side contains an tree. In this case, A must be deterministic. If A contains a superficial cut, then B must also be deterministic. Indeed, if A succeeds, the cut discards B; otherwise, if A fails before reaching the cut, then no solution is produced by A and B is evaluated.

Finally, if A does not contain a cut, then B must be equal to KO, that is, a tree already discarded by a cut. This condition is essential. Otherwise, if A succeeds without a superficial cut and B is not KO, then B could still produce a successful branch, leading to non-deterministic behaviour.

The following lemmas show that deterministic trees are preserved by the operations step and prune.

Lemma 6.5.12 (det_tree_step)**.** $\forall$ p v σ_1 A r, check_program p $\rightarrow$ step u p v σ_1 A = r $\rightarrow$ det_tree pr (snd r)

Lemma 6.5.13 (det_tree_prune)**.**

$$\forall\ p\ A\ B\ b,\ \text{det\_tree}\ p\ A \rightarrow \text{prune}\ b\ A = \text{Some}\ B \rightarrow \text{det\_tree}\ p\ B$$

Finally, we can prove the following lemma that entails det_check_call:

Lemma 6.5.14 (det_check_tree)**.**

$$\forall\ \sigma\ v\ p\ t,\ \text{check\_program}\ p \rightarrow \text{det\_tree}\ p\ t \rightarrow \text{is\_det}\ p\ \sigma\ v\ t$$

6.6 Related work

The only mechanization of Prolog's semantics that we could find is Pusch's Isabelle formalization [Pus96], which follows Debray and Mishra [DM88, Sec. 4.3]. That work

starts from this semantics and refines it by introducing optimizations commonly found in the Warren abstract machine [War83, AK91]. Unlike our case study, it does not use the semantics to prove properties of program execution; consequently, it does not encounter the difficulty described in section 6.2, which led us to develop a semantics in which the effect of cut is more explicit. In this sense, the two approaches are complementary: taken together, they suggest that the tree-based semantics is equivalent to an optimized stack-based one.

Another relevant work is due to Kriener and King [KK11], but we could not find their Rocq source code. Their semantics is denotational and cannot easily account for all the features covered in [FT26], although it would have been sufficient for the first-order fragment considered in this chapter.

Hanus and Prott [HP25] formalize a determinacy checker for the logic-functional Curry language. In this language choice points are explicitly denoted by the `?` node in the syntax of expressions. In this setting the mutual exclusion of rules does not need to be ensured resorting to unification: the overlap of rules is explicit in the syntax. Similarly the operator to absorb choice points, `allValues`, explicitly receives the expression containing them as an argument making the scope of this control operator trivial to analyze.

The proof technique we use to relate the two semantics is inspired by [Ber98], where Bertot mechanizes the correctness of a compiler. He relates the semantics of an imperative language to that of its compiled assembly code and proves their equivalence. His approach uses the compiler as the translation function from one language to the other, but does not introduce a decompiler, just as we do not define a function from stacks to trees. To show that the assembly code behaves like the imperative language, he proceeds by induction on the execution of the assembly program. We adopt the same strategy in the proof of theorem 6.4.7.

For the static analysis of determinacy, we use a semantics in which input arguments are *matched*, rather than unified, against rule heads. This choice agrees with the ELPI interpreter, which is in turn inspired by B-PROLOG's "matching-clauses" [AK91, Zho12]. Applying our results to a semantics based entirely on unification is possible by adding two extra assumptions to theorem 6.5.11. First, the initial query must have ground inputs. Second, the program must be well-moded. Well-moded predicates produce ground outputs when given ground inputs; hence, starting from an initial query with ground inputs, well-moded programs behave exactly as if input arguments were matched. In other words, lemma 6.5.5 could be reformulated using unification, at the cost of an additional assumption that the input arguments are ground; under that assumption, unification and matching coincide.

6.7 Conclusion

In this chapter, we presented a mechanized formalization of the first-order determinacy checker implemented in ELPI. The central challenge was to provide a mathematically precise account of determinacy in the presence of the cut operator, whose operational behavior directly affects the existence of alternative computations. To address this issue, we developed an axiom-free formalization in Rocq of both the stack-based semantics used

by the (first-order) Elpi interpreter and a novel And-Or tree semantics that makes the structure of the search space and the scope of cuts explicit.

The mechanization consists of approximately 2,500 lines of specifications and about 5,000 lines of ssreflect proofs based on the Mathematical Components library [**?**] and its finmap [mat26] extension. Roughly 30% of the code is dedicated to the tree semantics, in particular the proofs in section 6.3.4, 15% to the stack semantics, and 35% to the equivalence proof. The remaining 20% is devoted to the determinacy checker. The mechanization took us approximately 8 months. We believe that the ssreflect proof language, and the proof style it advocates, helped us carry out the many refactorings that the mechanization underwent. The finmap library provides comprehensive support for reasoning about finite maps and finite sets over a countable domain. In particular, it helped us express conditions such as the disjointness between sets of variables and the domain of a substitution, as well as define the union of disjoint substitutions.

To obtain an axiom-free mechanization, we had to provide an implementation of the first-order unification algorithm. It amounts to about 1,700 lines of ssreflect code, including 200 lines for the termination proof based on a lexicographic ordering and 800 lines for correctness and completeness. Because of the specific form of lemmas 6.5.5 and 6.5.8, which deals with matching and unification of terms with disjoint support, we did not need to prove that the algorithm produces most-general unifiers.

The And-Or tree semantics plays a crucial role in the formalization of the determinacy checker. By representing alternatives and subgoals as distinct layers of a tree, it becomes possible to reason locally about the pruning effects of cut and to characterize precisely when alternative computations remain available. This explicit representation enabled the development of a collection of structural lemmas describing the interaction between cut, conjunction, and disjunction, as well as the proof of equivalence between the tree-based semantics and the operational stack-based semantics.

Building on these foundations, we formalized the static determinacy analysis introduced in [FT26]. The analysis captures the mutual-exclusion and rule-checking conditions required for deterministic execution. The mechanization establishes that these syntactic conditions are sufficient to guarantee the absence of alternatives in the And-Or tree semantics. In other words, whenever determinacy checking succeeds, every call to a predicate declared deterministic produces at most one successful execution path.

Combining this result with the equivalence theorem between the two semantics yields the main correctness statement of the formalization: predicates accepted by the determinacy checker leave no observable choice points during execution in the Elpi interpreter. Consequently, the mechanized proof provides a formal guarantee for the determinacy analysis shipped with Elpi, connecting a practical static analysis tool with a fully verified semantic foundation. This work also lays the groundwork for future extensions toward the higher-order features of the language and the formal verification of additional static analyses.

Chapter 7

Conclusion and Perspectives

This thesis investigated the use of logic programming as a base for the implementation and analysis of elaboration mechanisms in Rocq. More specifically, we explored how an external logic-programming engine can be used not only as an implementation vehicle for components of an interactive theorem prover, but also as a framework in which such components can be studied, optimized, and formally reasoned about.

The main contributions of this thesis are threefold.

First, we developed an alternative type-class solver for Rocq based on the Elpi logic-programming language. We showed that classes and instances can be compiled into a logic-programming database and resolved using the existing Elpi execution engine. Experimental results demonstrate that the resulting solver achieves performance comparable to the native Rocq implementation, while requiring only a modest amount of compilation infrastructure. These results indicate that logic programming provides a practical and efficient foundation for implementing elaboration mechanisms traditionally embedded within the theorem prover itself.

Second, we addressed a fundamental mismatch between the unification mechanisms of Rocq and Elpi. Although both systems support higher-order unification and $\beta\eta$-conversion, differences in their internal representations lead to situations where terms that unify in Rocq fail to unify after translation into Elpi. To bridge this gap, we introduced a compilation technique based on links and delayed constraints. This approach allows difficult unification problems to be deferred and delegated back as constraints while preserving the advantages of the Elpi execution model, including mode-directed unification. The resulting infrastructure significantly extends the class of programs that can be handled by the solver and demonstrates how logic programming and a proof assistants can cooperate effectively despite differences in their underlying computational models.

Third, we developed a static determinacy analysis for Elpi capable of handling higher-order predicates, dynamic clauses, and the cut operator. The analysis provides guarantees that predicates declared deterministic do not introduce unnecessary choice points during execution, thereby improving the reliability and predictability of logic programs. To increase confidence in the correctness of this analysis, we carried out a mechanized verification effort in Rocq. The resulting formalization includes an operational semantics for a first-order fragment of Elpi, a formalization of first-order unification based on the Martelli-Montanari algorithm [MM82], and a machine-checked verification of the determinacy checker. To the best of our knowledge, this constitutes the first formal verification

effort for a Prolog-like language featuring both the cut operator and determinacy verification.

Taken together, these contributions illustrate two complementary roles of logic programming within theorem-proving environments. On the one hand, logic programming provides a concise and expressive framework for implementing elaboration mechanisms such as type-class resolution. On the other hand, it offers a setting in which these mechanisms can be analyzed, optimized, and formally verified. The integration of ROCQ and ELPI demonstrates that mature logic-programming technology can be effectively leveraged within proof assistants while maintaining the level of rigor expected of such systems.

More generally, this thesis suggests that logic programming should not be viewed merely as an implementation language for theorem-prover infrastructure. Rather, it provides a unifying framework in which elaboration mechanisms can be specified, experimented with, analyzed through static techniques, and ultimately subjected to machine-checked verification. We believe that this combination of expressiveness, extensibility, and formal rigor makes logic programming a promising foundation for future generations of proof-assistant infrastructure.

Several directions for future work naturally emerge from the results presented in this thesis. We discuss the most promising ones in the remainder of this chapter.

7.1 Perspective

7.1.1 Reusing links for other unification problems

Despite the improvements presented in chapter 4, unification remains one of the main limitations of our ELPI-based type-class solver. The techniques developed in that chapter considerably reduce unification failures, in particular it solves the unification issues coming from terms in the higher-order pattern fragment modulo $\beta\eta$-equivalence. However, when evaluating the solver on libraries beyond TLC and STDPP, in particular the IRIS [JKJ+18] ecosystem, we encountered a different class of unification problems arising from the interaction between type classes and canonical structures [MT13].

Canonical structures have so far been omitted from the presentation because they are not part of the type-class mechanism itself, although they are frequently used to implement forms of symbol overloading. According to the ROCQ reference manual, a canonical structure is *an instance of a record or structure type that can be used to solve unification problems involving a projection applied to an unknown structure instance and a value*. Consider the following example.

```
Record ofe := Ofe { car : Type; ... }.
Canonical Structure ofe_nat := Ofe nat ...
```

Through canonical structure resolution, the unification problem `car ?c = nat`, where `?c` is an object of type `ofe`, can be solved by consulting the canonical structure database and retrieving an instance of `ofe` whose carrier field is `nat`. In contrast to type-class search, canonical structure resolution is performed during unification and is largely driven by indexing on the projection being unified, making the process significantly more deterministic.

Canonical structures are extensively used in libraries such as Hierarchy Builder [CST20]. However, because their resolution is intertwined with unification, understanding why a particular instance is selected can be difficult in practice. In particular, users are typically provided with little information about the sequence of unification steps that led to a successful resolution. By contrast, type-class search follows a more explicit resolution procedure and offers detailed tracing facilities that allow users to inspect both successful and failed resolution attempts. This increased transparency comes at a cost, as type-class search is generally slower than canonical structure lookup.

The choice between canonical structures and type classes therefore involves a trade-off between efficiency and debuggability. Difficulties for our Elpi-based solver arise precisely when both mechanisms are used together.

For example, the Iris library contains type-class instances parameterized by projections of canonical structures.

```
Class C (t : Type) := ...
Instance c : ∀ (x : ofe), C (car x) := ...
```

In this example, the instance `c` of the class `C` depends on an instance of `ofe`. The Elpi compiler translates this declaration into the rule `tc-C {{car lp:X}} {{c lp:X}}`.

The resulting rule can only be applied when the argument of the class explicitly exposes the projection `car`. Consequently, our solver is unable to solve the goal `C nat`. The native Rocq type-class solver, however, succeeds on this goal because the unification problem `car ?c = nat` triggers canonical structure resolution, which retrieves the suitable `ofe` instance – `ofe_nat` in this case – and allows resolution to proceed.

Addressing this limitation would require extending the compiler so that occurrences of projections originating from canonical structures are detected and translated into suitable deferred constraints. The link-based infrastructure developed in this thesis appears applicable in this setting, as it already supports delegating difficult unification problems to the constraint solver. A more challenging aspect concerns how links are managed. In particular, we would need a compilation routine that translates declared records and canonical structures (similarly to type classes and instances) into the Elpi database and to investigate how their interaction works with the type-class solver.

An alternative approach would be to reformulate developments so as to rely exclusively on type classes, thereby eliminating this source of unification failures altogether. However, as observed in [GGMR09] and discussed in more detail by Jung,* replacing canonical structures with type classes may lead to exponential blow-ups in search *when using unbundled typeclasses to model algebraic hierarchies.* Consequently, while such a reformulation would simplify the interaction with our solver, it may also introduce significant performance regressions. This highlights a broader tension between expressiveness, predictability, and search efficiency that warrants further investigation.

7.1.2 Tabling

One of the major advantages of building elaboration mechanisms on top of Elpi is the ability to reuse techniques developed in the broader logic programming community. The

*https://www.ralfj.de/blog/2019/05/15/typeclasses-exponential-blowup.html

determinacy checker presented in this thesis illustrates this point: once implemented in the ELPI backend, it becomes available to every application built on top of the language, including the type-class solver developed in this work.

A particularly promising technique that has not yet been explored in the context of ROCQ-ELPI is tabling [TS86]. Tabling augments the standard resolution strategy of logic programming languages with a memoization mechanism that stores intermediate search results. Whenever the same goal is encountered again, the previously computed result can be reused rather than recomputed. This approach avoids redundant exploration of identical portions of the search space and can dramatically improve performance in applications that repeatedly encounter similar subgoals.

Tabling has already proved successful in theorem-proving environments. In particular, it plays an important role in the implementation of type-class resolution in LEAN [SUdM20], where large dependency graphs often contain multiple paths leading to the same instance or to the same failure. Without memoization, these paths may be explored repeatedly, resulting in significant computational overhead.

The type-class solver developed in this thesis provides a natural setting in which to investigate the impact of tabling on proof-assistant workloads. Because resolution is delegated to the ELPI engine, tabling could in principle be integrated without requiring substantial changes to the compilation of classes and instances. Such an extension would make it possible to experimentally evaluate the interaction between tabling, canonical structures, and the link-based compilation strategy introduced in this thesis.

More broadly, tabling may provide a practical way of mitigating the exponential search behavior discussed in the previous section. Understanding whether tabling can alleviate these performance issues in realistic ROCQ developments constitutes an interesting direction for future research.

7.1.3 Formalizing the full Elpi language

The mechanized development presented in this thesis establishes a first formal account of a substantial fragment of the ELPI language. Although the current formalization is restricted to first-order programs, it already provides a machine-checked semantics for a Prolog-like language with cut and supports the verification of non-trivial static analyses.

A natural continuation of this work is to extend the formalization so as to cover the higher-order features of ELPI. Such an extension would require formalizing the higher-order abstract syntax representation used by the language, together with the corresponding higher-order unification procedures. While technically challenging, this effort would significantly broaden the scope of properties that could be established about ELPI programs.

A complete formalization of ELPI would provide a trustworthy foundation for the verification of logic-programming tools and applications built on top of it. In particular, it would enable correctness proofs for static analyses, compiler transformations, and elaboration mechanisms implemented in the language. Since ELPI is increasingly used as a meta-programming framework for proof assistants, such guarantees would have practical consequences for the reliability of the systems that depend on it.

Beyond the specific analyses considered in this thesis, a formal account of the full ELPI language would constitute an important step toward a verified meta-programming framework for theorem provers.

Bibliography

[AK91] Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991.

[AP18] Andreas Abel and Brigitte Pientka. Extensions to miller's pattern unification for dependent types and records. Preprint, https://www.cs.mcgill.ca/~bpientka/papers/unif_miller60.pdf, 2018.

[Bar91] Henk Barendregt. Introduction to generalized type systems. *Journal of Functional Programming*, 1(2):125–154, 1991.

[BC04] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development. Coq'Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. Springer Verlag, 2004.

[BCC+23] Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional preprocessing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023.

[Ber98] Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998.

[BJZ+08] Jean-Philippe Bernardy, Patrik Jansson, Marcin Zalewski, Sibylle Schupp, and Andreas Priesnitz. A Comparison of c++ Concepts and Haskell Type Classes. In *Proceedings of the ACM SIGPLAN Workshop on Generic Programming*, WGP '08, page 37–48, New York, NY, USA, 2008. Association for Computing Machinery.

[BM26] Jasmin Blanchette and Assia Mahboubi, editors. *Proof Assistants and Their Applications in Mathematics and Computer Science*. Computer Science Foundations and Applied Logic. Springer, 2026.

[CCM24] Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof Transfer for Free, With or Without Univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.

[CH88] Thierry Coquand and Gérard Huet. The calculus of constructions. *Information and Computation*, 76(2):95–120, 1988.

[Cha21] Arthur Charguéraud. TLC: A library for classical coq. https://github.com/charguer/tlc, 2021. GitHub repository.

[Chl13] Adam Chlipala. *Certified Programming with Dependent Types: A Pragmatic Introduction to the Coq Proof Assistant*. The MIT Press, 2013.

[Chu32] Alonzo Church. A Set of Postulates for the Foundation of Logic. *Annals of Mathematics*, 33(2):346–366, 1932.

[Chu40] Alonzo Church. A Formulation of the Simple Theory of Types. *The Journal of Symbolic Logic*, 5(2):56–68, 1940.

[CST20] Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic Hierarchies Made Easy in Coq with Elpi. In Zena M. Ariola, editor, *Proceedings of FSCD*, volume 167 of *LIPIcs*, pages 34:1–34:21, 2020.

[DGSCT15] Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: Fast, Embeddable, λProlog Interpreter. In Martin Davis, Ansgar Fehnker, Annabelle McIver, and Andrei Voronkov, editors, *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015.

[DM88] Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988.

[dMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 Theorem Prover and Programming Language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, pages 625–635, Cham, 2021. Springer International Publishing.

[DW89] Saumya K. Debray and David S. Warren. Functional computations in logic programs. *ACM Trans. Program. Lang. Syst.*, 11(3):451–481, July 1989.

[Fel93] Amy Felty. Encoding the Calculus of Constructions in a Higher-Order Logic. In M. Vardi, editor, *lics93*, pages 233–244. IEEE, June 1993.

[Frü98] Thom Frühwirth. Theory and practice of constraint handling rules. *The Journal of Logic Programming*, 37(1):95–138, 1998.

[Frü15] Thom Frühwirth. Constraint Handling Rules - What Else? In Nick Bassiliades, Georg Gottlob, Fariba Sadri, Adrian Paschke, and Dumitru Roman, editors, *Rule Technologies: Foundations, Tools, and Applications*, pages 13–34, Cham, 2015. Springer International Publishing.

[FT23] Davide Fissore and Enrico Tassi. A new Type-Class Solver for Coq in Elpi. In *The Coq Workshop*, July 2023.

[FT24] Davide Fissore and Enrico Tassi. Higher-Order Unification for Free!: Reusing the Meta-Language Unification for the Object Language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024.

[FT26] Davide Fissore and Enrico Tassi. Determinacy Checking for Elpi: an Higher-Order Logic Programming Language with Cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.

[GAA+13] Georges Gonthier, Andrea Asperti, Jeremy Avigad, Yves Bertot, Cyril Cohen, François Garillot, Stéphane Le Roux, Assia Mahboubi, Russell O'Connor, Sidi Ould Biha, Ioana Pasca, Laurence Rideau, Alexey Solovyev, Enrico Tassi, and Laurent Théry. A Machine-Checked Proof of the Odd Order Theorem. In Sandrine Blazy, Christine Paulin-Mohring, and David Pichardie, editors, *Interactive Theorem Proving - 4th International Conference, ITP 2013, Rennes, France, July 22-26, 2013. Proceedings*, volume 7998 of *Lecture Notes in Computer Science*, pages 163–179. Springer, 2013.

[Gac08] Andrew Gacek. The Abella Interactive Theorem Prover (System Description). In *Proceedings of the 4th International Joint Conference on Automated Reasoning*, IJCAR '08, page 154–161, Berlin, Heidelberg, 2008. Springer-Verlag.

[Gal03] Jean Gallier. *Logic for Computer Science: Foundations of Automatic Theorem Proving*. Dover Publications, Mineola, NY, 2003.

[GGMR09] François Garillot, Georges Gonthier, Assia Mahboubi, and Laurence Rideau. Packaging Mathematical Structures. In Stefan Berghofer, Tobias Nipkow, Christian Urban, and Makarius Wenzel, editors, *Theorem Proving in Higher Order Logics*, pages 327–342, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.

[Gir86] Jean-Yves Girard. The system f of variable types, fifteen years later. *Theoretical Computer Science*, 45:159–192, 1986.

[GLT23] Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023.

[Gon23] Georges Gonthier. A Computer-Checked Proof of the Four Color Theorem. Technical report, Inria, March 2023.

[GSCT19] Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. *Mathematical Structures in Computer Science*, 29:1–26, 03 2019.

[HHPJW96] Cordelia V. Hall, Kevin Hammond, Simon L. Peyton Jones, and Philip L. Wadler. Type classes in haskell. *ACM Trans. Program. Lang. Syst.*, 18:109–138, March 1996.

[How80] William Alvin Howard. The Formulae-as-Types Notion of Construction. In Haskell Curry, Hindley B., Seldin J. Roger, and P. Jonathan, editors, *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus, and Formalism*. Academic Press, 1980.

[HP25] Michael Hanus and Kai-Oliver Prott. Determinism Types for Functional Logic Programming. In *Proceedings of PPDP*, pages 47–59, 2025.

[HSC96] Fergus Henderson, Zoltan Somogyi, and Thomas Conway. Determinism Analysis in the Mercury Compiler. In *Proceedings of Australian Computer Science Conference*, pages 337–346, 08 1996.

[Hue75] Gérard Huet. A unification algorithm for typed λ-calculus. *Theoretical Computer Science*, 1(1):27–57, 1975.

[JKJ+18] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018.

[KJJ+23] Robbert Krebbers, Jacques-Henri Jourdan, Ralf Jung, et al. std++: An extended standard library for coq. https://gitlab.mpi-sws.org/iris/stdpp, 2023. GitLab repository.

[KK11] Jael Kriener and Andy King. RedAlert: Determinacy inference for Prolog. *Theory and Practice of Logic Programming*, 11(4-5):537–553, 2011.

[KvdMT25] Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors, *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[LGBH05] P. López-García, F. Bueno, and M. Hermenegildo. Determinacy Analysis for Logic Programs Using Mode and Type Information. In Sandro Etalle, editor, *Logic Based Program Synthesis and Transformation*, pages 19–35, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[LK05] Lunjin Lu and Andy King. Determinacy inference for logic programs. *Lecture Notes in Computer Science*, 3444:108–123, 04 2005.

[mat26] math-comp. finmap — finite maps for mathcomp, 2026.

[McC92] William McCune. Experiments with discrimination-tree indexing and path indexing for term retrieval. *J. Autom. Reason.*, 9(2):147–167, October 1992.

[Mil91] Dale Miller. A Logic Programming Language with Lambda-Abstraction, Function Variables, and Simple Unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.

[Mil18] Dale Miller. Mechanized Metatheory Revisited. In *Journal of Automated Reasoning*, volume 63, pages 625–665. Springer, 2018.

[MM82] Alberto Martelli and Ugo Montanari. An efficient unification algorithm. *ACM Trans. Program. Lang. Syst.*, 4(2):258–282, April 1982.

[MN12] Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, New York, NY, USA, 2012.

[MP93] Spiro Michaylov and Frank Pfenning. Higher-Order Logic Programming as Constraint Logic Programming. In *Position Papers for the First Workshop on Principles and Practice of Constraint Programming*, pages 221–229, Newport, Rhode Island, April 1993. Brown University.

[MT13] Assia Mahboubi and Enrico Tassi. Canonical Structures for the Working Coq User. In Sandrine Blazy, Christine Paulin-Mohring, and David Pichardie, editors, *Interactive Theorem Proving*, pages 19–34, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[MT22] Assia Mahboubi and Enrico Tassi. *Mathematical Components*. Zenodo, September 2022.

[Nad01] Gopalan Nadathur. The Metalanguage λprolog and Its Implementation. In Herbert Kuchen and Kazunori Ueda, editors, *Functional and Logic Programming*, pages 1–20, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.

[Nak86] Katsuhiko Nakamura. Control of Logic Program Execution Based on the Functional Relations. In *Proceedings of ICLP*, page 505–512. Springer, 1986.

[NM88] Gopalan Nadathur and Dale Miller. An Overview of Lambda-Prolog. In *Proceedings of the Fifth International Conference and Symposium on Logic Programming*, pages 810–827, Seattle, WA, USA, 1988. MIT Press.

[OMO10] Bruno C.d.S. Oliveira, Adriaan Moors, and Martin Odersky. Type classes as objects and implicits. *SIGPLAN Not.*, 45(10):341–360, October 2010.

[PE88] Frank Pfenning and Conal Elliott. Higher-order Abstract Syntax. In *Proceedings of PLDI*, page 199–208. Association for Computing Machinery, 1988.

[Pus96] Cornelia Pusch. Verification of Compiler Correctness for the WAM. In Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors, *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.

[Roc] Rocq Development Team. Class. https://rocq-prover.org/doc/v9.1/refman/addendum/type-classes.html#coq:cmd.Class. Rocq Prover Reference Manual, version 9.1. Accessed: 2026-03-11.

[Sak22] Kazuhiko Sakaguchi. Reflexive Tactics for Algebra, Revisited. In June Andronick and Leonardo de Moura, editors, *13th International Conference on Interactive Theorem Proving (ITP 2022)*, volume 237 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 29:1–29:22, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[SHC96] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. The execution algorithm of mercury, an efficient purely declarative logic programming language. *The Journal of Logic Programming*, 29(1):17–64, 1996. High-Performance Implementations of Logic Programming Systems.

[SO08] Matthieu Sozeau and Nicolas Oury. First-Class Type Classes. In *Proceedings of TPHOLs*, volume 5170 of *LNCS*, pages 278–293. Springer, 2008.

[ST14] Matthieu Sozeau and Nicolas Tabareau. Universe Polymorphism in Coq. In Gerwin Klein and Ruben Gamboa, editors, *Interactive Theorem Proving*, pages 499–514, Cham, 2014. Springer International Publishing.

[Str00] Christopher Strachey. Fundamental concepts in programming languages. *Higher Order Symbol. Comput.*, 13(1–2):11–49, April 2000.

[SUdM20] Daniel Selsam, Sebastian Ullrich, and Leonardo de Moura. Tabled typeclass resolution. *CoRR*, abs/2001.04301, 2020.

[SWI26] SWI-Prolog Documentation. Predicate `!/0` — cut (built-in predicate), 2026. Accessed via SWI-Prolog PlDoc reference.

[Tas18] Enrico Tassi. Elpi: an Extension Language for Coq (Metaprogramming Coq in the Elpi λProlog dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January 2018.

[Tas19] Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of *LIPIcs*, pages 29:1–29:18, September 2019.

[Tas25] Enrico Tassi. Elpi: Rule-based extension language. https://github.com/gares/HDR/blob/main/hdr.pdf, 2025.

[TS86] Hisao Tamaki and Taisuke Sato. OLD Resolution with Tabulation. In Ehud Shapiro, editor, *Third International Conference on Logic Programming*, pages 84–98, Berlin, Heidelberg, 1986. Springer Berlin Heidelberg.

[vdM24] Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi. Master's thesis, Radboud University Nijmegen, 2024.

[Wad89] Philip Wadler. Theorems for Free! In *Proceedings of FPCA*, FPCA '89, page 347–359. Association for Computing Machinery, 1989.

[War83] David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309, SRI International, Artificial Intelligence Center, Computer Science and Technology Division, Menlo Park, CA, USA, October 1983.

[WB89] Philip Wadler and Stephen Blott. How to Make ad-hoc Polymorphism less ad-hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '89, page 60–76, New York, NY, USA, 1989. Association for Computing Machinery.

[Wen97] Markus Wenzel. Type classes and Overloading in Higher-Order Logic. In Elsa L. Gunter and Amy Felty, editors, *Theorem Proving in Higher Order Logics*, pages 307–322, Berlin, Heidelberg, 1997. Springer Berlin Heidelberg.

[Zho12] Neng-fa Zhou. The language features and architecture of b-prolog. *Theory Pract. Log. Program.*, 12(1–2):189–218, January 2012.

[ZS15] Beta Ziliani and Matthieu Sozeau. A Unification Algorithm for Coq featuring Universe Polymorphism and Overloading. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming*, ICFP 2015, page 179–191, New York, NY, USA, 2015. Association for Computing Machinery.

Appendix

Appendix A

Reference for Elpi Predicates and Built-ins

This appendix provides the implementations and signatures of the Elpi predicates and built-in procedures referenced throughout the code snippets in the manuscript.

```
% [rex.split Rex Subject Out] Out is obtained by splitting Subject at all
% occurrences of Rex. See also OCaml's Str.split
external func rex.split string, string -> list string.

func append list A, list A -> list A.
append [X|XS] L [X|L1] :- append XS L L1 .
append [] L L .

func map list A, (func A -> B) -> list B.
map [] _ [].
map [X|XS] F [Y|YS] :- F X Y, map XS F YS.

func map-filter list A, (func A -> B) -> list B.
map-filter [] _ [].
map-filter [X|XS] F [Y|YS] :- F X Y, !, map-filter XS F YS.
map-filter [_|XS] F YS :- map-filter XS F YS.

func rev list A -> list A.
rev L RL  :- rev.aux L []  RL.

pred mem list A -> A.
mem [X|_] X.
mem [_|L] X :- mem L X.

func neg bool -> bool.
neg tt ff.
neg ff tt.

func rev.aux list A, list A -> list A.
rev.aux [X|XS] ACC R :- rev.aux XS [X|ACC] R.
rev.aux [] L L.

% [coq.TC.class? GR] checks if GR is a class
external func coq.TC.class? gref.

pred get-tm-gref term -> gref.
```

```
get-tm-gref (prod _ _ T) G :- pi x\ get-tm-gref (T x) G.
get-tm-gref (global G) G.
get-tm-gref (app [T | _]) G :- get-tm-gref T G.

pred is-class? term ->.
is-class? A :- get-class? A _.

pred get-class? term -> gref.
get-class? T G:- get-tm-gref T G, coq.TC.class? G.

func mk-app term, list term -> term.
mk-app HD [] HD :- !.
mk-app (app L) Args R :- !, append L Args LArgs, R = app LArgs.
mk-app (fun _ _ F) [A|Args] R :- !, mk-app (F A) Args R.
mk-app (let _ _ A F) Args R :- !, mk-app (F A) Args R.
mk-app HD Args (app [HD|Args]).

func coq.safe-dest-app term -> term, list term.
coq.safe-dest-app (app [X|XS]) HD AllArgs :- !,
  coq.safe-dest-app X HD ARGS, append ARGS XS AllArgs.
coq.safe-dest-app X X [].
```

Programmation logique pour les preuves interactives : inférence des classes de type et analyse du déterminisme

Résumé

La résolution de classes de types est un mécanisme central de l'élaboration dans les assistants de preuve modernes, assurant l'inférence d'arguments implicites et la désambiguïsation des symboles surchargés. Sa proximité avec la programmation logique suggère des implémentations alternatives fondées sur l'exécution de programmes logiques.

Cette thèse propose une implémentation d'un solveur de classes de types pour Rocq basé sur ELPI, un interpréteur Lambda-Prolog emarqué. Nous introduisons une compilation des classes de types et instances vers une base de connaissances logique, permettant de déléguer la résolution à ELPI. Nous analysons l'expressivité de cette approche et ses performances en la comparant au solveur natif de Rocq.

Un défi majeur provient des divergences entre Rocq et ELPI en matière d'unification et de représentation des termes. Bien que les deux systèmes supportent l'unification d'ordre supérieur et la $\beta\eta$-conversion, leurs encodages diffèrent, ce qui peut empêcher la préservation de l'unification après traduction. Pour remédier à ce problème, nous développons une technique de compilation fondée sur des contraintes différées : les sous-termes problématiques sont remplacés par des liens enregistrant des contraintes qui seront vérifiées au cours de l'exécution, ce qui permet de retrouver le comportement d'unification attendu.

Nous étudions également la prédictibilité des programmes logiques au moyen d'une analyse de déterminisme pour ELPI. Celle-ci garantit qu'un prédicat donné produit au plus une solution, en supportant prédicats d'ordre supérieur, clauses dynamiques et cut. Cette analyse est particulièrement pertinente dans le contexte de la résolution de classes de types, où un comportement déterministe est généralement attendu. Enfin, nous formalisons la correction de cette analyse dans Rocq via un modèle d'ordre un de l'interpréteur ELPI et une preuve de sûreté du vérificateur statique.

Dans l'ensemble, ce travail met en évidence le rôle des techniques de programmation logique dans la mise en œuvre et l'analyse de mécanismes d'élaboration, éclairant les liens entre résolution de classes de types, programmation logique et vérification formelle.

Mots-clés : Programmation logique, Théorie des types, Elpi, Rocq, Classes de type, Analyse de déterminisme.